

Agentic SegFormer-B2 Framework for Landslide Segmentation using Multi-Temporal InSAR Data

Overview

This work presents an Agentic SegFormer-B2 framework for automatic landslide segmentation using multi-temporal InSAR imagery. The objective is to accurately identify landslide regions while reducing manual intervention during model optimization through an intelligent agent-assisted training strategy.

The proposed framework combines:

- SegFormer-B2 Transformer Architecture
 - Multi-Temporal InSAR Data
 - Phase Decomposition
 - Hybrid BCE + Dice Loss
 - Agentic Training Framework
 - Adaptive Learning Rate Control
 - Dynamic Dice Weight Adjustment
-

Dataset Preparation

The dataset consists of:

- Coherence Images
- Phase Images
- Ground Truth Segmentation Masks

The dataset is organized into three temporal baselines:

- 6 Days
- 12 Days
- 18 Days

Each baseline contains data from four geographic regions:

- EcrinPark
- OutsideTheBox
- QueyrasPark
- Vanoise

Phase Representation

To preserve the periodic nature of phase information, phase values were decomposed into:

- $\cos(\text{Phase})$
- $\sin(\text{Phase})$

The final three-channel input becomes:

[Coherence, $\cos(\text{Phase})$, $\sin(\text{Phase})$]

This representation improves learning stability and reduces phase discontinuity effects.

Data Augmentation

To improve generalization and reduce overfitting, the following augmentations were applied during training:

- Horizontal Flip
- Vertical Flip
- RandomRotate90
- Transpose

These transformations increase sample diversity and improve robustness to spatial variations.

Dataset Split

The complete dataset contains 13,230 samples.

Dataset	Ratio
Training	70%
Validation	15%
Testing	15%

Resulting samples:

Dataset	Samples
Training	9,261
Validation	1,984
Testing	1,985

SegFormer-B2 Architecture

The backbone model used is:

nvidia/segformer-b2-finetuned-ade-512-512

Architectural Modifications

The original SegFormer-B2 architecture was modified by introducing:

- Binary Segmentation Head
- Dropout Layer ($p = 0.10$)
- Bilinear Upsampling Layer
- Output Resolution = 100×100

Explicit Dropout Layer

A Dropout2D layer was added after decoder logits:

Decoder Logits $\rightarrow$ Dropout $\rightarrow$ Bilinear Upsampling $\rightarrow$ Final Prediction

This helps reduce overfitting and improves generalization performance.

Hybrid Loss Function

A hybrid loss strategy was adopted to improve segmentation quality.

Binary Cross Entropy Loss

Optimizes pixel-level classification accuracy.

Dice Loss

Optimizes overlap between predicted and ground-truth masks.

Final Loss Function

Hybrid Loss = $0.50 \times \text{BCE Loss}$

- $0.50 \times \text{Dice Loss}$

Benefits

- Better overlap optimization
 - Improved segmentation quality
 - Better handling of class imbalance
 - Stable convergence
-

Agentic Training Framework

An intelligent training agent was integrated into the optimization loop.

The agent continuously monitors:

- Validation Loss
- Validation Dice Score
- Validation IoU Score

Agent Decisions

Learning Rate Adaptation

When validation loss increases:

Learning Rate = Learning Rate $\times$ 0.8

Dynamic Dice Weight Adjustment

When IoU performance decreases:

Dice Weight $\pm$ 0.05

BCE Weight = 1 – Dice Weight

This shifts optimization toward region-level segmentation quality.

Dynamic Threshold Adjustment

When:

Recall < 0.60

Threshold = 0.45

This helps recover missed landslide pixels.

Optimization Strategy

Parameter	Value
Optimizer	AdamW
Initial Learning Rate	1e-4
Weight Decay	1e-4
Scheduler	ReduceLROnPlateau
Epochs	70
Early Stopping Patience	25
Gradient Clipping	1.0

Evaluation Metrics

The framework was evaluated using:

Accuracy

Measures overall pixel classification correctness.

Precision

Measures correctness of predicted landslide pixels.

Recall

Measures the ability to detect actual landslide regions.

Dice Score

Measures segmentation overlap quality.

IoU Score

Measures region-level agreement between prediction and ground truth masks.

Temporal Baseline Analysis

After training, an additional temporal baseline analysis was conducted using:

- 6-Day Baseline
- 12-Day Baseline
- 18-Day Baseline

For each region, identical samples were evaluated across different temporal baselines.

The following metrics were analyzed:

- Mean Coherence
- Phase Standard Deviation
- Dice Score
- IoU Score
- Ground Truth Area
- Predicted Area
- Area Difference

The objective was to investigate how temporal baselines influence segmentation performance.

Visual Analysis

To improve interpretability, qualitative analysis was performed on model outputs.

Each prediction includes:

- Input Coherence Image
- Ground Truth Mask
- Predicted Mask
- Probability Heatmap
- Overlay Visualization

This enables both visual and quantitative assessment of landslide behavior.

Results Summary

The proposed Agentic SegFormer-B2 framework achieved strong and stable performance for landslide segmentation using multi-temporal InSAR data.

Best Training Results

Metric	Value
Accuracy	98.60%
Precision	82.17%
Recall	80.56%
Dice Score	80.70%
IoU Score	68.41%
Validation Loss	0.1342

Best Validation Results

Metric	Value
Accuracy	98.60%
Precision	82.17%
Recall	80.56%
Dice Score	80.70%
IoU Score	68.41%
Validation Loss	0.1342

Key Findings

- SegFormer-B2 successfully learned landslide patterns from multi-temporal InSAR data.

- Phase decomposition improved representation of interferometric phase information.
 - Hybrid BCE + Dice Loss improved segmentation quality.
 - Agentic training reduced manual hyperparameter tuning.
 - Adaptive learning rate control improved training stability.
 - Dynamic Dice weighting enhanced overlap-based optimization.
 - Temporal baseline analysis revealed performance variations across 6-day, 12-day, and 18-day intervals.
 - Visual analysis improved model interpretability.
-

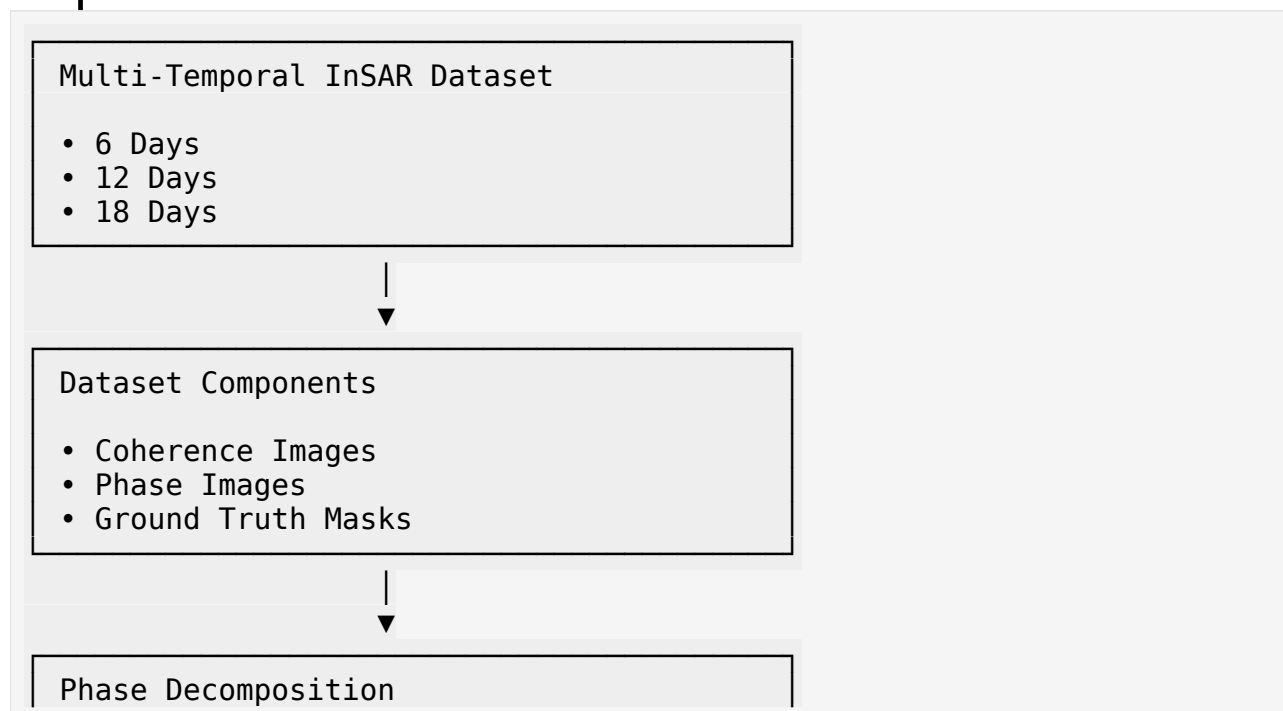
Conclusion

The proposed Agentic SegFormer-B2 framework successfully integrates transformer-based semantic segmentation with intelligent training adaptation for landslide detection using multi-temporal InSAR data.

By combining phase decomposition, hybrid loss optimization, adaptive thresholding, automated agent decisions, and temporal baseline analysis, the framework achieves robust segmentation performance while improving training efficiency and model interpretability.

The results demonstrate the potential of agent-assisted deep learning systems for remote sensing, geospatial monitoring, and automated landslide assessment.

Complete Agentic SegFormer-B2 Training Pipeline



Phase $\rightarrow$ Cos(Phase)
Phase $\rightarrow$ Sin(Phase)



Input Construction

Channel 1 $\rightarrow$ Coherence
Channel 2 $\rightarrow$ Cos(Phase)
Channel 3 $\rightarrow$ Sin(Phase)



Channel-wise Normalization

Mean = 0
Std = 1



Data Augmentation

- Horizontal Flip
- Vertical Flip
- RandomRotate90
- Transpose



SEGFORMER-B2 ARCHITECTURE



Transformer Encoder

- Hierarchical Features
- Global Context Learning
- Efficient Self-Attention



Multi-Scale Feature Extraction

Stage 1 Features
Stage 2 Features
Stage 3 Features
Stage 4 Features



SegFormer Decoder Head

Multi-Scale Feature Fusion
Binary Segmentation Output



Decoder Logits



Dropout2D ($p = 0.10$)

Regularization Layer
Overfitting Prevention



Bilinear Upsampling

Output Resolution: 100×100



Predicted Landslide Mask



HYBRID LOSS ENGINE



▼

BCE Loss

Pixel-Level Classification

Dice Loss

Region Overlap Optimization

Hybrid Loss

$0.50 \times \text{BCE}$

+

$0.50 \times \text{Dice}$

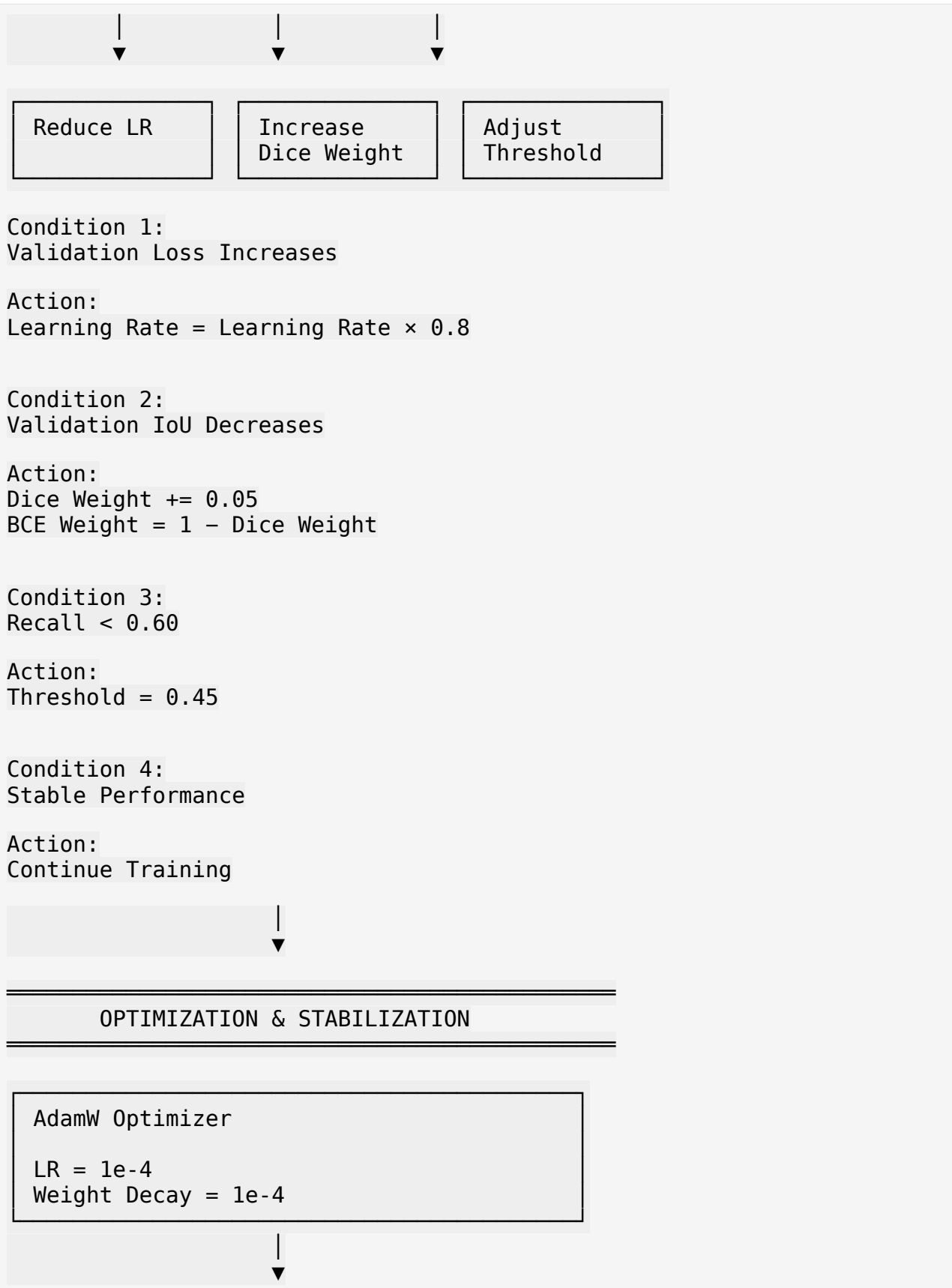
AGENTIC TRAINING LAYER

Agent Monitoring System

Monitors:

- Validation Loss
- Validation Dice Score
- Validation IoU Score

Agent Decision Engine



ReduceLRonPlateau Scheduler



Gradient Clipping

Max Norm = 1.0



Early Stopping

Patience = 25 Epochs



EVALUATION

Accuracy
Precision
Recall
Dice Score
IoU Score



Final Landslide Segmentation
Performance Evaluation



POST-TRAINING ANALYSIS

Temporal Baseline Analysis

- 6 Days
- 12 Days

- 18 Days



Region-wise Analysis

- EcrinPark
- OutsideTheBox
- QueyrasPark
- Vanoise



Visual Output Analysis

Input
Ground Truth
Prediction
Heatmap
Overlay

One-Line Workflow

Multi-Temporal InSAR Dataset



Phase Decomposition



3-Channel Input Construction



Normalization



Data Augmentation



SegFormer-B2 Encoder



Multi-Scale Feature Fusion



SegFormer Decoder



Dropout2D (0.10)



Bilinear Upsampling



Predicted Landslide Mask



Hybrid BCE + Dice Loss

↓
Agentic Monitoring Layer
↓
Adaptive LR / Dice Weight / Threshold
↓
AdamW Optimization
↓
Evaluation Metrics
↓
Final Landslide Segmentation
↓
Temporal & Visual Analysis

```
import os
import random

import cv2

import tifffile as tiff

import numpy as np

import matplotlib.pyplot as plt

import albumentations as A
from torch.utils.data import Dataset

from collections import deque

import torch

import torch.nn as nn

import torch.nn.functional as F
from torch.utils.data import random_split

from torch.utils.data import Dataset

from torch.utils.data import DataLoader

from transformers import (
    SegformerForSemanticSegmentation
)

from sklearn.metrics import (
    confusion_matrix,
```

```
ConfusionMatrixDisplay
)
device = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else
    "cpu"
)
print(
    "Device:",
    device
)
Device: cuda
root_dir = "/kaggle/input/datasets/rishikeshgopal45/landslide-insar-isslide/ReadyToBeUsedDataset"
print(
    "Dataset Path:",
    root_dir
)
Dataset Path: /kaggle/input/datasets/rishikeshgopal45/landslide-insar-isslide/ReadyToBeUsedDataset
train_transform = A.Compose([
    A.HorizontalFlip(
        p=0.5
    ),
    A.VerticalFlip(
        p=0.5
```

```

    ),
    A.RandomRotate90(
        p=1.0
    ),
    A.Transpose(
        p=0.5
    )
])
val_transform = None
test_transform = None
class SegFormerDataset(Dataset):
    def __init__(
        self,
        root_dir,
        regions=None,
        transform=None
    ):
        self.samples = []
        self.transform = transform
        temporal_folders = [
            "6days",
            "12days",
            "18days"
        ]
        for temporal in temporal_folders:
            temporal_path = os.path.join(
                root_dir,
                temporal
            )

```



```
for region in sorted(
    os.listdir(
        temporal_path
    )
):
    if (
        regions is not None
        and
        region not in regions
    ):
        continue
    region_path = os.path.join(
        temporal_path,
        region
    )
    folders = os.listdir(
        region_path
    )
    for folder in folders:
        if folder == "Segmentations":
            continue
        coh_path = os.path.join(
            region_path,
            folder,
            "coherence"
        )
```

```
phase_path = os.path.join(
    region_path,
    folder,
    "phase"
)
mask_path = os.path.join(
    region_path,
    "Segmentations",
    folder
)
if not (
    os.path.exists(coh_path)
    and
    os.path.exists(phase_path)
    and
    os.path.exists(mask_path)
):
    continue
files = sorted(
    os.listdir(
        coh_path
    )
)
for file in files:
    if file.endswith(".tif"):
        self.samples.append(
```

```

        (
            temporal,
            region,
            os.path.join(
                coh_path,
                file
            ),
            os.path.join(
                phase_path,
                file
            ),
            os.path.join(
                mask_path,
                file
            )
        )
    )

    print(
        "Samples:",
        len(self.samples)
    )

    def __len__(self):
        return len(
            self.samples
        )

    def __getitem__(

```

```
        self,
        idx
    ):
        temporal, region, coh_file, phase_file, mask_file =
self.samples[idx]
        coherence = tiff.imread(
            coh_file
        ).astype(
            np.float32
        )
        phase = tiff.imread(
            phase_file
        ).astype(
            np.float32
        )
        mask = tiff.imread(
            mask_file
        ).astype(
            np.float32
        )
        cos_phase = np.cos(
            phase
        )
        sin_phase = np.sin(
            phase
        )
```

```

image = np.stack(
    [
        coherence,
        cos_phase,
        sin_phase
    ],
    axis=-1
)
if self.transform is not None:
    transformed = self.transform(
        image=image,
        mask=mask
    )
    image = transformed["image"]
    mask = transformed["mask"]
image = np.transpose(
    image,
    (2, 0, 1)
)
image = image.astype(
    np.float32
)
for c in range(
    image.shape[0]
):
    image[c] = (

```

```

        image[c]
        -
        image[c].mean()
    ) / (
        image[c].std()
        +
        1e-8
    )
    mask = np.expand_dims(
        mask,
        axis=0
    )
    return (
        torch.tensor(
            image,
            dtype=torch.float32
        ),
        torch.tensor(
            mask,
            dtype=torch.float32
        )
    )
)

full_dataset = SegFormerDataset(
    root_dir,
    regions=None,
    transform=None
)

```

```
)  
total_size = len(  
    full_dataset  
)  
train_size = int(  
    0.70 * total_size  
)  
val_size = int(  
    0.15 * total_size  
)  
test_size = (  
    total_size  
    -  
    train_size  
    -  
    val_size  
)  
train_subset, val_subset, test_subset = random_split(  
    full_dataset,  
    [  
        train_size,  
        val_size,  
        test_size  
    ],  
    generator=torch.Generator().manual_seed(  
        42  
    )  
)
```

```
)  
train_dataset = torch.utils.data.Subset(  
    SegFormerDataset(  
        root_dir,  
        regions=None,  
        transform=train_transform  
    ),  
    train_subset.indices  
)  
val_dataset = torch.utils.data.Subset(  
    SegFormerDataset(  
        root_dir,  
        regions=None,  
        transform=None  
    ),  
    val_subset.indices  
)  
test_dataset = torch.utils.data.Subset(  
    SegFormerDataset(  
        root_dir,  
        regions=None,  
        transform=None  
    ),  
    test_subset.indices  
)  
print(  

```



```
    "Train Samples:",  
    len(train_dataset)  
)  
print(  
    "Validation Samples:",  
    len(val_dataset)  
)  
print(  
    "Test Samples:",  
    len(test_dataset)  
)
```

```
Samples: 13230  
Samples: 13230  
Samples: 13230  
Samples: 13230  
Train Samples: 9261  
Validation Samples: 1984  
Test Samples: 1985
```

```
train_loader = DataLoader(  
    train_dataset,  
    batch_size=8,  
    shuffle=True,  
    num_workers=2  
)  
  
val_loader = DataLoader(  
    val_dataset,  
    batch_size=8,  
    shuffle=False,  
    num_workers=2
```

```

)
test_loader = DataLoader(
    test_dataset,
    batch_size=8,
    shuffle=False,
    num_workers=2
)

print("\n")
print(
    "Train Samples:",
    len(train_dataset)
)

print(
    "Validation Samples:",
    len(val_dataset)
)

print(
    "Test Samples:",
    len(test_dataset)
)

Train Samples: 9261
Validation Samples: 1984
Test Samples: 1985

class SegFormerModel(nn.Module):
    def __init__(self):
        super().__init__()

```

```

        self.segformer =
SegformerForSemanticSegmentation.from_pretrained(
    "nvidia/segformer-b2-finetuned-ade-512-512",
    num_labels=1,
    ignore_mismatched_sizes=True
)

self.dropout = nn.Dropout2d(
    p=0.10
)

def forward(self, x):
    outputs = self.segformer(
        pixel_values=x
    )

    logits = outputs.logits

    logits = self.dropout(
        logits
    )

    logits = F.interpolate(
        logits,
        size=(100,100),
        mode="bilinear",
        align_corners=False
    )

    return logits

model = SegFormerModel().to(device)

print("Model Loaded Successfully")

{"model_id":"d96ce4dd68224591a33544fdc36a6b9d","version_major":2,"version_minor":0}

SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-
b2-finetuned-ade-512-512

```

Key	Status	
-----+-----		
+-----		

decode_head.classifier.weight	MISMATCH	Reinit due to size mismatch ckpt: torch.Size([150, 768, 1, 1]) vs model:torch.Size([1, 768, 1, 1])
decode_head.classifier.bias	MISMATCH	Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size([1])

Notes:

- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

Model Loaded Successfully

```

sample_batch = next(
    iter(train_loader)
)

images, masks = sample_batch
images = images.to(device)

with torch.no_grad():
    outputs = model(images)

print(
    "Input Shape :",
    images.shape
)

print(
    "Output Shape:",
    outputs.shape
)

Input Shape : torch.Size([8, 3, 100, 100])
Output Shape: torch.Size([8, 1, 100, 100])

class TrainingAgent:
    def __init__(self):
        self.best_iou = 0

```

```

        self.best_dice = 0

        self.history = deque(
            maxlen=5
        )
def analyze(
    self,
    val_loss,
    val_iou,
    val_dice,
    optimizer
):
    self.history.append(
        (
            val_loss,
            val_iou,
            val_dice
        )
    )
    if len(self.history) < 5:
        return "warmup"
    losses = [
        x[0]
        for x in self.history
    ]
    ious = [
        x[1]
        for x in self.history

```

```

    ]
    if losses[-1] > losses[0]:
        for param_group in optimizer.param_groups:
            param_group["lr"] *= 0.8
        return "reduce_lr"
    elif ious[-1] < ious[0]:
        return "increase_dice_weight"
    else:
        return "continue"
bce_weight = 0.50
dice_weight = 0.50

def dice_loss(
    pred,
    target
):
    pred = torch.sigmoid(
        pred
    )
    smooth = 1e-8
    intersection = (
        pred * target
    ).sum()
    union = (
        pred.sum()
        +
        target.sum()

```

```

    )
    dice = (
        2 * intersection + smooth
    ) / (
        union + smooth
    )
    return 1 - dice
bce_loss = nn.BCEWithLogitsLoss()
def hybrid_loss(
    pred,
    target
):
    bce = bce_loss(
        pred,
        target
    )
    dice = dice_loss(
        pred,
        target
    )
    total_loss = (
        bce_weight * bce
        +
        dice_weight * dice
    )
    return total_loss

```

```

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-4,
    weight_decay=1e-4
)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=5
)
agent = TrainingAgent()
print(
    "Optimizer, Scheduler and Agent Ready"
)
Optimizer, Scheduler and Agent Ready
def calculate_all_metrics(
    pred,
    target,
    threshold=0.5
):
    pred = torch.sigmoid(
        pred
    )
    pred = (
        pred > threshold
    ).float()
    TP = (

```



```

    pred * target
).sum()
TN = (
    (1-pred)
    *
    (1-target)
).sum()
FP = (
    pred
    *
    (1-target)
).sum()
FN = (
    (1-pred)
    *
    target
).sum()
accuracy = (
    TP + TN
) / (
    TP + TN + FP + FN + 1e-8
)
precision = TP / (
    TP + FP + 1e-8
)
recall = TP / (

```

```

        TP + FN + 1e-8
    )
    f1 = (
        2 * TP
    ) / (
        2 * TP + FP + FN + 1e-8
    )
    dice = f1
    iou = TP / (
        TP + FP + FN + 1e-8
    )
    return (
        accuracy.item(),
        precision.item(),
        recall.item(),
        f1.item(),
        dice.item(),
        iou.item()
    )
train_losses = []
train_acc = []
train_prec = []
train_rec = []
val_losses = []
val_acc = []
val_prec = []
val_rec = []
val_f1 = []
val_dice = []

```

```

val_iou = []
epochs = 70
best_loss = float("inf")
patience = 25
counter = 0
threshold = 0.5
for epoch in range(epochs):

    # =====
    # TRAINING
    # =====

    model.train()

    total_loss = 0

    correct = 0
    total = 0

    TP = 0
    FP = 0
    FN = 0

    print(f"\nEpoch {epoch+1}/{epochs}")

    for i, (images, masks) in enumerate(train_loader):

        images = images.to(device)

        masks = masks.to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = hybrid_loss(
            outputs,
            masks
        )

        loss.backward()

        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            1.0

```

```

    )
    optimizer.step()
    total_loss += loss.item()
    pred = torch.sigmoid(outputs)
    pred_bin = (
        pred > threshold
    ).float()
    correct += (
        pred_bin == masks
    ).sum().item()
    total += masks.numel()

    TP += (
        pred_bin * masks
    ).sum().item()

    FP += (
        pred_bin * (1 - masks)
    ).sum().item()

    FN += (
        (1 - pred_bin) * masks
    ).sum().item()

    progress = (
        (i + 1)
        /
        len(train_loader)
    ) * 100

    print(
        f"\rProgress: {progress:.2f}% | Loss: {loss.item():.4f}",
        end=""
    )

# =====
# TRAIN METRICS
# =====

acc = correct / total

precision = TP / (
    TP + FP + 1e-8
)

```

```

recall = TP / (
    TP + FN + 1e-8
)

train_losses.append(
    total_loss / len(train_loader)
)

train_acc.append(acc)

train_prec.append(
    precision
)

train_rec.append(
    recall
)

# =====
# VALIDATION
# =====

model.eval()

v_loss = 0

v_acc = 0
v_prec = 0
v_rec = 0

v_f1 = 0
v_dice = 0
v_iou = 0

count = 0

with torch.no_grad():
    for images, masks in val_loader:
        images = images.to(device)
        masks = masks.to(device)
        outputs = model(images)
        loss = hybrid_loss(
            outputs,
            masks
        )

```

```

        v_loss += loss.item()

        a, p, r, f1, d, i = calculate_all_metrics(
            outputs,
            masks,
            threshold
        )

        v_acc += a
        v_prec += p
        v_rec += r

        v_f1 += f1
        v_dice += d
        v_iou += i

        count += 1

    val_losses.append(
        v_loss / count
    )

    val_acc.append(
        v_acc / count
    )

    val_prec.append(
        v_prec / count
    )

    val_rec.append(
        v_rec / count
    )

    val_f1.append(
        v_f1 / count
    )

    val_dice.append(
        v_dice / count
    )

    val_iou.append(
        v_iou / count
    )

# =====
# AGENT DECISION
# =====

```

```

decision = agent.analyze(
    val_losses[-1],
    val_iou[-1],
    val_dice[-1],
    optimizer
)

if decision == "increase_dice_weight":
    dice_weight = min(
        dice_weight + 0.05,
        0.90
    )

    bce_weight = (
        1 - dice_weight
    )

if val_rec[-1] < 0.60:
    threshold = 0.45

scheduler.step(
    val_losses[-1]
)

# =====
# PRINT METRICS
# =====

print("\n")

print("=" * 70)

print(
    f"Epoch {epoch+1}/{epochs}"
)

print("=" * 70)

print(

```

```
        f"Train Loss      : {train_losses[-1]:.4f}"
    )

    print(
        f"Train Accuracy  : {train_acc[-1]:.4f}"
    )

    print(
        f"Train Precision : {train_prec[-1]:.4f}"
    )

    print(
        f"Train Recall    : {train_rec[-1]:.4f}"
    )

    print()

    print(
        f"Val Loss          : {val_losses[-1]:.4f}"
    )

    print(
        f"Val Accuracy     : {val_acc[-1]:.4f}"
    )

    print(
        f"Val Precision    : {val_prec[-1]:.4f}"
    )

    print(
        f"Val Recall       : {val_rec[-1]:.4f}"
    )

    print(
        f"Val F1           : {val_f1[-1]:.4f}"
    )

    print(
        f"Val Dice          : {val_dice[-1]:.4f}"
    )

    print(
        f"Val IoU           : {val_iou[-1]:.4f}"
    )

    print()

    print(
        f"Current LR          : {optimizer.param_groups[0]['lr']:.8f}"
    )
```



```

print(
    f"Threshold      : {threshold:.2f}"
)

print(
    f"BCE Weight      : {bce_weight:.2f}"
)

print(
    f"Dice Weight      : {dice_weight:.2f}"
)

print(
    f"Agent Decision   : {decision}"
)

print("=" * 70)

# =====
# MODEL SAVING
# =====

if val_losses[-1] < best_loss:
    best_loss = val_losses[-1]
    counter = 0
    torch.save(
        model.state_dict(),
        "segformer_best.pth"
    )
    print(
        "\nModel Saved!"
    )
else:
    counter += 1
    print(
        f"\nEarly Stop Counter: {counter}/{patience}"
    )
    if counter >= patience:
        print(

```

```

        "\nEarly Stopping Triggered"
    )

    break

# =====
# FINAL TRAINING SUMMARY
# =====

print("\n")

print("=" * 80)

print("FINAL TRAINING SUMMARY")

print("=" * 80)

print(
    f"Best Validation Loss : {best_loss:.4f}"
)

print()

print(
    f"Final Train Accuracy : {train_acc[-1]:.4f}"
)

print(
    f"Final Train Precision: {train_prec[-1]:.4f}"
)

print(
    f"Final Train Recall   : {train_rec[-1]:.4f}"
)

print()

print(
    f"Final Val Accuracy    : {val_acc[-1]:.4f}"
)

print(
    f"Final Val Precision   : {val_prec[-1]:.4f}"
)

print(
    f"Final Val Recall      : {val_rec[-1]:.4f}"
)

print(
    f"Final Val F1          : {val_f1[-1]:.4f}"

```

```

)
print(
    f"Final Val Dice      : {val_dice[-1]:.4f}"
)
print(
    f"Final Val IoU       : {val_iou[-1]:.4f}"
)
print("=" * 80)

```

Epoch 1/70
Progress: 100.00% | Loss: 0.2102

```

=====
Epoch 1/70
=====

```

```

Train Loss      : 0.4016
Train Accuracy  : 0.9622
Train Precision : 0.5383
Train Recall    : 0.5930

```

```

Val Loss        : 0.2407
Val Accuracy     : 0.9713
Val Precision    : 0.5946
Val Recall       : 0.7602
Val F1           : 0.6536
Val Dice         : 0.6536
Val IoU          : 0.4952

```

```

Current LR      : 0.00010000
Threshold       : 0.50
BCE Weight      : 0.50
Dice Weight     : 0.50
Agent Decision  : warmup

```

```

=====
Model Saved!

```

Epoch 2/70
Progress: 100.00% | Loss: 0.2495

```

=====
Epoch 2/70
=====

```

```

Train Loss      : 0.3391
Train Accuracy  : 0.9729
Train Precision : 0.6860

```

Train Recall : 0.6355

Val Loss : 0.2188

Val Accuracy : 0.9761

Val Precision : 0.6869

Val Recall : 0.6829

Val F1 : 0.6719

Val Dice : 0.6719

Val IoU : 0.5164

Current LR : 0.00010000

Threshold : 0.50

BCE Weight : 0.50

Dice Weight : 0.50

Agent Decision : warmup

Model Saved!

Epoch 3/70

Progress: 100.00% | Loss: 0.4443

Epoch 3/70

Train Loss : 0.3276

Train Accuracy : 0.9746

Train Precision : 0.7043

Train Recall : 0.6639

Val Loss : 0.2080

Val Accuracy : 0.9757

Val Precision : 0.6525

Val Recall : 0.7636

Val F1 : 0.6917

Val Dice : 0.6917

Val IoU : 0.5402

Current LR : 0.00010000

Threshold : 0.50

BCE Weight : 0.50

Dice Weight : 0.50

Agent Decision : warmup

Model Saved!

Epoch 4/70

Progress: 100.00% | Loss: 0.1316

=====
Epoch 4/70
=====

Train Loss : 0.3178
Train Accuracy : 0.9761
Train Precision : 0.7201
Train Recall : 0.6882

Val Loss : 0.1969
Val Accuracy : 0.9788
Val Precision : 0.7325
Val Recall : 0.6954
Val F1 : 0.7002
Val Dice : 0.7002
Val IoU : 0.5505

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : warmup
=====

Model Saved!

Epoch 5/70
Progress: 100.00% | Loss: 0.4552

=====
Epoch 5/70
=====

Train Loss : 0.3163
Train Accuracy : 0.9766
Train Precision : 0.7276
Train Recall : 0.6918

Val Loss : 0.2131
Val Accuracy : 0.9721
Val Precision : 0.5964
Val Recall : 0.8375
Val F1 : 0.6857
Val Dice : 0.6857
Val IoU : 0.5326

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue
=====

Early Stop Counter: 1/25

Epoch 6/70

Progress: 100.00% | Loss: 0.4413

=====
Epoch 6/70
=====

Train Loss : 0.3123
Train Accuracy : 0.9775
Train Precision : 0.7389
Train Recall : 0.7035

Val Loss : 0.1939
Val Accuracy : 0.9768
Val Precision : 0.6638
Val Recall : 0.7893
Val F1 : 0.7097
Val Dice : 0.7097
Val IoU : 0.5607

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue
=====

Model Saved!

Epoch 7/70

Progress: 100.00% | Loss: 0.2298

=====
Epoch 7/70
=====

Train Loss : 0.3027
Train Accuracy : 0.9784
Train Precision : 0.7506
Train Recall : 0.7130

Val Loss : 0.1844
Val Accuracy : 0.9800
Val Precision : 0.7402
Val Recall : 0.7256
Val F1 : 0.7223
Val Dice : 0.7223
Val IoU : 0.5757

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 8/70
Progress: 100.00% | Loss: 0.1088

=====

Epoch 8/70

=====

Train Loss : 0.2960
Train Accuracy : 0.9793
Train Precision : 0.7618
Train Recall : 0.7238

Val Loss : 0.1845
Val Accuracy : 0.9785
Val Precision : 0.6917
Val Recall : 0.7824
Val F1 : 0.7238
Val Dice : 0.7238
Val IoU : 0.5780

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 1/25

Epoch 9/70
Progress: 100.00% | Loss: 0.4171

=====

Epoch 9/70

=====

Train Loss : 0.2903
Train Accuracy : 0.9796
Train Precision : 0.7645
Train Recall : 0.7307

Val Loss : 0.1760
Val Accuracy : 0.9811

Val Precision : 0.7612
Val Recall : 0.7251
Val F1 : 0.7329
Val Dice : 0.7329
Val IoU : 0.5888

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Model Saved!

Epoch 10/70
Progress: 100.00% | Loss: 0.2603

=====
Epoch 10/70
=====

Train Loss : 0.2979
Train Accuracy : 0.9801
Train Precision : 0.7733
Train Recall : 0.7319

Val Loss : 0.1718
Val Accuracy : 0.9810
Val Precision : 0.7413
Val Recall : 0.7593
Val F1 : 0.7398
Val Dice : 0.7398
Val IoU : 0.5965

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Model Saved!

Epoch 11/70
Progress: 100.00% | Loss: 0.4236

=====
Epoch 11/70
=====

Train Loss : 0.2869

Train Accuracy : 0.9809
Train Precision : 0.7813
Train Recall : 0.7458

Val Loss : 0.1732
Val Accuracy : 0.9804
Val Precision : 0.7218
Val Recall : 0.7796
Val F1 : 0.7398
Val Dice : 0.7398
Val IoU : 0.5968

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Early Stop Counter: 1/25

Epoch 12/70
Progress: 100.00% | Loss: 0.3990

=====
Epoch 12/70
=====

Train Loss : 0.2896
Train Accuracy : 0.9813
Train Precision : 0.7860
Train Recall : 0.7505

Val Loss : 0.1678
Val Accuracy : 0.9814
Val Precision : 0.7404
Val Recall : 0.7720
Val F1 : 0.7477
Val Dice : 0.7477
Val IoU : 0.6057

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Model Saved!

Epoch 13/70

Progress: 100.00% | Loss: 0.4527

=====
Epoch 13/70
=====

Train Loss : 0.2854
Train Accuracy : 0.9816
Train Precision : 0.7929
Train Recall : 0.7504

Val Loss : 0.1698
Val Accuracy : 0.9802
Val Precision : 0.7024
Val Recall : 0.8193
Val F1 : 0.7484
Val Dice : 0.7484
Val IoU : 0.6070

Current LR : 0.00010000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue
=====

Early Stop Counter: 1/25

Epoch 14/70

Progress: 100.00% | Loss: 0.0885

=====
Epoch 14/70
=====

Train Loss : 0.2761
Train Accuracy : 0.9824
Train Precision : 0.7998
Train Recall : 0.7657

Val Loss : 0.1727
Val Accuracy : 0.9792
Val Precision : 0.6855
Val Recall : 0.8336
Val F1 : 0.7440
Val Dice : 0.7440
Val IoU : 0.6025

Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50

Agent Decision : reduce_lr

Early Stop Counter: 2/25

Epoch 15/70

Progress: 100.00% | Loss: 0.3473

Epoch 15/70

Train Loss : 0.2717
Train Accuracy : 0.9829
Train Precision : 0.8096
Train Recall : 0.7672

Val Loss : 0.1576
Val Accuracy : 0.9827
Val Precision : 0.7574
Val Recall : 0.7821
Val F1 : 0.7622
Val Dice : 0.7622
Val IoU : 0.6248

Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

Model Saved!

Epoch 16/70

Progress: 100.00% | Loss: 0.0804

Epoch 16/70

Train Loss : 0.2694
Train Accuracy : 0.9834
Train Precision : 0.8125
Train Recall : 0.7759

Val Loss : 0.1656
Val Accuracy : 0.9810
Val Precision : 0.7173
Val Recall : 0.8110
Val F1 : 0.7536
Val Dice : 0.7536

Val IoU : 0.6132
Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 1/25

Epoch 17/70
Progress: 100.00% | Loss: 0.1226

=====

Epoch 17/70

=====

Train Loss : 0.2673
Train Accuracy : 0.9836
Train Precision : 0.8187
Train Recall : 0.7740

Val Loss : 0.1570
Val Accuracy : 0.9826
Val Precision : 0.7584
Val Recall : 0.7834
Val F1 : 0.7633
Val Dice : 0.7633
Val IoU : 0.6259

Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 18/70
Progress: 100.00% | Loss: 0.1087

=====

Epoch 18/70

=====

Train Loss : 0.2632
Train Accuracy : 0.9840
Train Precision : 0.8226
Train Recall : 0.7795

Val Loss : 0.1516
Val Accuracy : 0.9835
Val Precision : 0.7850
Val Recall : 0.7727
Val F1 : 0.7715
Val Dice : 0.7715
Val IoU : 0.6364

Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

Model Saved!

Epoch 19/70
Progress: 100.00% | Loss: 0.4943

Epoch 19/70

Train Loss : 0.2615
Train Accuracy : 0.9843
Train Precision : 0.8248
Train Recall : 0.7862

Val Loss : 0.1521
Val Accuracy : 0.9832
Val Precision : 0.7714
Val Recall : 0.7876
Val F1 : 0.7712
Val Dice : 0.7712
Val IoU : 0.6363

Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

Early Stop Counter: 1/25

Epoch 20/70
Progress: 100.00% | Loss: 0.1504

Epoch 20/70

Train Loss : 0.2672
Train Accuracy : 0.9839
Train Precision : 0.8279
Train Recall : 0.7707

Val Loss : 0.1522
Val Accuracy : 0.9836
Val Precision : 0.7947
Val Recall : 0.7642
Val F1 : 0.7709
Val Dice : 0.7709
Val IoU : 0.6359

Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 2/25

Epoch 21/70
Progress: 100.00% | Loss: 0.1802

=====

Epoch 21/70

=====

Train Loss : 0.2653
Train Accuracy : 0.9847
Train Precision : 0.8300
Train Recall : 0.7928

Val Loss : 0.1527
Val Accuracy : 0.9832
Val Precision : 0.7765
Val Recall : 0.7829
Val F1 : 0.7708
Val Dice : 0.7708
Val IoU : 0.6363

Current LR : 0.00008000
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 3/25

Epoch 22/70
Progress: 100.00% | Loss: 0.1057

=====
Epoch 22/70
=====

Train Loss	: 0.2669
Train Accuracy	: 0.9847
Train Precision	: 0.8322
Train Recall	: 0.7878

Val Loss	: 0.1556
Val Accuracy	: 0.9820
Val Precision	: 0.7300
Val Recall	: 0.8314
Val F1	: 0.7688
Val Dice	: 0.7688
Val IoU	: 0.6336

Current LR	: 0.00006400
Threshold	: 0.50
BCE Weight	: 0.50
Dice Weight	: 0.50
Agent Decision	: reduce_lr

=====

Early Stop Counter: 4/25

Epoch 23/70
Progress: 100.00% | Loss: 0.3610

=====
Epoch 23/70
=====

Train Loss	: 0.2581
Train Accuracy	: 0.9854
Train Precision	: 0.8408
Train Recall	: 0.7985

Val Loss	: 0.1476
Val Accuracy	: 0.9836
Val Precision	: 0.7697
Val Recall	: 0.8068
Val F1	: 0.7801
Val Dice	: 0.7801
Val IoU	: 0.6480

Current LR	: 0.00006400
Threshold	: 0.50
BCE Weight	: 0.50

Dice Weight : 0.50
Agent Decision : continue

Model Saved!

Epoch 24/70
Progress: 100.00% | Loss: 0.0589

Epoch 24/70

Train Loss : 0.2637
Train Accuracy : 0.9855
Train Precision : 0.8418
Train Recall : 0.7981

Val Loss : 0.1470
Val Accuracy : 0.9842
Val Precision : 0.7911
Val Recall : 0.7874
Val F1 : 0.7807
Val Dice : 0.7807
Val IoU : 0.6496

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

Model Saved!

Epoch 25/70
Progress: 100.00% | Loss: 0.4325

Epoch 25/70

Train Loss : 0.2607
Train Accuracy : 0.9854
Train Precision : 0.8445
Train Recall : 0.7909

Val Loss : 0.1442
Val Accuracy : 0.9842
Val Precision : 0.7871
Val Recall : 0.7981
Val F1 : 0.7850

Val Dice : 0.7850
Val IoU : 0.6542

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 26/70
Progress: 86.36% | Loss: 0.0773

=====

Epoch 27/70

=====

Train Loss : 0.2533
Train Accuracy : 0.9860
Train Precision : 0.8491
Train Recall : 0.8026

Val Loss : 0.1430
Val Accuracy : 0.9841
Val Precision : 0.7659
Val Recall : 0.8250
Val F1 : 0.7873
Val Dice : 0.7873
Val IoU : 0.6573

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 28/70
Progress: 100.00% | Loss: 0.6305

=====

Epoch 28/70

=====

Train Loss : 0.2536
Train Accuracy : 0.9861
Train Precision : 0.8542
Train Recall : 0.8008

Val Loss : 0.1432
Val Accuracy : 0.9841
Val Precision : 0.7682
Val Recall : 0.8228
Val F1 : 0.7873
Val Dice : 0.7873
Val IoU : 0.6582

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Early Stop Counter: 1/25

Epoch 29/70
Progress: 100.00% | Loss: 0.1203

=====
Epoch 29/70
=====

Train Loss : 0.2466
Train Accuracy : 0.9864
Train Precision : 0.8548
Train Recall : 0.8065

Val Loss : 0.1436
Val Accuracy : 0.9848
Val Precision : 0.8085
Val Recall : 0.7801
Val F1 : 0.7870
Val Dice : 0.7870
Val IoU : 0.6568

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Early Stop Counter: 2/25

Epoch 30/70
Progress: 100.00% | Loss: 0.0327

=====

Epoch 30/70

```
=====
Train Loss      : 0.2500
Train Accuracy  : 0.9866
Train Precision : 0.8576
Train Recall    : 0.8100
```

```
Val Loss        : 0.1416
Val Accuracy    : 0.9847
Val Precision    : 0.7951
Val Recall      : 0.8019
Val F1          : 0.7907
Val Dice        : 0.7907
Val IoU         : 0.6622
```

```
Current LR      : 0.00006400
Threshold       : 0.50
BCE Weight      : 0.50
Dice Weight     : 0.50
Agent Decision  : continue
=====
```

Model Saved!

Epoch 31/70
Progress: 100.00% | Loss: 0.5152

Epoch 31/70

```
=====
Train Loss      : 0.2522
Train Accuracy  : 0.9866
Train Precision : 0.8583
Train Recall    : 0.8088
```

```
Val Loss        : 0.1399
Val Accuracy    : 0.9851
Val Precision    : 0.8007
Val Recall      : 0.7966
Val F1          : 0.7918
Val Dice        : 0.7918
Val IoU         : 0.6633
```

```
Current LR      : 0.00006400
Threshold       : 0.50
BCE Weight      : 0.50
Dice Weight     : 0.50
Agent Decision  : continue
=====
```

Model Saved!

Epoch 32/70

Progress: 100.00% | Loss: 0.5564

=====
Epoch 32/70
=====

Train Loss	: 0.2608
Train Accuracy	: 0.9867
Train Precision	: 0.8585
Train Recall	: 0.8112

Val Loss	: 0.1379
Val Accuracy	: 0.9849
Val Precision	: 0.7834
Val Recall	: 0.8242
Val F1	: 0.7970
Val Dice	: 0.7970
Val IoU	: 0.6698

Current LR	: 0.00006400
Threshold	: 0.50
BCE Weight	: 0.50
Dice Weight	: 0.50
Agent Decision	: continue

=====

Model Saved!

Epoch 33/70

Progress: 100.00% | Loss: 0.5738

=====
Epoch 33/70
=====

Train Loss	: 0.2610
Train Accuracy	: 0.9864
Train Precision	: 0.8589
Train Recall	: 0.8026

Val Loss	: 0.1366
Val Accuracy	: 0.9856
Val Precision	: 0.8139
Val Recall	: 0.7938
Val F1	: 0.7971
Val Dice	: 0.7971
Val IoU	: 0.6707

Current LR	: 0.00006400
------------	--------------

Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 34/70
Progress: 100.00% | Loss: 0.0780

=====

Epoch 34/70

=====

Train Loss : 0.2526
Train Accuracy : 0.9870
Train Precision : 0.8633
Train Recall : 0.8144

Val Loss : 0.1404
Val Accuracy : 0.9843
Val Precision : 0.7663
Val Recall : 0.8338
Val F1 : 0.7926
Val Dice : 0.7926
Val IoU : 0.6642

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 1/25

Epoch 35/70
Progress: 100.00% | Loss: 0.4847

=====

Epoch 35/70

=====

Train Loss : 0.2496
Train Accuracy : 0.9872
Train Precision : 0.8654
Train Recall : 0.8169

Val Loss : 0.1345
Val Accuracy : 0.9853
Val Precision : 0.7848

Val Recall : 0.8306
Val F1 : 0.8021
Val Dice : 0.8021
Val IoU : 0.6765

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 36/70
Progress: 100.00% | Loss: 0.3643

=====

Epoch 36/70

=====

Train Loss : 0.2471
Train Accuracy : 0.9871
Train Precision : 0.8664
Train Recall : 0.8134

Val Loss : 0.1364
Val Accuracy : 0.9849
Val Precision : 0.7742
Val Recall : 0.8414
Val F1 : 0.8007
Val Dice : 0.8007
Val IoU : 0.6758

Current LR : 0.00006400
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 1/25

Epoch 37/70
Progress: 100.00% | Loss: 0.0610

=====

Epoch 37/70

=====

Train Loss : 0.2407
Train Accuracy : 0.9876

Train Precision : 0.8682
Train Recall : 0.8237

Val Loss : 0.1385
Val Accuracy : 0.9849
Val Precision : 0.7887
Val Recall : 0.8181
Val F1 : 0.7962
Val Dice : 0.7962
Val IoU : 0.6692

Current LR : 0.00005120
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : reduce_lr

=====
Early Stop Counter: 2/25

Epoch 38/70
Progress: 100.00% | Loss: 0.3496

=====
Epoch 38/70
=====

Train Loss : 0.2409
Train Accuracy : 0.9877
Train Precision : 0.8732
Train Recall : 0.8224

Val Loss : 0.1341
Val Accuracy : 0.9851
Val Precision : 0.7794
Val Recall : 0.8420
Val F1 : 0.8037
Val Dice : 0.8037
Val IoU : 0.6797

Current LR : 0.00005120
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue
=====

Model Saved!

Epoch 39/70
Progress: 100.00% | Loss: 0.3939

=====
Epoch 39/70
=====

Train Loss : 0.2499
Train Accuracy : 0.9877
Train Precision : 0.8740
Train Recall : 0.8205

Val Loss : 0.1354
Val Accuracy : 0.9852
Val Precision : 0.7850
Val Recall : 0.8351
Val F1 : 0.8029
Val Dice : 0.8029
Val IoU : 0.6784

Current LR : 0.00004096
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : reduce_lr
=====

Early Stop Counter: 1/25

Epoch 40/70
Progress: 100.00% | Loss: 0.5000

=====
Epoch 40/70
=====

Train Loss : 0.2453
Train Accuracy : 0.9876
Train Precision : 0.8764
Train Recall : 0.8159

Val Loss : 0.1335
Val Accuracy : 0.9856
Val Precision : 0.8003
Val Recall : 0.8235
Val F1 : 0.8054
Val Dice : 0.8054
Val IoU : 0.6816

Current LR : 0.00004096
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Model Saved!

Epoch 41/70

Progress: 100.00% | Loss: 0.5218

=====
Epoch 41/70
=====

Train Loss	: 0.2390
Train Accuracy	: 0.9880
Train Precision	: 0.8801
Train Recall	: 0.8226

Val Loss	: 0.1327
Val Accuracy	: 0.9858
Val Precision	: 0.8053
Val Recall	: 0.8186
Val F1	: 0.8062
Val Dice	: 0.8062
Val IoU	: 0.6829

Current LR	: 0.00004096
Threshold	: 0.50
BCE Weight	: 0.50
Dice Weight	: 0.50
Agent Decision	: continue

=====

Model Saved!

Epoch 42/70

Progress: 100.00% | Loss: 0.4642

=====
Epoch 42/70
=====

Train Loss	: 0.2324
Train Accuracy	: 0.9885
Train Precision	: 0.8825
Train Recall	: 0.8330

Val Loss	: 0.1312
Val Accuracy	: 0.9858
Val Precision	: 0.7944
Val Recall	: 0.8338
Val F1	: 0.8081
Val Dice	: 0.8081
Val IoU	: 0.6847

Current LR : 0.00004096
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 43/70
Progress: 100.00% | Loss: 0.3940

=====

Epoch 43/70

=====

Train Loss : 0.2462
Train Accuracy : 0.9879
Train Precision : 0.8820
Train Recall : 0.8158

Val Loss : 0.1287
Val Accuracy : 0.9860
Val Precision : 0.7969
Val Recall : 0.8391
Val F1 : 0.8123
Val Dice : 0.8123
Val IoU : 0.6903

Current LR : 0.00004096
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 44/70
Progress: 100.00% | Loss: 0.5332

=====

Epoch 44/70

=====

Train Loss : 0.2419
Train Accuracy : 0.9884
Train Precision : 0.8834
Train Recall : 0.8292

Val Loss : 0.1312

Val Accuracy : 0.9864
Val Precision : 0.8269
Val Recall : 0.8005
Val F1 : 0.8078
Val Dice : 0.8078
Val IoU : 0.6854

Current LR : 0.00004096
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Early Stop Counter: 1/25

Epoch 45/70
Progress: 100.00% | Loss: 0.0579

=====
Epoch 45/70
=====

Train Loss : 0.2455
Train Accuracy : 0.9882
Train Precision : 0.8845
Train Recall : 0.8211

Val Loss : 0.1273
Val Accuracy : 0.9863
Val Precision : 0.8040
Val Recall : 0.8323
Val F1 : 0.8140
Val Dice : 0.8140
Val IoU : 0.6925

Current LR : 0.00004096
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====
Model Saved!

Epoch 46/70
Progress: 100.00% | Loss: 0.4084

=====
Epoch 46/70
=====

Train Loss : 0.2347
Train Accuracy : 0.9884
Train Precision : 0.8862
Train Recall : 0.8258

Val Loss : 0.1291
Val Accuracy : 0.9860
Val Precision : 0.7949
Val Recall : 0.8376
Val F1 : 0.8109
Val Dice : 0.8109
Val IoU : 0.6885

Current LR : 0.00004096
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 1/25

Epoch 47/70
Progress: 100.00% | Loss: 0.0622

=====

Epoch 47/70

=====

Train Loss : 0.2374
Train Accuracy : 0.9884
Train Precision : 0.8876
Train Recall : 0.8230

Val Loss : 0.1295
Val Accuracy : 0.9862
Val Precision : 0.8029
Val Recall : 0.8309
Val F1 : 0.8115
Val Dice : 0.8115
Val IoU : 0.6898

Current LR : 0.00003277
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : reduce_lr

=====

Early Stop Counter: 2/25

Epoch 48/70
Progress: 100.00% | Loss: 0.0797

=====
Epoch 48/70
=====

Train Loss	: 0.2392
Train Accuracy	: 0.9889
Train Precision	: 0.8890
Train Recall	: 0.8347

Val Loss	: 0.1272
Val Accuracy	: 0.9863
Val Precision	: 0.8032
Val Recall	: 0.8366
Val F1	: 0.8150
Val Dice	: 0.8150
Val IoU	: 0.6942

Current LR	: 0.00003277
Threshold	: 0.50
BCE Weight	: 0.50
Dice Weight	: 0.50
Agent Decision	: continue

=====

Model Saved!

Epoch 49/70
Progress: 100.00% | Loss: 0.0581

=====
Epoch 49/70
=====

Train Loss	: 0.2409
Train Accuracy	: 0.9882
Train Precision	: 0.8892
Train Recall	: 0.8171

Val Loss	: 0.1273
Val Accuracy	: 0.9866
Val Precision	: 0.8238
Val Recall	: 0.8161
Val F1	: 0.8149
Val Dice	: 0.8149
Val IoU	: 0.6944

Current LR	: 0.00003277
Threshold	: 0.50
BCE Weight	: 0.50

Dice Weight : 0.50
Agent Decision : continue

Early Stop Counter: 1/25

Epoch 50/70
Progress: 100.00% | Loss: 0.0866

Epoch 50/70

Train Loss : 0.2358
Train Accuracy : 0.9888
Train Precision : 0.8916
Train Recall : 0.8294

Val Loss : 0.1270
Val Accuracy : 0.9867
Val Precision : 0.8233
Val Recall : 0.8160
Val F1 : 0.8147
Val Dice : 0.8147
Val IoU : 0.6944

Current LR : 0.00003277
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

Model Saved!

Epoch 51/70
Progress: 100.00% | Loss: 0.3536

Epoch 51/70

Train Loss : 0.2374
Train Accuracy : 0.9888
Train Precision : 0.8937
Train Recall : 0.8275

Val Loss : 0.1262
Val Accuracy : 0.9866
Val Precision : 0.8140
Val Recall : 0.8296
Val F1 : 0.8169

Val Dice : 0.8169
Val IoU : 0.6971

Current LR : 0.00003277
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 52/70
Progress: 100.00% | Loss: 0.0980

=====

Epoch 52/70

=====

Train Loss : 0.2333
Train Accuracy : 0.9887
Train Precision : 0.8933
Train Recall : 0.8241

Val Loss : 0.1278
Val Accuracy : 0.9865
Val Precision : 0.8186
Val Recall : 0.8219
Val F1 : 0.8147
Val Dice : 0.8147
Val IoU : 0.6944

Current LR : 0.00002621
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : reduce_lr

=====

Early Stop Counter: 1/25

Epoch 53/70
Progress: 100.00% | Loss: 0.0571

=====

Epoch 53/70

=====

Train Loss : 0.2340
Train Accuracy : 0.9894
Train Precision : 0.8963
Train Recall : 0.8400

Val Loss : 0.1280
Val Accuracy : 0.9864
Val Precision : 0.8118
Val Recall : 0.8297
Val F1 : 0.8149
Val Dice : 0.8149
Val IoU : 0.6946

Current LR : 0.00002097
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : reduce_lr

=====
Early Stop Counter: 2/25

Epoch 54/70
Progress: 100.00% | Loss: 0.4314

=====
Epoch 54/70
=====

Train Loss : 0.2375
Train Accuracy : 0.9890
Train Precision : 0.8970
Train Recall : 0.8297

Val Loss : 0.1287
Val Accuracy : 0.9865
Val Precision : 0.8169
Val Recall : 0.8238
Val F1 : 0.8147
Val Dice : 0.8147
Val IoU : 0.6943

Current LR : 0.00001678
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : reduce_lr

=====
Early Stop Counter: 3/25

Epoch 55/70
Progress: 100.00% | Loss: 0.1282

=====

Epoch 55/70

```
=====
Train Loss      : 0.2292
Train Accuracy  : 0.9896
Train Precision : 0.9001
Train Recall    : 0.8402
```

```
Val Loss        : 0.1266
Val Accuracy    : 0.9866
Val Precision    : 0.8166
Val Recall      : 0.8314
Val F1          : 0.8183
Val Dice        : 0.8183
Val IoU         : 0.6994
```

```
Current LR      : 0.00001342
Threshold       : 0.50
BCE Weight      : 0.50
Dice Weight     : 0.50
Agent Decision  : reduce_lr
=====
```

Early Stop Counter: 4/25

Epoch 56/70
Progress: 100.00% | Loss: 0.0620

Epoch 56/70

```
=====
Train Loss      : 0.2277
Train Accuracy  : 0.9896
Train Precision : 0.9003
Train Recall    : 0.8425
```

```
Val Loss        : 0.1267
Val Accuracy    : 0.9868
Val Precision    : 0.8251
Val Recall      : 0.8214
Val F1          : 0.8178
Val Dice        : 0.8178
Val IoU         : 0.6984
```

```
Current LR      : 0.00001342
Threshold       : 0.50
BCE Weight      : 0.50
Dice Weight     : 0.50
Agent Decision  : continue
=====
```

Early Stop Counter: 5/25

Epoch 57/70

Progress: 100.00% | Loss: 0.0447

=====
Epoch 57/70
=====

Train Loss	: 0.2246
Train Accuracy	: 0.9900
Train Precision	: 0.9023
Train Recall	: 0.8497

Val Loss	: 0.1265
Val Accuracy	: 0.9867
Val Precision	: 0.8187
Val Recall	: 0.8282
Val F1	: 0.8179
Val Dice	: 0.8179
Val IoU	: 0.6990

Current LR	: 0.00000671
Threshold	: 0.50
BCE Weight	: 0.50
Dice Weight	: 0.50
Agent Decision	: continue

=====

Early Stop Counter: 6/25

Epoch 58/70

Progress: 100.00% | Loss: 0.3641

=====
Epoch 58/70
=====

Train Loss	: 0.2279
Train Accuracy	: 0.9897
Train Precision	: 0.9026
Train Recall	: 0.8402

Val Loss	: 0.1269
Val Accuracy	: 0.9867
Val Precision	: 0.8207
Val Recall	: 0.8256
Val F1	: 0.8175
Val Dice	: 0.8175
Val IoU	: 0.6984

Current LR	: 0.00000671
------------	--------------

Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Early Stop Counter: 7/25

Epoch 59/70
Progress: 100.00% | Loss: 0.3654

=====

Epoch 59/70

=====

Train Loss : 0.2417
Train Accuracy : 0.9893
Train Precision : 0.9019
Train Recall : 0.8305

Val Loss : 0.1260
Val Accuracy : 0.9867
Val Precision : 0.8167
Val Recall : 0.8331
Val F1 : 0.8194
Val Dice : 0.8194
Val IoU : 0.7009

Current LR : 0.00000671
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 60/70
Progress: 100.00% | Loss: 0.0503

=====

Epoch 60/70

=====

Train Loss : 0.2241
Train Accuracy : 0.9897
Train Precision : 0.9038
Train Recall : 0.8413

Val Loss : 0.1272
Val Accuracy : 0.9868
Val Precision : 0.8220
Val Recall : 0.8272

Val F1 : 0.8191
Val Dice : 0.8191
Val IoU : 0.7006

Current LR : 0.00000537
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : reduce_lr

=====

Early Stop Counter: 1/25

Epoch 61/70
Progress: 100.00% | Loss: 0.0770

=====

Epoch 61/70

=====

Train Loss : 0.2308
Train Accuracy : 0.9898
Train Precision : 0.9041
Train Recall : 0.8432

Val Loss : 0.1259
Val Accuracy : 0.9868
Val Precision : 0.8172
Val Recall : 0.8340
Val F1 : 0.8202
Val Dice : 0.8202
Val IoU : 0.7020

Current LR : 0.00000537
Threshold : 0.50
BCE Weight : 0.50
Dice Weight : 0.50
Agent Decision : continue

=====

Model Saved!

Epoch 62/70
Progress: 100.00% | Loss: 0.4676

=====

Epoch 62/70

=====

Train Loss : 0.2321
Train Accuracy : 0.9896
Train Precision : 0.9039

Train Recall : 0.8373

Val Loss : 0.1265

Val Accuracy : 0.9868

Val Precision : 0.8206

Val Recall : 0.8300

Val F1 : 0.8198

Val Dice : 0.8198

Val IoU : 0.7016

Current LR : 0.00000537

Threshold : 0.50

BCE Weight : 0.50

Dice Weight : 0.50

Agent Decision : continue

=====
Early Stop Counter: 1/25

Epoch 63/70

Progress: 100.00% | Loss: 0.3919

=====
Epoch 63/70
=====

Train Loss : 0.2302

Train Accuracy : 0.9896

Train Precision : 0.9052

Train Recall : 0.8354

Val Loss : 0.1261

Val Accuracy : 0.9868

Val Precision : 0.8210

Val Recall : 0.8304

Val F1 : 0.8202

Val Dice : 0.8202

Val IoU : 0.7022

Current LR : 0.00000429

Threshold : 0.50

BCE Weight : 0.50

Dice Weight : 0.50

Agent Decision : reduce_lr

=====
Early Stop Counter: 2/25

Epoch 64/70

Progress: 100.00% | Loss: 0.4064

=====
Epoch 64/70
=====

Train Loss : 0.2285
Train Accuracy : 0.9898
Train Precision : 0.9052
Train Recall : 0.8420

Val Loss : 0.1269
Val Accuracy : 0.9868
Val Precision : 0.8210
Val Recall : 0.8278
Val F1 : 0.8189
Val Dice : 0.8189
Val IoU : 0.7004

Current LR : 0.00000429
Threshold : 0.50
BCE Weight : 0.45
Dice Weight : 0.55
Agent Decision : increase_dice_weight
=====

Early Stop Counter: 3/25

Epoch 65/70
Progress: 100.00% | Loss: 0.0671

=====
Epoch 65/70
=====

Train Loss : 0.2340
Train Accuracy : 0.9904
Train Precision : 0.9058
Train Recall : 0.8562

Val Loss : 0.1330
Val Accuracy : 0.9868
Val Precision : 0.8234
Val Recall : 0.8271
Val F1 : 0.8198
Val Dice : 0.8198
Val IoU : 0.7018

Current LR : 0.00000344
Threshold : 0.50
BCE Weight : 0.45
Dice Weight : 0.55
Agent Decision : reduce_lr
=====

Early Stop Counter: 4/25

Epoch 66/70

Progress: 100.00% | Loss: 0.0504

=====
Epoch 66/70
=====

Train Loss : 0.2456
Train Accuracy : 0.9902
Train Precision : 0.9045
Train Recall : 0.8526

Val Loss : 0.1334
Val Accuracy : 0.9867
Val Precision : 0.8134
Val Recall : 0.8375
Val F1 : 0.8198
Val Dice : 0.8198
Val IoU : 0.7015

Current LR : 0.00000275
Threshold : 0.50
BCE Weight : 0.45
Dice Weight : 0.55
Agent Decision : reduce_lr
=====

Early Stop Counter: 5/25

Epoch 67/70

Progress: 100.00% | Loss: 0.0636

=====
Epoch 67/70
=====

Train Loss : 0.2471
Train Accuracy : 0.9896
Train Precision : 0.9039
Train Recall : 0.8369

Val Loss : 0.1333
Val Accuracy : 0.9868
Val Precision : 0.8177
Val Recall : 0.8336
Val F1 : 0.8201
Val Dice : 0.8201
Val IoU : 0.7021

Current LR : 0.00000110
Threshold : 0.50
BCE Weight : 0.45
Dice Weight : 0.55
Agent Decision : reduce_lr

=====

Early Stop Counter: 6/25

Epoch 68/70
Progress: 100.00% | Loss: 0.0569

=====

Epoch 68/70

=====

Train Loss : 0.2462
Train Accuracy : 0.9897
Train Precision : 0.9049
Train Recall : 0.8382

Val Loss : 0.1332
Val Accuracy : 0.9868
Val Precision : 0.8208
Val Recall : 0.8307
Val F1 : 0.8203
Val Dice : 0.8203
Val IoU : 0.7022

Current LR : 0.00000088
Threshold : 0.50
BCE Weight : 0.45
Dice Weight : 0.55
Agent Decision : reduce_lr

=====

Early Stop Counter: 7/25

Epoch 69/70
Progress: 100.00% | Loss: 0.5043

=====

Epoch 69/70

=====

Train Loss : 0.2547
Train Accuracy : 0.9896
Train Precision : 0.9055
Train Recall : 0.8366

Val Loss : 0.1337
Val Accuracy : 0.9867

Val Precision : 0.8154
Val Recall : 0.8363
Val F1 : 0.8202
Val Dice : 0.8202
Val IoU : 0.7022

Current LR : 0.00000070
Threshold : 0.50
BCE Weight : 0.45
Dice Weight : 0.55
Agent Decision : reduce_lr

=====
Early Stop Counter: 8/25

Epoch 70/70
Progress: 100.00% | Loss: 0.0453

=====
Epoch 70/70
=====

Train Loss : 0.2397
Train Accuracy : 0.9899
Train Precision : 0.9051
Train Recall : 0.8448

Val Loss : 0.1323
Val Accuracy : 0.9869
Val Precision : 0.8252
Val Recall : 0.8265
Val F1 : 0.8203
Val Dice : 0.8203
Val IoU : 0.7023

Current LR : 0.00000070
Threshold : 0.50
BCE Weight : 0.45
Dice Weight : 0.55
Agent Decision : continue

=====
Early Stop Counter: 9/25

=====
=====

FINAL TRAINING SUMMARY

=====

=====

Best Validation Loss : 0.1259

```
Final Train Accuracy : 0.9899
Final Train Precision: 0.9051
Final Train Recall   : 0.8448
```

```
Final Val Accuracy   : 0.9869
Final Val Precision   : 0.8252
Final Val Recall      : 0.8265
Final Val F1          : 0.8203
Final Val Dice        : 0.8203
Final Val IoU         : 0.7023
```

```
=====
=====
```

```
model.load_state_dict(
    torch.load(
        "segformer_best.pth"
    )
)

model.eval()

acc = 0
prec = 0
rec = 0

f1_total = 0
dice_total = 0
iou_total = 0

count = 0

for images, masks in val_loader:
    images = images.to(device)
    masks = masks.to(device)
    with torch.no_grad():
        pred = model(images)
        a,p,r,f1,d,i = calculate_all_metrics(
            pred,
            masks
```

```

    )

    acc += a
    prec += p
    rec += r

    f1_total += f1
    dice_total += d
    iou_total += i

    count += 1

print("\nVALIDATION RESULTS\n")
print("Validation Accuracy :", acc/count)
print("Validation Precision:", prec/count)
print("Validation Recall   :", rec/count)
print("Validation F1 Score :", f1_total/count)
print("Validation Dice      :", dice_total/count)
print("Validation IoU       :", iou_total/count)

```

VALIDATION RESULTS

```

Validation Accuracy : 0.9867753035599186
Validation Precision: 0.8172441738387269
Validation Recall   : 0.8339891604358151
Validation F1 Score : 0.8202124051749706
Validation Dice      : 0.8202124051749706
Validation IoU       : 0.7020397168253699

```

```
model.eval()
```

```

acc = 0
prec = 0
rec = 0

```

```

f1_total = 0
dice_total = 0
iou_total = 0

```

```
count = 0
```

```

all_preds = []
all_targets = []

```

```
for images, masks in test_loader:
    images = images.to(device)
    masks = masks.to(device)
    with torch.no_grad():
        pred = model(images)
        a,p,r,f1,d,i = calculate_all_metrics(
            pred,
            masks
        )
        acc += a
        prec += p
        rec += r

        f1_total += f1
        dice_total += d
        iou_total += i

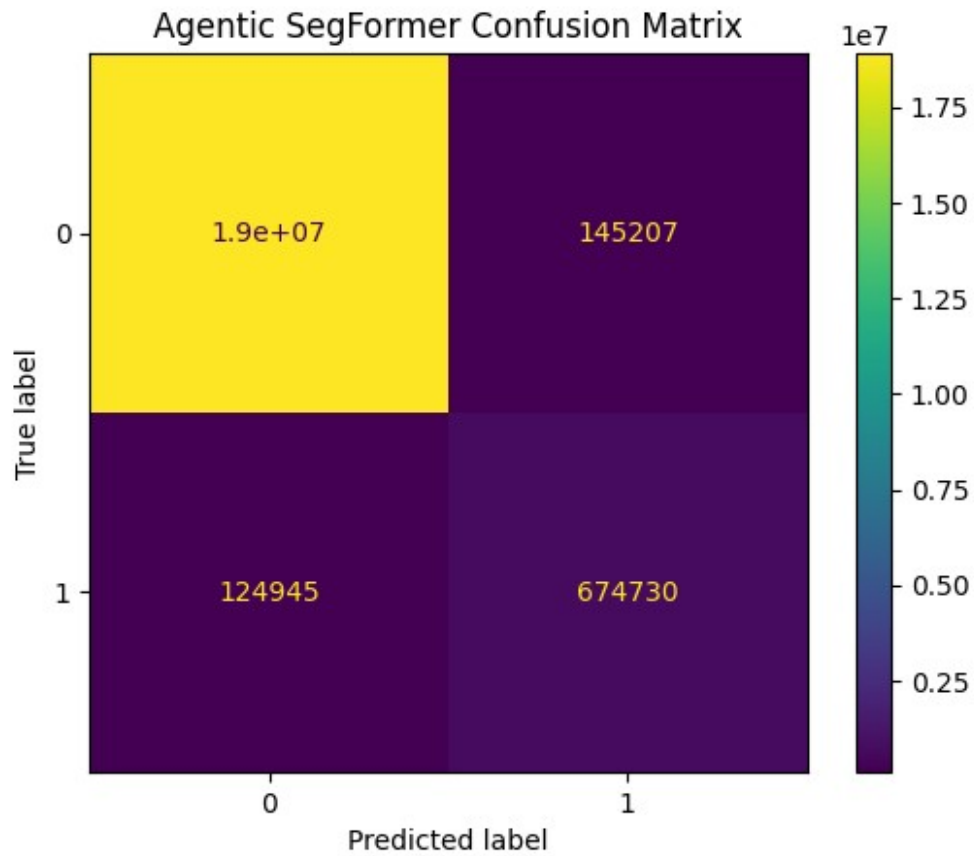
        count += 1
        pred_bin = (
            torch.sigmoid(pred)
            > 0.5
        ).cpu().numpy().flatten()
        target_bin = masks.cpu().numpy().flatten()
        all_preds.extend(
            pred_bin
        )
        all_targets.extend(
            target_bin
        )
```

```
print("\nFINAL TEST RESULTS\n")
print("Test Accuracy :", acc/count)
print("Test Precision:", prec/count)
print("Test Recall   :", rec/count)
print("Test F1 Score :", f1_total/count)
print("Test Dice     :", dice_total/count)
print("Test IoU      :", iou_total/count)

cm = confusion_matrix(
    all_targets,
    all_preds
)
ConfusionMatrixDisplay(
    cm
).plot()
plt.title(
    "Agentic SegFormer Confusion Matrix"
)
plt.show()
```

FINAL TEST RESULTS

```
Test Accuracy : 0.9863942279873124
Test Precision: 0.8173871740519282
Test Recall   : 0.8369871632880476
Test F1 Score : 0.8206561156305443
Test Dice     : 0.8206561156305443
Test IoU      : 0.7040808777732542
```



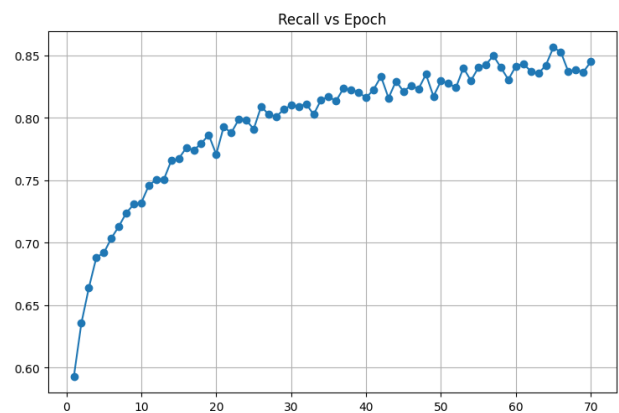
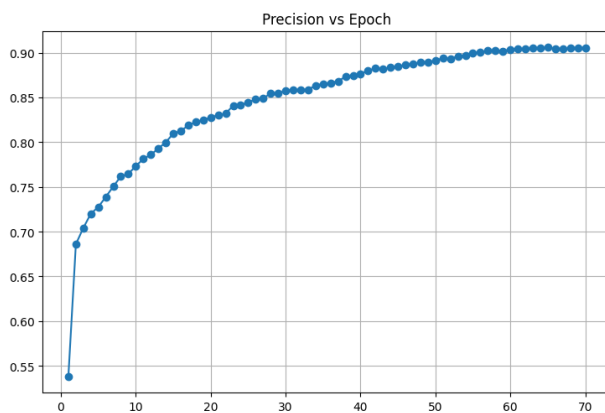
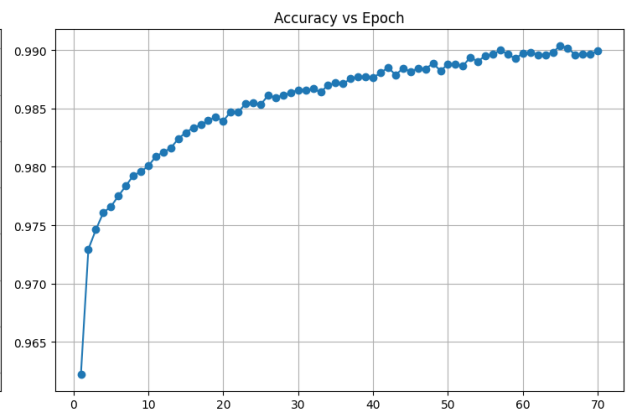
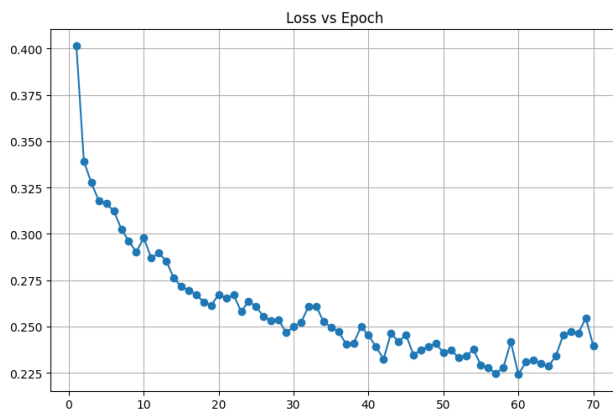
```
epochs_range = range(
    1,
    len(train_losses)+1
)
plt.figure(figsize=(15,10))
# ===== LOSS =====
plt.subplot(2,2,1)
plt.plot(
    epochs_range,
    train_losses,
    marker='o'
)
```

```
plt.title(
    "Loss vs Epoch"
)
plt.grid()
# ===== ACCURACY =====
plt.subplot(2,2,2)
plt.plot(
    epochs_range,
    train_acc,
    marker='o'
)
plt.title(
    "Accuracy vs Epoch"
)
plt.grid()
# ===== PRECISION =====
plt.subplot(2,2,3)
plt.plot(
    epochs_range,
    train_prec,
    marker='o'
)
plt.title(
    "Precision vs Epoch"
)
plt.grid()
```

```

# ===== RECALL =====
plt.subplot(2,2,4)
plt.plot(
    epochs_range,
    train_rec,
    marker='o'
)
plt.title(
    "Recall vs Epoch"
)
plt.grid()
plt.tight_layout()
plt.show()

```




```

import matplotlib.pyplot as plt
epochs_range = range(
    1,
    len(train_acc) + 1
)

# =====
# ACCURACY
# =====

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    train_acc,
    label="Train Accuracy"
)
plt.plot(
    epochs_range,
    val_acc,
    label="Validation Accuracy"
)
plt.title("Accuracy vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# =====
# PRECISION
# =====

```

```

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    train_prec,
    label="Train Precision"
)
plt.plot(
    epochs_range,
    val_prec,
    label="Validation Precision"
)
plt.title("Precision vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Precision")
plt.legend()
plt.grid(True)
plt.show()

# =====
# RECALL
# =====

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    train_rec,
    label="Train Recall"
)
plt.plot(

```

```
        epochs_range,
        val_rec,
        label="Validation Recall"
    )
plt.title("Recall vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Recall")
plt.legend()
plt.grid(True)
plt.show()

# =====
# VALIDATION F1 SCORE
# =====

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    val_f1,
    label="Validation F1 Score"
)
plt.title("Validation F1 Score vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("F1 Score")
plt.legend()
plt.grid(True)
plt.show()

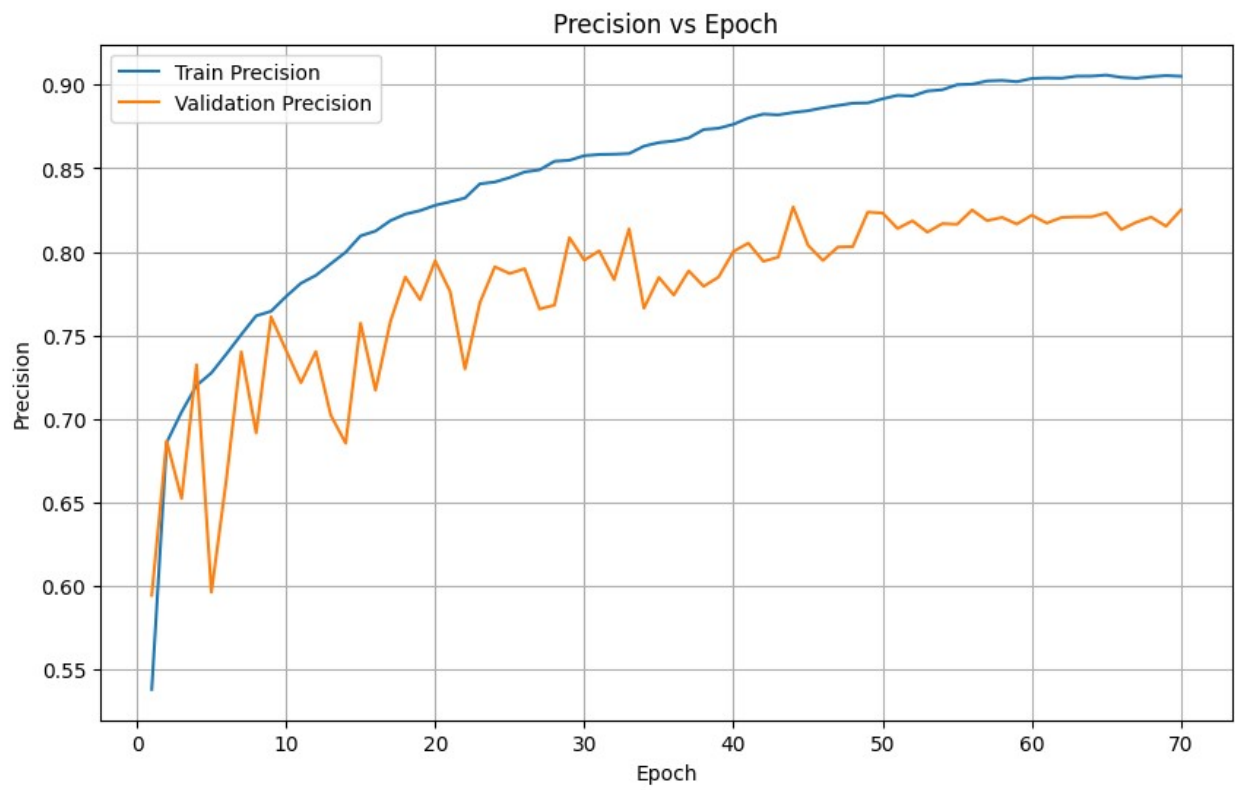
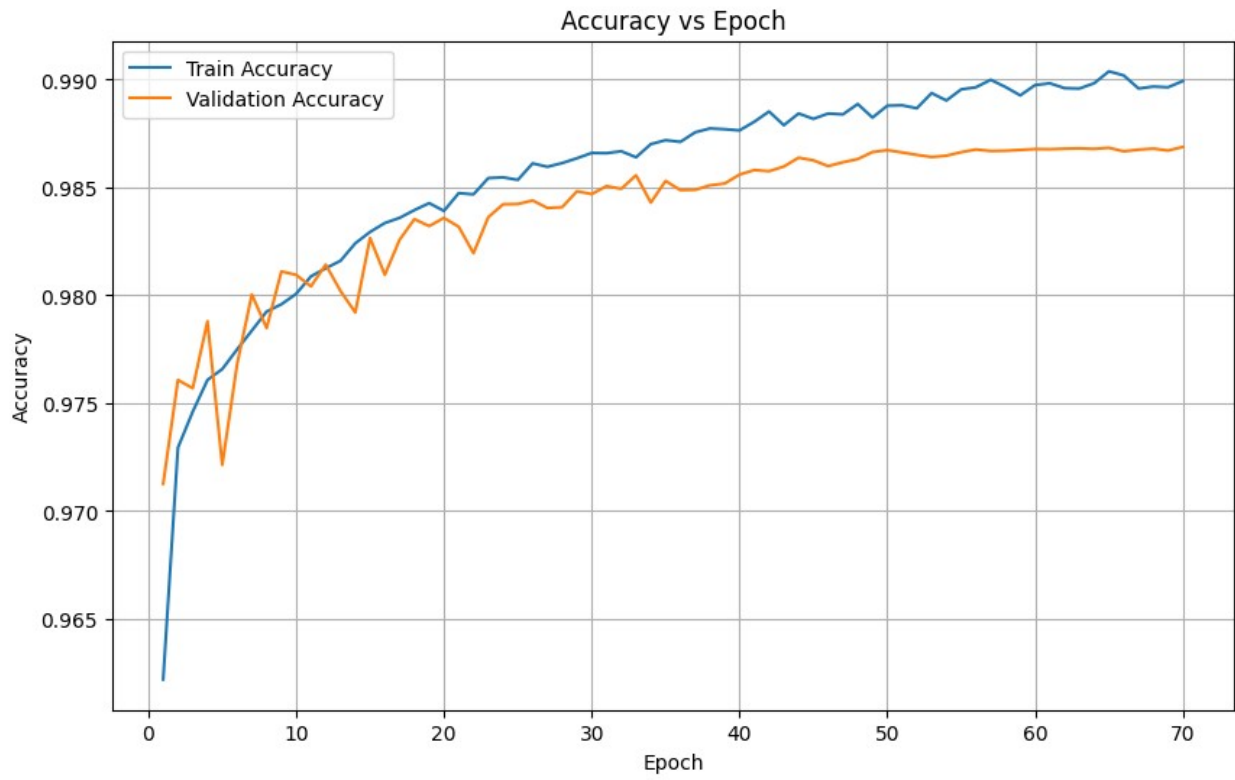
# =====
# VALIDATION DICE
# =====
```

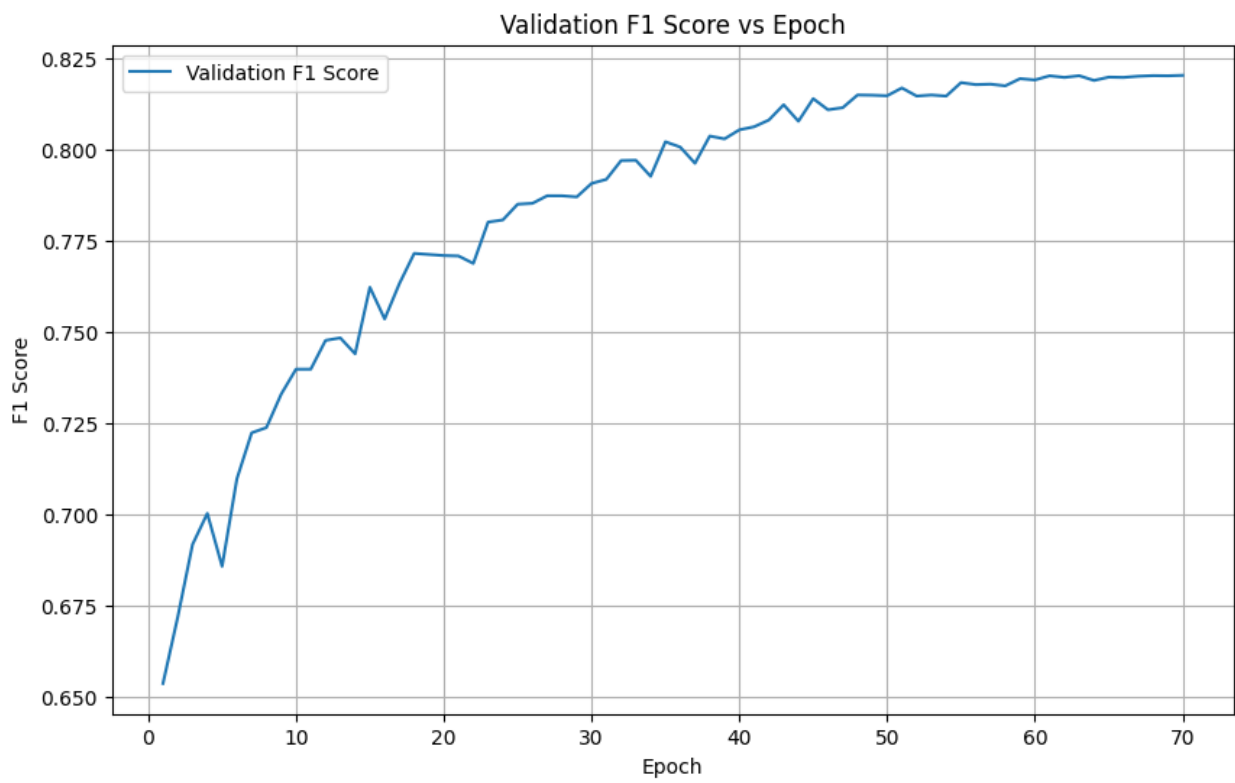
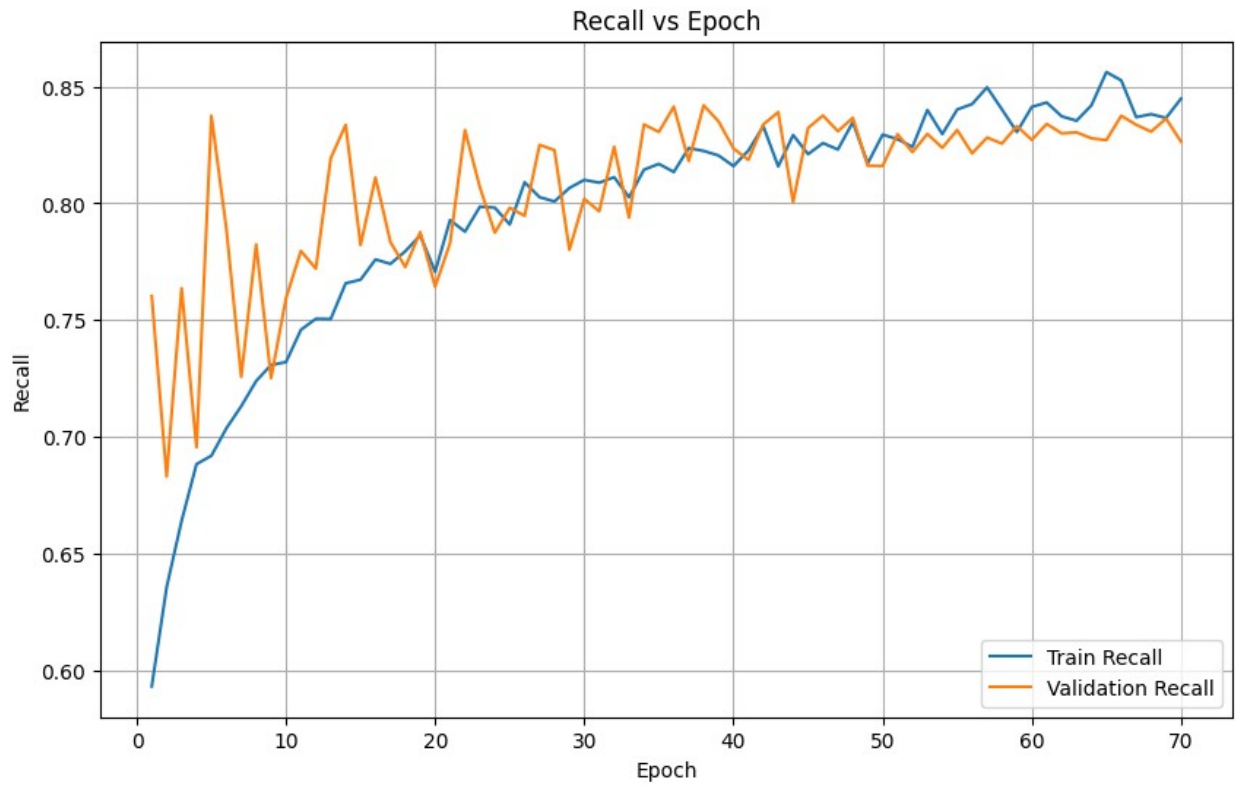
```
plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    val_dice,
    label="Validation Dice"
)
plt.title("Validation Dice vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Dice")
plt.legend()
plt.grid(True)
plt.show()

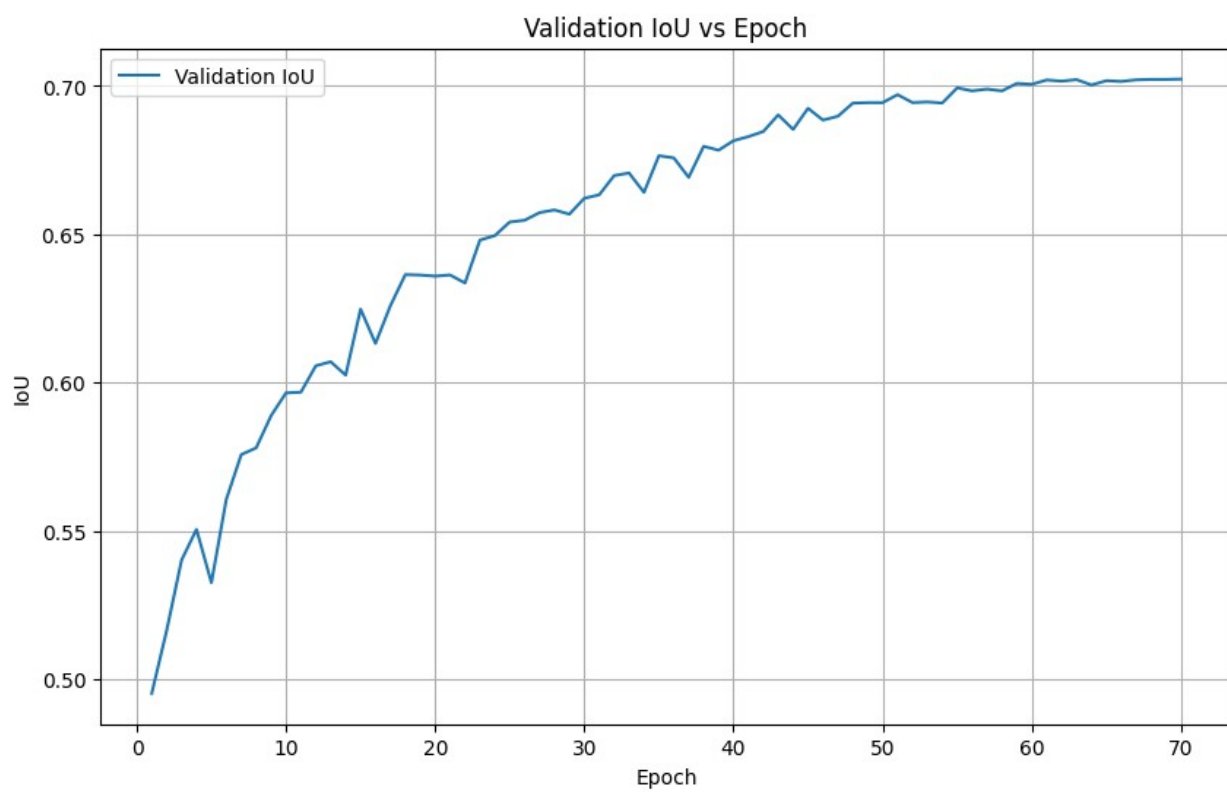
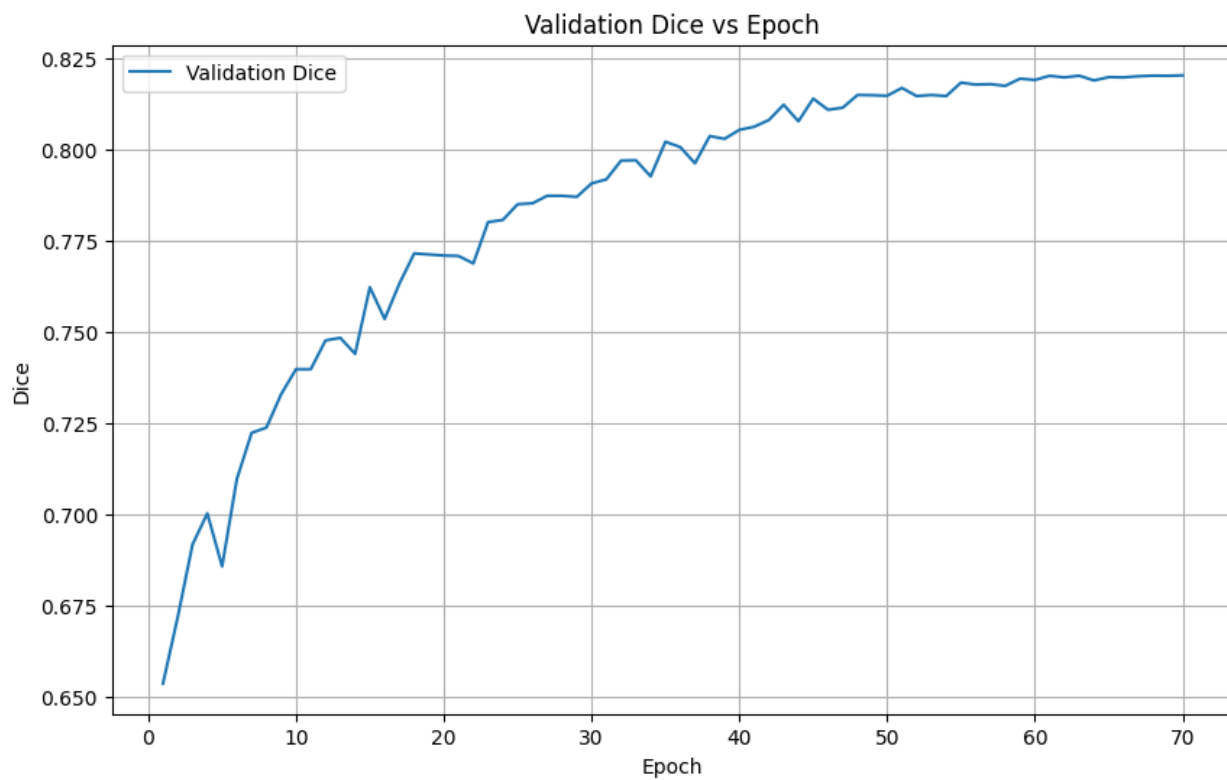
# =====
# VALIDATION IoU
# =====

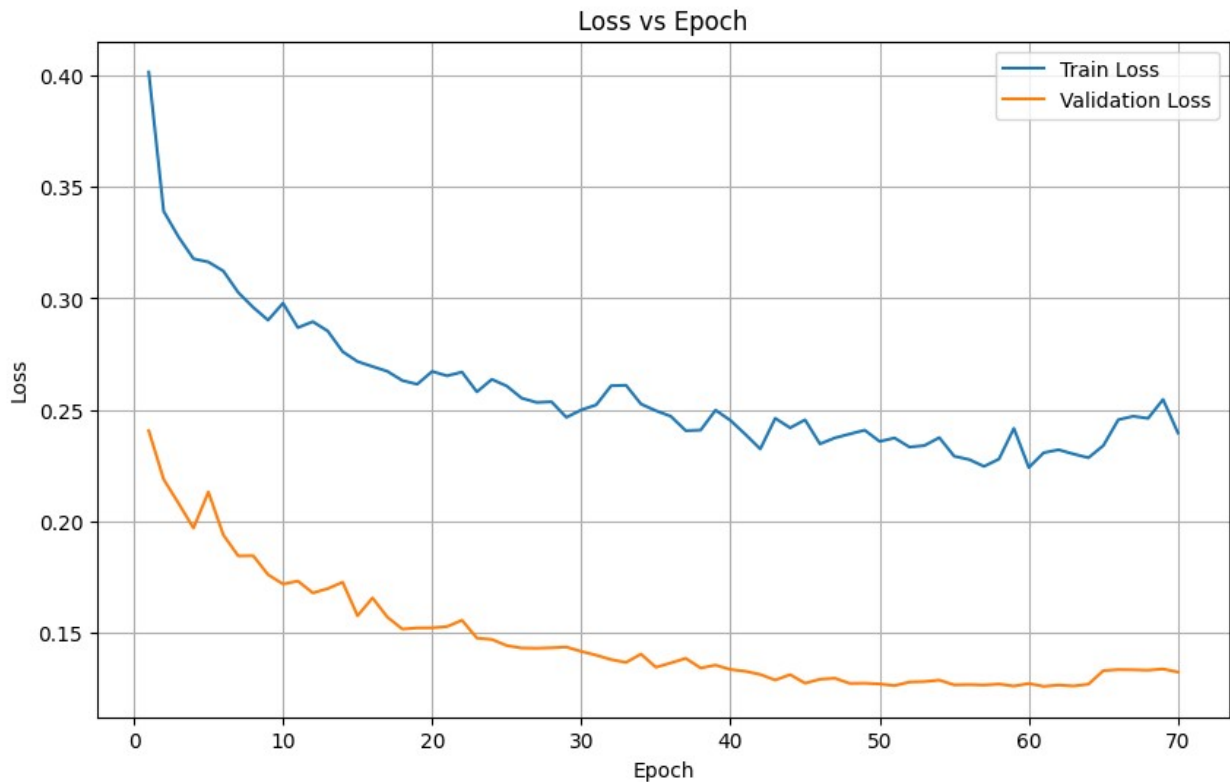
plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    val_iou,
    label="Validation IoU"
)
plt.title("Validation IoU vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.legend()
plt.grid(True)
plt.show()
```

```
# =====  
# LOSS  
# =====  
  
plt.figure(figsize=(10,6))  
plt.plot(  
    epochs_range,  
    train_losses,  
    label="Train Loss"  
)  
plt.plot(  
    epochs_range,  
    val_losses,  
    label="Validation Loss"  
)  
plt.title("Loss vs Epoch")  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.legend()  
plt.grid(True)  
plt.show()
```









Loss Curve Analysis

Key Observations

- Training loss decreased from **0.41** → **0.25**, indicating stable learning.
- Validation loss decreased from **0.23** → **0.13**, showing improved generalization.
- Validation loss remained consistently lower than training loss.
- No significant divergence was observed between training and validation curves.
- Accuracy, Precision, and Recall improved steadily throughout training.

Reason for the Loss Gap

- Training data used **data augmentation**, making training samples more challenging.
- **Dropout regularization** was active during training and disabled during validation.
- Validation data was evaluated on cleaner, unaugmented samples.
- Hybrid **BCE + Dice Loss** introduced additional regularization effects.

Conclusion

- The gap between training and validation loss is expected and normal.
- The model shows **good generalization ability**.
- No evidence of severe overfitting was observed.
- The training process remained stable and converged successfully.

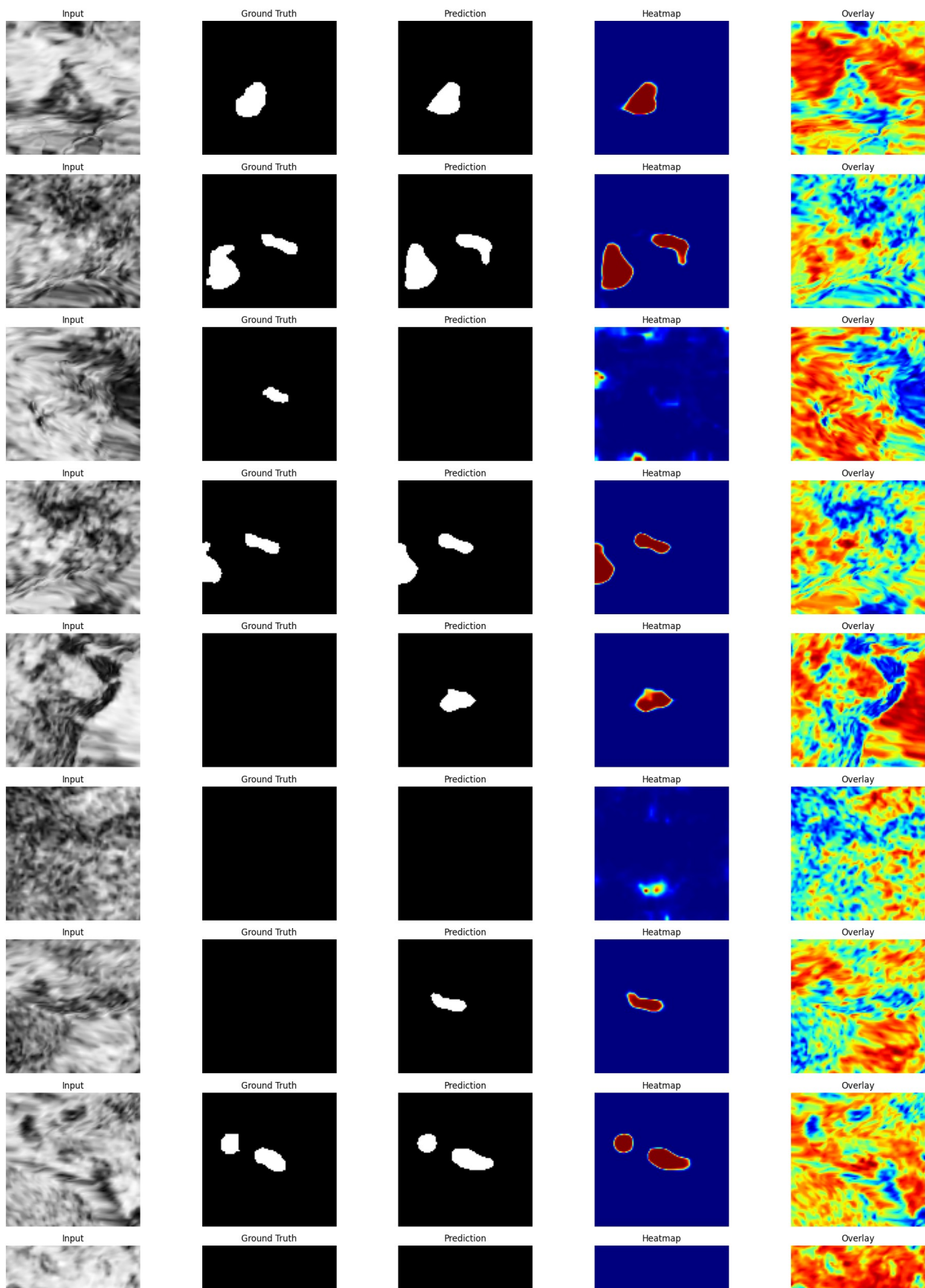
```
model.eval()
```

```
images_collected = 0
num_images = 10
all_inputs = []
all_masks = []
all_preds = []
for images, masks in test_loader:
    images = images.to(device)
    with torch.no_grad():
        pred = model(images)
        pred = torch.sigmoid(pred)
    all_inputs.extend(
        images.cpu().numpy()
    )
    all_masks.extend(
        masks.numpy()
    )
    all_preds.extend(
        pred.cpu().numpy()
    )
    images_collected += images.shape[0]
    if images_collected >= num_images:
        break
all_inputs = np.array(all_inputs)
all_masks = np.array(all_masks)
all_preds = np.array(all_preds)
plt.figure(figsize=(20,30))
```

```
for i in range(num_images):
    input_img = all_inputs[i][0]
    true_mask = all_masks[i][0]
    pred_mask = all_preds[i][0]
    binary_pred = (
        pred_mask > 0.5
    ).astype(float)
    heatmap = pred_mask
    overlay = (
        0.6 * input_img
        +
        0.4 * heatmap
    )
    # INPUT
    plt.subplot(
        num_images,
        5,
        i*5 + 1
    )
    plt.title("Input")
    plt.imshow(
        input_img,
        cmap='gray'
    )
    plt.axis('off')
    # GROUND TRUTH
```

```
plt.subplot(
    num_images,
    5,
    i*5 + 2
)
plt.title("Ground Truth")
plt.imshow(
    true_mask,
    cmap='gray'
)
plt.axis('off')
# PREDICTION
plt.subplot(
    num_images,
    5,
    i*5 + 3
)
plt.title("Prediction")
plt.imshow(
    binary_pred,
    cmap='gray'
)
plt.axis('off')
# HEATMAP
plt.subplot(
    num_images,
```

```
        5,
        i*5 + 4
    )
plt.title("Heatmap")
plt.imshow(
    heatmap,
    cmap='jet'
)
plt.axis('off')
# OVERLAY
plt.subplot(
    num_images,
    5,
    i*5 + 5
)
plt.title("Overlay")
plt.imshow(
    overlay,
    cmap='jet'
)
plt.axis('off')
plt.tight_layout()
plt.show()
```



Cell 25-A : Find Common Samples Across 6 / 12 / 18 Days

```
import os

dataset = full_dataset

regions = [

    "EcrinPark",

    "OutsideTheBox",

    "QueyrasPark",

    "Vanoise"

]

common_samples = {}

print(
    "Total Samples:",
    len(dataset.samples)
)

for region in regions:
    files_6 = set()
    files_12 = set()
    files_18 = set()
    for sample in dataset.samples:
        temporal, reg, coh_file, phase_file, mask_file = sample
        filename = os.path.basename(
            coh_file
        )
        if (
            reg == region
            and
            temporal == "6days"
```

```
    ):
        files_6.add(
            filename
        )
    elif (
        reg == region
        and
        temporal == "12days"
    ):
        files_12.add(
            filename
        )
    elif (
        reg == region
        and
        temporal == "18days"
    ):
        files_18.add(
            filename
        )
common = sorted(
    files_6
    &
    files_12
    &
    files_18
)
common_samples[region] = common
print("\n")
```



```

print("=" * 70)

print(
    f"Region : {region}"
)

print("=" * 70)

print(
    "6 Days Samples :",
    len(files_6)
)

print(
    "12 Days Samples:",
    len(files_12)
)

print(
    "18 Days Samples:",
    len(files_18)
)

print(
    "Common Samples :",
    len(common)
)

if len(common) > 0:
    print(
        "Example Sample:",
        common[0]
    )

print("\n")

print("=" * 70)

print(
    "COMMON SAMPLE SEARCH COMPLETED"
)

print("=" * 70)

Total Samples: 13230

=====
Region : EcrinPark

```

```
=====
6 Days Samples : 35
12 Days Samples: 35
18 Days Samples: 35
Common Samples : 35
Example Sample: 106_23.tif
```

```
=====
Region : OutsideTheBox
=====
```

```
6 Days Samples : 84
12 Days Samples: 84
18 Days Samples: 84
Common Samples : 84
Example Sample: 100_65.tif
```

```
=====
Region : QueyrasPark
=====
```

```
6 Days Samples : 69
12 Days Samples: 69
18 Days Samples: 69
Common Samples : 69
Example Sample: 140_50.tif
```

```
=====
Region : Vanoise
=====
```

```
6 Days Samples : 57
12 Days Samples: 57
18 Days Samples: 57
Common Samples : 57
Example Sample: 10_48.tif
```

```
=====
COMMON SAMPLE SEARCH COMPLETED
=====
```

Cell 25-B : Region-wise Temporal Visualization + Analysis

```
import matplotlib.pyplot as plt
import numpy as np
```

```

import torch
import tifffile as tiff
import os

regions = [
    "EcrinPark",
    "OutsideTheBox",
    "QueyrasPark",
    "Vanoise"
]

baselines = [
    "6days",
    "12days",
    "18days"
]

summary_results = []

for region in regions:
    print("\n")
    print("="*90)
    print(f"REGION : {region}")
    print("="*90)

    if len(common_samples[region]) == 0:
        print("No common sample found")
        continue

    sample_name = common_samples[region][
        len(common_samples[region]) // 2
    ]

    print(
        f"Selected Sample : {sample_name}"
    )

    for baseline in baselines:

```

```

selected_sample = None
for sample in dataset.samples:
    temporal, reg, coh_file, phase_file, mask_file = sample
    if (
        temporal == baseline
        and
        reg == region
        and
        os.path.basename(
            coh_file
        ) == sample_name
    ):
        selected_sample = sample
        break
if selected_sample is None:
    continue
temporal, reg, coh_file, phase_file, mask_file =
selected_sample
coherence = tiff.imread(
    coh_file
).astype(np.float32)
phase = tiff.imread(
    phase_file
).astype(np.float32)
gt_mask = tiff.imread(
    mask_file
).astype(np.float32)
cos_phase = np.cos(
    phase
)
sin_phase = np.sin(

```

```

        phase
    )
    image = np.stack(
        [
            coherence,
            cos_phase,
            sin_phase
        ],
        axis=0
    )
    for c in range(
        image.shape[0]
    ):
        image[c] = (
            image[c]
            -
            image[c].mean()
        ) / (
            image[c].std()
            +
            1e-8
        )
    tensor = torch.tensor(
        image,
        dtype=torch.float32
    ).unsqueeze(0).to(device)
    with torch.no_grad():

```

```

        output = model(
            tensor
        )

        prob = torch.sigmoid(
            output
        )

    pred_mask = (
        prob > 0.5
    ).float()

    pred_mask_np = pred_mask.squeeze().cpu().numpy()

    heatmap = prob.squeeze().cpu().numpy()

    # =====
    # ANALYSIS METRICS
    # =====

    coh_mean = coherence.mean()

    phase_std = phase.std()

    gt_area = np.sum(
        gt_mask > 0.5
    )

    pred_area = np.sum(
        pred_mask_np > 0.5
    )

    area_diff = abs(
        gt_area
        -
        pred_area
    )

    intersection = np.sum(
        gt_mask
        *
        pred_mask_np

```

```
)  
dice = (  
    2 * intersection  
) / (  
    gt_mask.sum()  
    +  
    pred_mask_np.sum()  
    +  
    1e-8  
)  
iou = (  
    intersection  
) / (  
    gt_mask.sum()  
    +  
    pred_mask_np.sum()  
    -  
    intersection  
    +  
    1e-8  
)  
precision = intersection / (  
    pred_mask_np.sum()  
    +  
    1e-8  
)
```

```
recall = intersection / (  
    gt_mask.sum()  
    +  
    1e-8  
)  
f1 = (  
    2  
    *  
    precision  
    *  
    recall  
) / (  
    precision  
    +  
    recall  
    +  
    1e-8  
)  
summary_results.append(  
    [  
        region,  
        baseline,  
        coh_mean,  
        phase_std,  
        gt_area,  
        pred_area,
```



```

        area_diff,
        dice,
        iou,
        f1
    ]
)

# =====
# VISUALIZATION
# =====

fig, ax = plt.subplots(
    1,
    5,
    figsize=(18,4)
)

fig.suptitle(
    f"{region} | {baseline}"
)

ax[0].imshow(
    coherence,
    cmap="gray"
)

ax[0].set_title(
    "Input"
)

ax[1].imshow(
    gt_mask,
    cmap="gray"
)

ax[1].set_title(
    "Ground Truth"
)

```

```

ax[2].imshow(
    pred_mask_np,
    cmap="gray"
)

ax[2].set_title(
    "Prediction"
)

ax[3].imshow(
    heatmap,
    cmap="jet"
)

ax[3].set_title(
    "Heatmap"
)

ax[4].imshow(
    coherence,
    cmap="gray"
)

ax[4].imshow(
    pred_mask_np,
    cmap="jet",
    alpha=0.5
)

ax[4].set_title(
    "Overlay"
)

plt.show()

# =====
# TEXTUAL ANALYSIS
# =====

print()

print("-"*70)

print(
    f"{region} | {baseline}"
)

```

```
print("-"*70)

print(
    f"Mean Coherence : {coh_mean:.4f}"
)

print(
    f"Phase Std      : {phase_std:.4f}"
)

print(
    f"GT Area        : {gt_area}"
)

print(
    f"Pred Area       : {pred_area}"
)

print(
    f"Area Diff       : {area_diff}"
)

print(
    f"Dice            : {dice:.4f}"
)

print(
    f"IoU             : {iou:.4f}"
)

print(
    f"F1 Score        : {f1:.4f}"
)

print()

print(
    f"Coherence Trend : {'Stable Terrain' if coh_mean > 0.7
else 'Possible Ground Change'}"
)

print(
    f"Phase Activity  : {'High Deformation' if phase_std > 1.0
else 'Low Deformation'}"
)

print(
    f"Area Error       : {area_diff} pixels"
)
```

```

print()
if dice > 0.85:
    print(
        "Interpretation: Landslide region is accurately
localized with excellent overlap."
    )
elif dice > 0.70:
    print(
        "Interpretation: Landslide region is detected
correctly but boundary mismatch exists."
    )
else:
    print(
        "Interpretation: Significant mismatch exists between
prediction and ground truth."
    )

if coh_mean < 0.60:
    print(
        "Observation: Low coherence suggests unstable
terrain."
    )

if phase_std > 1.0:
    print(
        "Observation: High phase variation indicates stronger
deformation."
    )

if area_diff < 20:
    print(
        "Observation: Predicted landslide area closely matches
actual area."
    )

print()

```

```

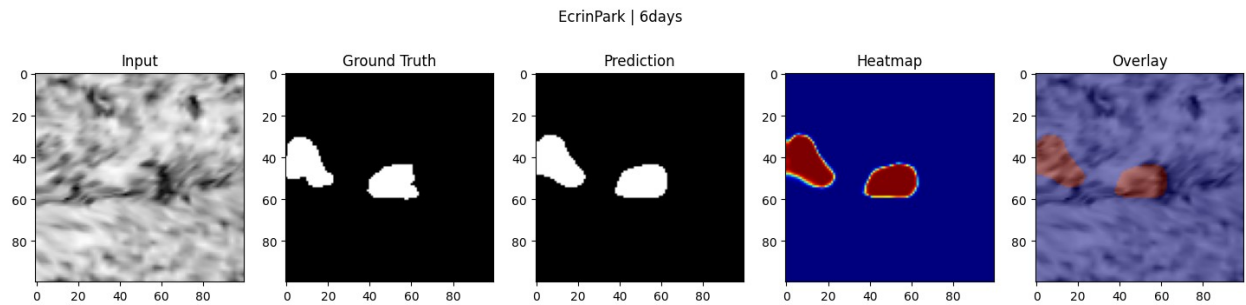
=====
=====
REGION : EcrinPark

```

=====

=====

Selected Sample : 121_5.tif

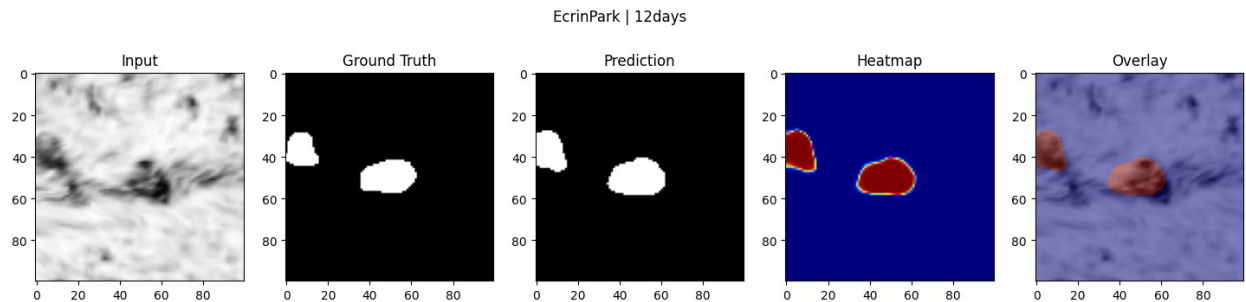


EcrinPark | 6days

Mean Coherence : 0.7441
Phase Std : 1.1995
GT Area : 677
Pred Area : 726
Area Diff : 49
Dice : 0.9252
IoU : 0.8607
F1 Score : 0.9252

Coherence Trend : Stable Terrain
Phase Activity : High Deformation
Area Error : 49 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.
Observation: High phase variation indicates stronger deformation.

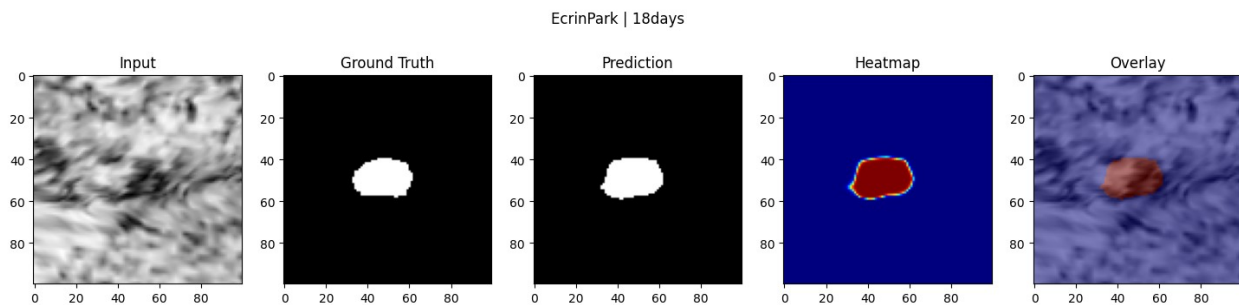


EcrinPark | 12days

Mean Coherence : 0.8279
Phase Std : 2.6657
GT Area : 549
Pred Area : 627
Area Diff : 78
Dice : 0.8861
IoU : 0.7954
F1 Score : 0.8861

Coherence Trend : Stable Terrain
Phase Activity : High Deformation
Area Error : 78 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.
Observation: High phase variation indicates stronger deformation.



EcrinPark | 18days

Mean Coherence : 0.6725
Phase Std : 0.9705
GT Area : 429
Pred Area : 462
Area Diff : 33
Dice : 0.9338
IoU : 0.8758
F1 Score : 0.9338

Coherence Trend : Possible Ground Change
Phase Activity : Low Deformation
Area Error : 33 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.

=====

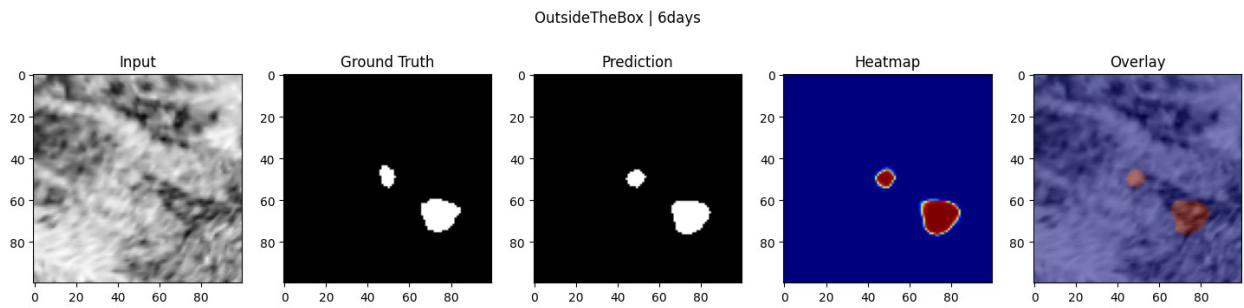
=====

REGION : OutsideTheBox

=====

=====

Selected Sample : 69_77.tif

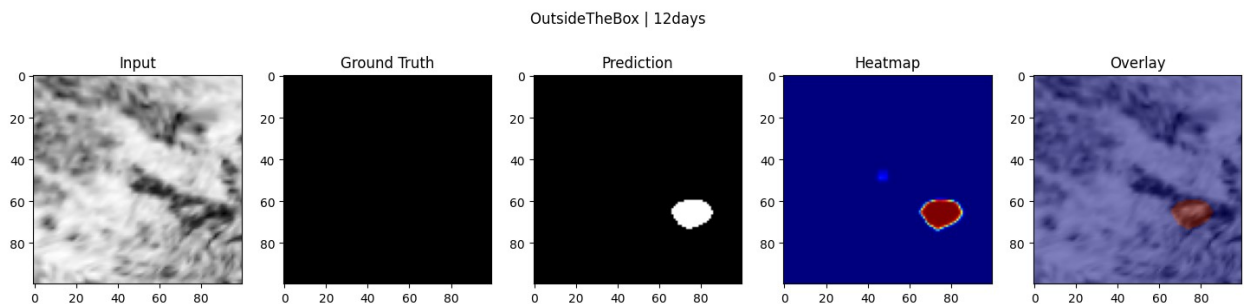


OutsideTheBox | 6days

Mean Coherence : 0.6293
Phase Std : 2.2891
GT Area : 279
Pred Area : 287
Area Diff : 8
Dice : 0.8975
IoU : 0.8141
F1 Score : 0.8975

Coherence Trend : Possible Ground Change
Phase Activity : High Deformation
Area Error : 8 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.
Observation: High phase variation indicates stronger deformation.
Observation: Predicted landslide area closely matches actual area.

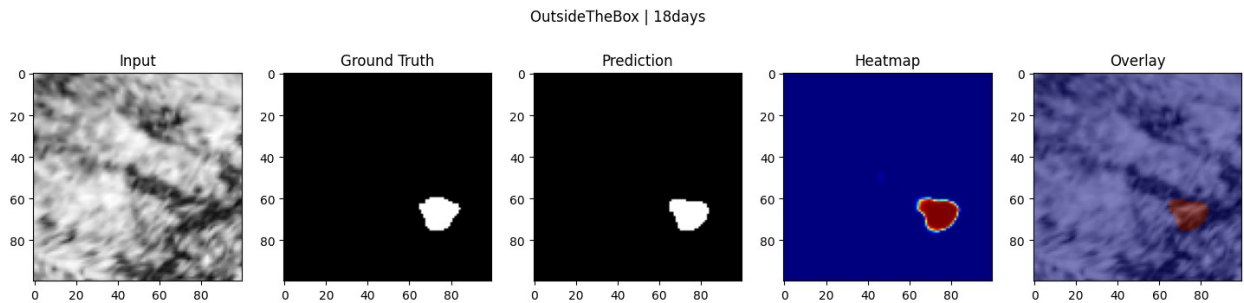


OutsideTheBox | 12days

Mean Coherence : 0.7035
Phase Std : 1.9143
GT Area : 0
Pred Area : 205
Area Diff : 205
Dice : 0.0000
IoU : 0.0000
F1 Score : 0.0000

Coherence Trend : Stable Terrain
Phase Activity : High Deformation
Area Error : 205 pixels

Interpretation: Significant mismatch exists between prediction and ground truth.
Observation: High phase variation indicates stronger deformation.



OutsideTheBox | 18days

Mean Coherence : 0.6219
Phase Std : 0.5559
GT Area : 223
Pred Area : 214
Area Diff : 9
Dice : 0.9291
IoU : 0.8675
F1 Score : 0.9291

Coherence Trend : Possible Ground Change
Phase Activity : Low Deformation
Area Error : 9 pixels

Interpretation: Landslide region is accurately localized with

excellent overlap.
Observation: Predicted landslide area closely matches actual area.

=====

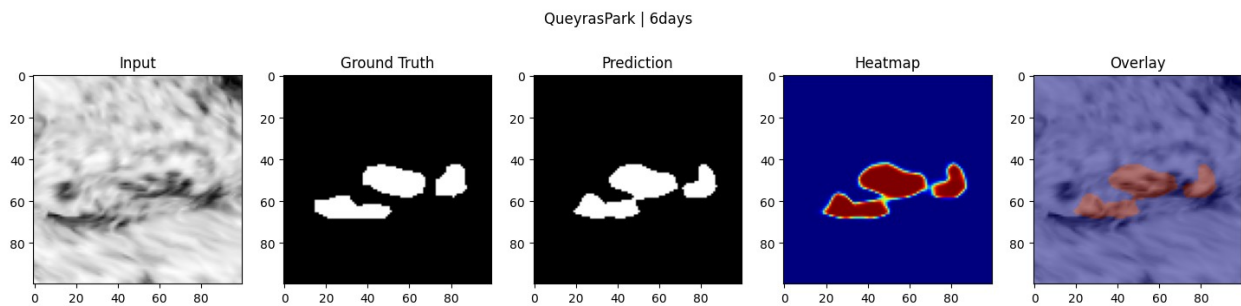
=====

REGION : QueyrasPark

=====

=====

Selected Sample : 169_35.tif



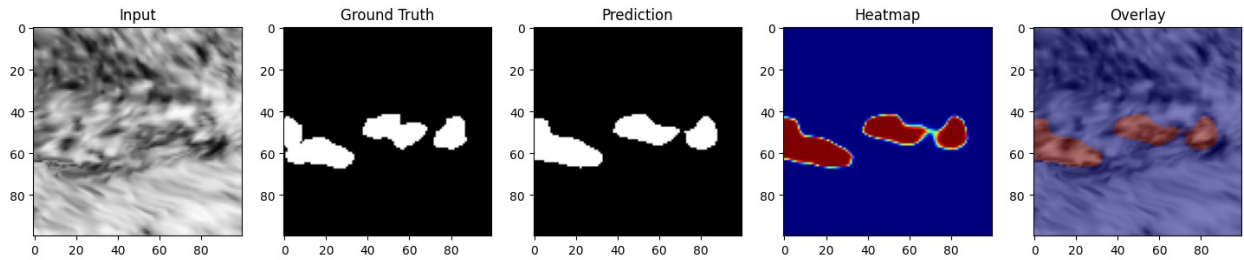
QueyrasPark | 6days

Mean Coherence : 0.7699
Phase Std : 1.8943
GT Area : 805
Pred Area : 791
Area Diff : 14
Dice : 0.8709
IoU : 0.7714
F1 Score : 0.8709

Coherence Trend : Stable Terrain
Phase Activity : High Deformation
Area Error : 14 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.
Observation: High phase variation indicates stronger deformation.
Observation: Predicted landslide area closely matches actual area.

QueyrasPark | 12days



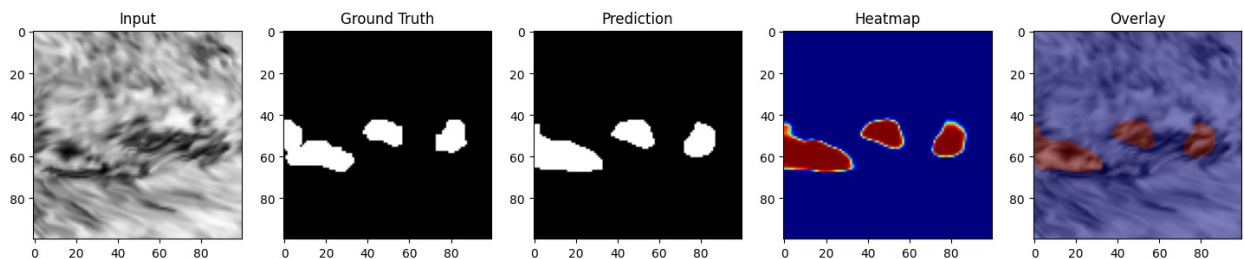
QueyrasPark | 12days

Mean Coherence : 0.6351
 Phase Std : 1.0808
 GT Area : 936
 Pred Area : 911
 Area Diff : 25
 Dice : 0.9258
 IoU : 0.8619
 F1 Score : 0.9258

Coherence Trend : Possible Ground Change
 Phase Activity : High Deformation
 Area Error : 25 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.
 Observation: High phase variation indicates stronger deformation.

QueyrasPark | 18days



QueyrasPark | 18days

Mean Coherence : 0.6457
 Phase Std : 0.6860
 GT Area : 792
 Pred Area : 829

Area Diff : 37
Dice : 0.8587
IoU : 0.7524
F1 Score : 0.8587

Coherence Trend : Possible Ground Change
Phase Activity : Low Deformation
Area Error : 37 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.

=====

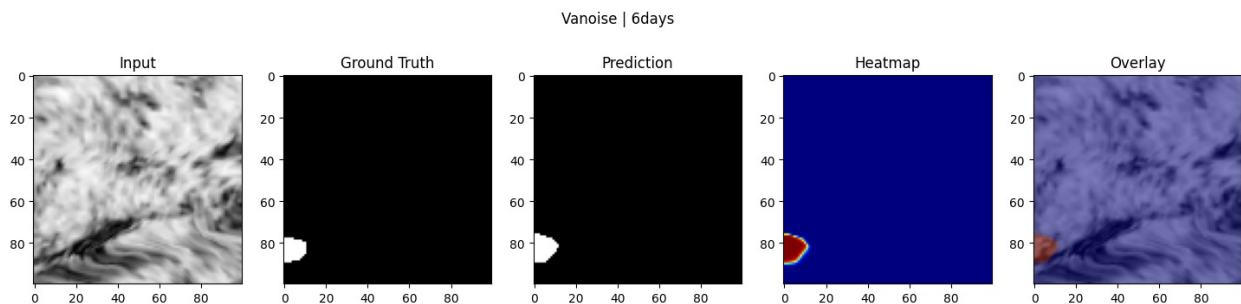
=====

REGION : Vanoise

=====

=====

Selected Sample : 29_24.tif



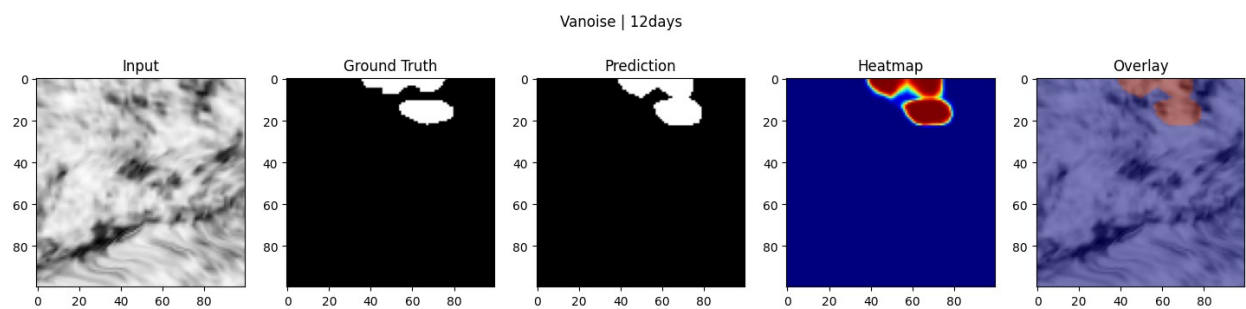
Vanoise | 6days

Mean Coherence : 0.6977
Phase Std : 2.2860
GT Area : 111
Pred Area : 125
Area Diff : 14
Dice : 0.9068
IoU : 0.8295
F1 Score : 0.9068

Coherence Trend : Possible Ground Change
Phase Activity : High Deformation
Area Error : 14 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.

Observation: High phase variation indicates stronger deformation.
Observation: Predicted landslide area closely matches actual area.

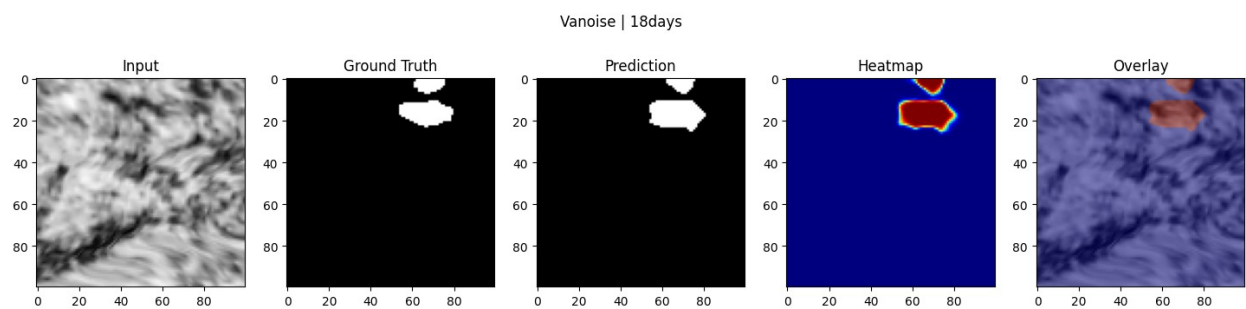


Vanoise | 12days

Mean Coherence : 0.7447
Phase Std : 0.7304
GT Area : 468
Pred Area : 531
Area Diff : 63
Dice : 0.8609
IoU : 0.7557
F1 Score : 0.8609

Coherence Trend : Stable Terrain
Phase Activity : Low Deformation
Area Error : 63 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.



Vanoise | 18days

Mean Coherence : 0.6359
Phase Std : 2.7451
GT Area : 335
Pred Area : 388
Area Diff : 53
Dice : 0.8658
IoU : 0.7634
F1 Score : 0.8658

Coherence Trend : Possible Ground Change
Phase Activity : High Deformation
Area Error : 53 pixels

Interpretation: Landslide region is accurately localized with excellent overlap.
Observation: High phase variation indicates stronger deformation.

Cell 26 : Final Research Analysis Table

```
import pandas as pd
import numpy as np

columns = [
    "Region",
    "Baseline",
    "Coh Mean",
    "Phase Std",
    "GT Area",
    "Pred Area",
    "Area Diff",
    "Dice",
    "IoU",
    "F1"
]

df = pd.DataFrame(
```

```

        summary_results,
        columns=columns
    )
    print("\n")
    print("=" * 120)
    print("COMPLETE REGION-WISE TEMPORAL ANALYSIS")
    print("=" * 120)
    display(df)
    print("\n")
    print("=" * 120)
    print("REGION-WISE STATISTICAL SUMMARY")
    print("=" * 120)
    display(
        df.groupby(
            "Region"
        )[
            "Coh Mean",
            "Phase Std",
            "GT Area",
            "Pred Area",
            "Area Diff",
            "Dice",
            "IoU",
            "F1"
        ].mean()
    )

```

=====

=====

COMPLETE REGION-WISE TEMPORAL ANALYSIS

=====

=====

	Region	Baseline	Coh Mean	Phase Std	GT Area	Pred Area	\
0	EcrinPark	6days	0.744078	1.199477	677	726	
1	EcrinPark	12days	0.827928	2.665709	549	627	
2	EcrinPark	18days	0.672502	0.970503	429	462	
3	OutsideTheBox	6days	0.629318	2.289106	279	287	
4	OutsideTheBox	12days	0.703515	1.914295	0	205	
5	OutsideTheBox	18days	0.621899	0.555948	223	214	
6	QueyrasPark	6days	0.769927	1.894331	805	791	
7	QueyrasPark	12days	0.635109	1.080801	936	911	
8	QueyrasPark	18days	0.645689	0.686026	792	829	
9	Vanoise	6days	0.697740	2.286012	111	125	
10	Vanoise	12days	0.744709	0.730421	468	531	
11	Vanoise	18days	0.635876	2.745089	335	388	

	Area Diff	Dice	IoU	F1
0	49	0.925160	0.860743	0.925160
1	78	0.886054	0.795420	0.886054
2	33	0.933782	0.875789	0.933782
3	8	0.897527	0.814103	0.897527
4	205	0.000000	0.000000	0.000000
5	9	0.929062	0.867521	0.929062
6	14	0.870927	0.771365	0.870927
7	25	0.925826	0.861895	0.925826
8	37	0.858729	0.752432	0.858729
9	14	0.906780	0.829457	0.906780
10	63	0.860861	0.755712	0.860861
11	53	0.865837	0.763415	0.865837

=====

=====

REGION-WISE STATISTICAL SUMMARY

=====

=====

	Coh Mean	Phase Std	GT Area	Pred Area	Area Diff
\					
Region					
EcrinPark	0.748169	1.611896	551.666667	605.000000	53.333333
OutsideTheBox	0.651577	1.586450	167.333333	235.333333	74.000000

QueyrasPark	0.683575	1.220386	844.333333	843.666667	25.333333
Vanoise	0.692775	1.920507	304.666667	348.000000	43.333333

	Dice	IoU	F1
Region			
EcrinPark	0.914999	0.843984	0.914999
OutsideTheBox	0.608863	0.560541	0.608863
QueyrasPark	0.885161	0.795231	0.885161
Vanoise	0.877826	0.782861	0.877826

Temporal Change Analysis

```
regions = [
    "EcrinPark",
    "OutsideTheBox",
    "QueyrasPark",
    "Vanoise"
]

change_results = []
for region in regions:
    temp_df = df[
        df["Region"] == region
    ]
    temp_df = temp_df.set_index(
        "Baseline"
    )
    try:
        # =====
        # COHERENCE
        # =====

        coh_6 = temp_df.loc[
            "6days",
```



```

        "Coh Mean"
    ]

    coh_12 = temp_df.loc[
        "12days",
        "Coh Mean"
    ]

    coh_18 = temp_df.loc[
        "18days",
        "Coh Mean"
    ]

    # =====
    # PHASE
    # =====

    phase_6 = temp_df.loc[
        "6days",
        "Phase Std"
    ]

    phase_12 = temp_df.loc[
        "12days",
        "Phase Std"
    ]

    phase_18 = temp_df.loc[
        "18days",
        "Phase Std"
    ]

    # =====
    # DICE
    # =====

    dice_6 = temp_df.loc[
        "6days",
        "Dice"
    ]

    dice_12 = temp_df.loc[
        "12days",
        "Dice"
    ]

    dice_18 = temp_df.loc[
        "18days",
        "Dice"
    ]

```

```

# =====
# IOU
# =====

iou_6 = temp_df.loc[
    "6days",
    "IoU"
]

iou_12 = temp_df.loc[
    "12days",
    "IoU"
]

iou_18 = temp_df.loc[
    "18days",
    "IoU"
]

# =====
# F1
# =====

f1_6 = temp_df.loc[
    "6days",
    "F1"
]

f1_12 = temp_df.loc[
    "12days",
    "F1"
]

f1_18 = temp_df.loc[
    "18days",
    "F1"
]

# =====
# GT AREA
# =====

gt_6 = temp_df.loc[
    "6days",
    "GT Area"
]

gt_12 = temp_df.loc[
    "12days",

```

```

        "GT Area"
    ]

    gt_18 = temp_df.loc[
        "18days",
        "GT Area"
    ]

    # =====
    # PRED AREA
    # =====

    pred_6 = temp_df.loc[
        "6days",
        "Pred Area"
    ]

    pred_12 = temp_df.loc[
        "12days",
        "Pred Area"
    ]

    pred_18 = temp_df.loc[
        "18days",
        "Pred Area"
    ]

    # =====
    # AREA DIFFERENCE
    # =====

    diff_6 = temp_df.loc[
        "6days",
        "Area Diff"
    ]

    diff_12 = temp_df.loc[
        "12days",
        "Area Diff"
    ]

    diff_18 = temp_df.loc[
        "18days",
        "Area Diff"
    ]

    change_results.append(
        [

```

```

        region,

        coh_12 - coh_6,
        coh_18 - coh_12,
        coh_18 - coh_6,

        phase_12 - phase_6,
        phase_18 - phase_12,
        phase_18 - phase_6,

        dice_12 - dice_6,
        dice_18 - dice_12,
        dice_18 - dice_6,

        iou_12 - iou_6,
        iou_18 - iou_12,
        iou_18 - iou_6,

        f1_12 - f1_6,
        f1_18 - f1_12,
        f1_18 - f1_6,

        gt_12 - gt_6,
        gt_18 - gt_12,
        gt_18 - gt_6,

        pred_12 - pred_6,
        pred_18 - pred_12,
        pred_18 - pred_6,

        diff_12 - diff_6,
        diff_18 - diff_12,
        diff_18 - diff_6
    ]

)

except:

    pass

```

Change Summary Table

```

change_df = pd.DataFrame(
    change_results,
    columns=[

```

```

"Region",

# =====
# COHERENCE
# =====

"Coh  $\Delta$  6→12",

"Coh  $\Delta$  12→18",

"Coh  $\Delta$  6→18",

# =====
# PHASE
# =====

"Phase  $\Delta$  6→12",

"Phase  $\Delta$  12→18",

"Phase  $\Delta$  6→18",

# =====
# DICE
# =====

"Dice  $\Delta$  6→12",

"Dice  $\Delta$  12→18",

"Dice  $\Delta$  6→18",

# =====
# IoU
# =====

"IoU  $\Delta$  6→12",

"IoU  $\Delta$  12→18",

"IoU  $\Delta$  6→18",

# =====
# F1
# =====

"F1  $\Delta$  6→12",

"F1  $\Delta$  12→18",

"F1  $\Delta$  6→18",

```

```

# =====
# GT AREA
# =====

"GT Area  $\Delta$  6→12",

"GT Area  $\Delta$  12→18",

"GT Area  $\Delta$  6→18",

# =====
# PRED AREA
# =====

"Pred Area  $\Delta$  6→12",

"Pred Area  $\Delta$  12→18",

"Pred Area  $\Delta$  6→18",

# =====
# AREA DIFFERENCE
# =====

"Area Diff  $\Delta$  6→12",

"Area Diff  $\Delta$  12→18",

"Area Diff  $\Delta$  6→18"

]

)

print("\n")

print("=" * 140)

print("TEMPORAL CHANGE ANALYSIS")

print("=" * 140)

display(change_df)

```

```

=====
=====
TEMPORAL CHANGE ANALYSIS

```

=====						
=====						
	Region	Coh Δ 6→12	Coh Δ 12→18	Coh Δ 6→18	Phase Δ 6→12 \	
0	EcrinPark	0.083850	-0.155425	-0.071575	1.466232	
1	OutsideTheBox	0.074197	-0.081616	-0.007419	-0.374811	
2	QueyrasPark	-0.134818	0.010580	-0.124238	-0.813530	
3	Vanoise	0.046970	-0.108833	-0.061864	-1.555591	
	Phase Δ 12→18	Phase Δ 6→18	Dice Δ 6→12	Dice Δ 12→18	Dice Δ 6→18	
...	\					
0	-1.695206	-0.228974	-0.039106	0.047728	0.008622	
...						
1	-1.358347	-1.733158	-0.897527	0.929062	0.031535	
...						
2	-0.394774	-1.208304	0.054898	-0.067096	-0.012198	
...						
3	2.014668	0.459077	-0.045919	0.004976	-0.040943	
...						
	F1 Δ 6→18	GT Area Δ 6→12	GT Area Δ 12→18	GT Area Δ 6→18 \		
0	0.008622	-128	-120	-248		
1	0.031535	-279	223	-56		
2	-0.012198	131	-144	-13		
3	-0.040943	357	-133	224		
	Pred Area Δ 6→12	Pred Area Δ 12→18	Pred Area Δ 6→18	Area Diff Δ		
6→12 \						
0	-99	-165	-264			
29						
1	-82	9	-73			
197						
2	120	-82	38			
11						
3	406	-143	263			
49						
	Area Diff Δ 12→18	Area Diff Δ 6→18				
0	-45	-16				
1	-196	1				
2	12	23				
3	-10	39				
[4 rows x 25 columns]						

6 DAYS ANALYSIS

cell 27A -> Find Two-Month Date Pairs

```
import os

regions = [
    "EcrinPark",
    "OutsideTheBox",
    "QueyrasPark",
    "Vanoise"
]

selected_pairs = {}

for region in regions:
    date_folders = []
    for sample in dataset.samples:
        temporal, reg, coh_file, _, _ = sample
        if (
            temporal == "6days"
            and
            reg == region
        ):
            folder_name = os.path.basename(
                os.path.dirname(
                    os.path.dirname(
                        coh_file
                    )
                )
            )
```



```
        date_folders.append(
            folder_name
        )
date_folders = sorted(
    list(
        set(date_folders)
    )
)
if len(date_folders) < 4:
    continue
pair1 = (
    date_folders[0],
    date_folders[8]
)
pair2 = (
    date_folders[4],
    date_folders[12]
)
selected_pairs[region] = [
    pair1,
    pair2
]
print()
print("="*80)
print(region)
print("="*80)
print(
```

```

        "Pair 1 :",
        pair1[0],
        "vs",
        pair1[1]
    )
    print(
        "Pair 2 :",
        pair2[0],
        "vs",
        pair2[1]
    )

```

```

=====
=====
EcrinPark
=====
=====
Pair 1 : 01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018
Pair 2 : 07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018
=====
=====
OutsideTheBox
=====
=====
Pair 1 : 01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018
Pair 2 : 07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018
=====
=====
QueyrasPark
=====
=====
Pair 1 : 01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018
Pair 2 : 07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018
=====
=====
Vanoise
=====
=====

```

=====

Pair 1 : 01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018

Pair 2 : 07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018

cell 27B - Visual Comparison (8 Comparisons / 16 Images)

```
import matplotlib.pyplot as plt
import tiffio as tiff
import numpy as np
import torch

temporal_results = []

for region in regions:
    if region not in selected_pairs:
        continue

    print("\n")
    print("="*100)
    print(region)
    print("="*100)

    for pair_id, pair in enumerate(
        selected_pairs[region]
    ):
        print()
        print(
            f"Comparison {pair_id+1}"
        )
        print(
            pair[0],
            "vs",
            pair[1]
        )

        selected_images = []

        for target_folder in pair:
            found = False
```

```

        for sample in dataset.samples:
            temporal, reg, coh_file, phase_file, mask_file =
sample
                folder_name = os.path.basename(
                    os.path.dirname(
                        os.path.dirname(
                            coh_file
                        )
                    )
                )
                if (
                    temporal == "6days"
                    and
                    reg == region
                    and
                    folder_name == target_folder
                ):
                    selected_images.append(
                        sample
                    )
                    found = True
                    break
            if not found:
                selected_images.append(
                    None
                )
    fig, ax = plt.subplots(
        2,

```

```

    5,
    figsize=(18,8)
)
for row, sample in enumerate(
    selected_images
):
    if sample is None:
        continue

    temporal, reg, coh_file, phase_file, mask_file = sample
    coherence = tiff.imread(
        coh_file
    ).astype(np.float32)
    phase = tiff.imread(
        phase_file
    ).astype(np.float32)
    gt_mask = tiff.imread(
        mask_file
    ).astype(np.float32)
    cos_phase = np.cos(
        phase
    )
    sin_phase = np.sin(
        phase
    )
    image = np.stack(
        [

```

```

        coherence,
        cos_phase,
        sin_phase
    ],
    axis=0
)
for c in range(
    image.shape[0]
):
    image[c] = (
        image[c]
        -
        image[c].mean()
    ) / (
        image[c].std()
        +
        1e-8
    )
tensor = torch.tensor(
    image,
    dtype=torch.float32
).unsqueeze(0).to(device)
with torch.no_grad():
    output = model(
        tensor
    )
    prob = torch.sigmoid(

```

```

        output
    )
pred = (
    prob > 0.5
).float()
pred_np = pred.squeeze().cpu().numpy()
heatmap = prob.squeeze().cpu().numpy()

ax[row][0].imshow(
    coherence,
    cmap="gray"
)

ax[row][0].set_title(
    "Input"
)

ax[row][1].imshow(
    gt_mask,
    cmap="gray"
)

ax[row][1].set_title(
    "GT"
)

ax[row][2].imshow(
    pred_np,
    cmap="gray"
)

ax[row][2].set_title(
    "Prediction"
)

ax[row][3].imshow(
    heatmap,
    cmap="jet"
)

ax[row][3].set_title(
    "Heatmap"
)

ax[row][4].imshow(
    coherence,

```

```

        cmap="gray"
    )

    ax[row][4].imshow(

        pred_np,

        cmap="jet",

        alpha=0.5

    )

    ax[row][4].set_title(
        "Overlay"
    )

plt.show()

```

=====

=====

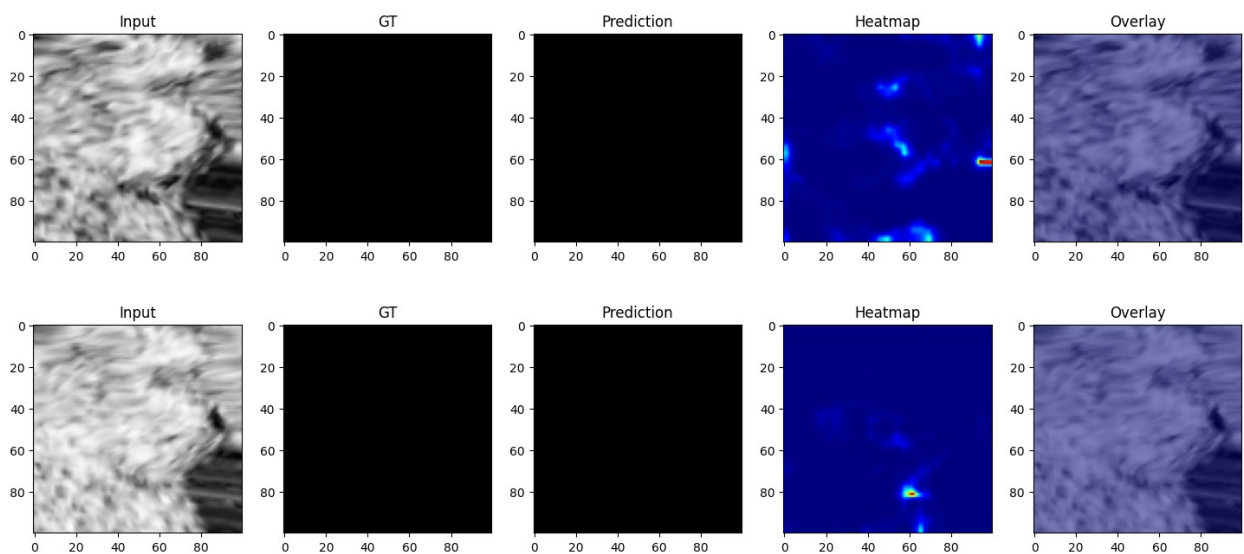
EcrinPark

=====

=====

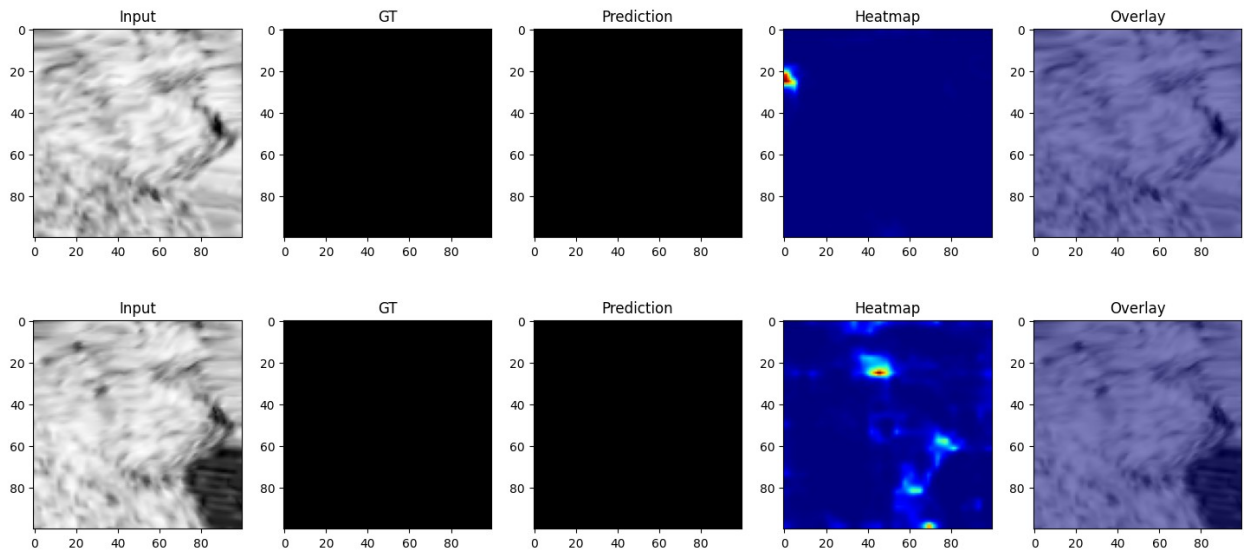
Comparison 1

01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018



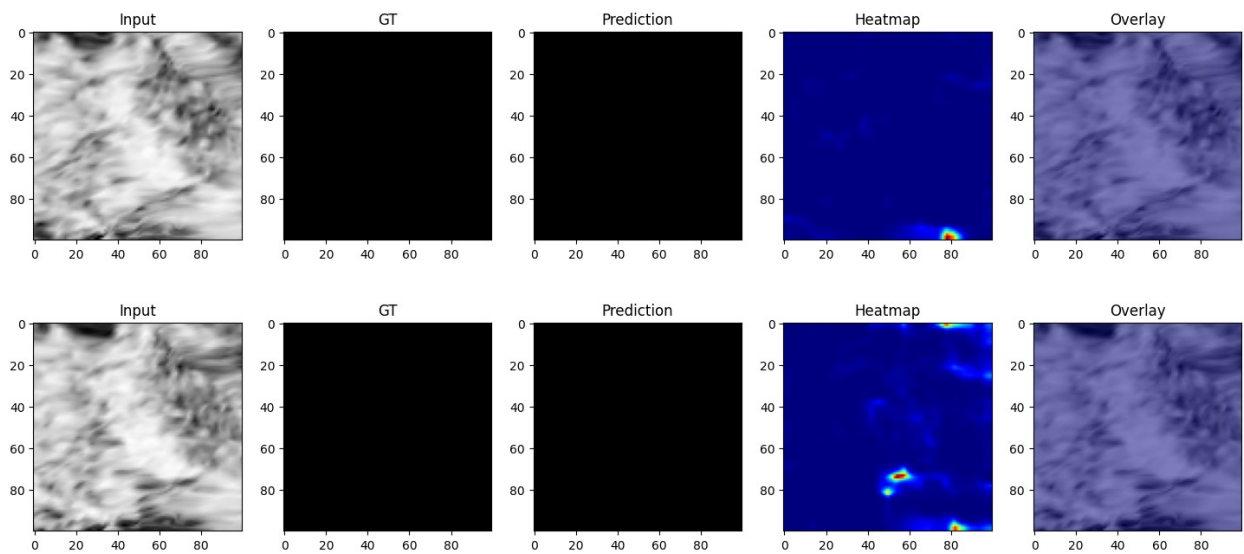
Comparison 2

07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018

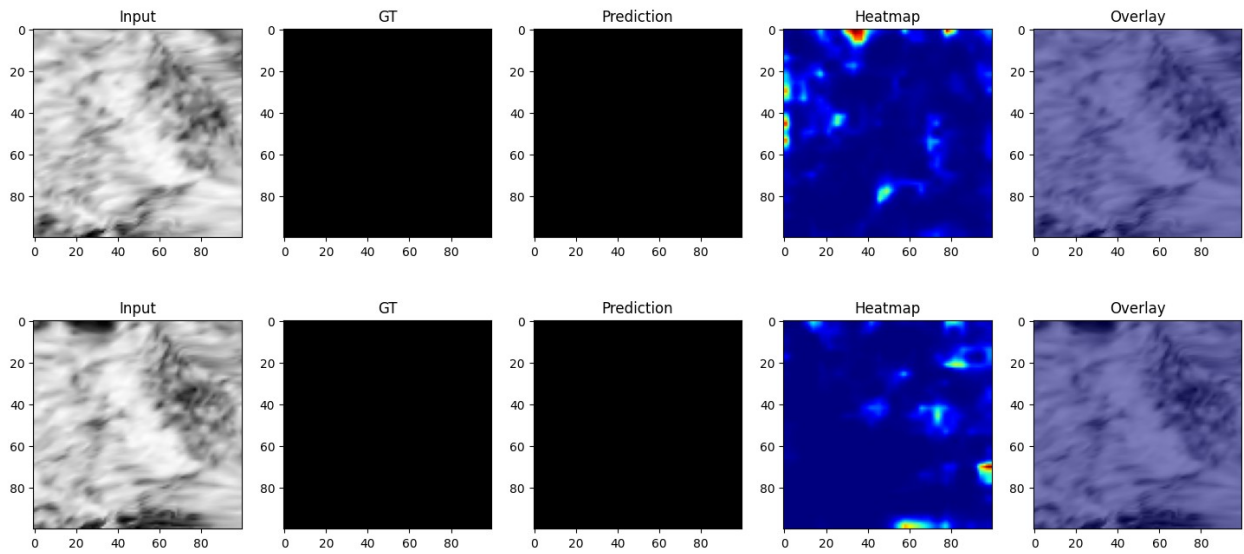


=====
 =====
 OutsideTheBox
 =====
 =====

Comparison 1
 01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018



Comparison 2
 07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018



=====

=====

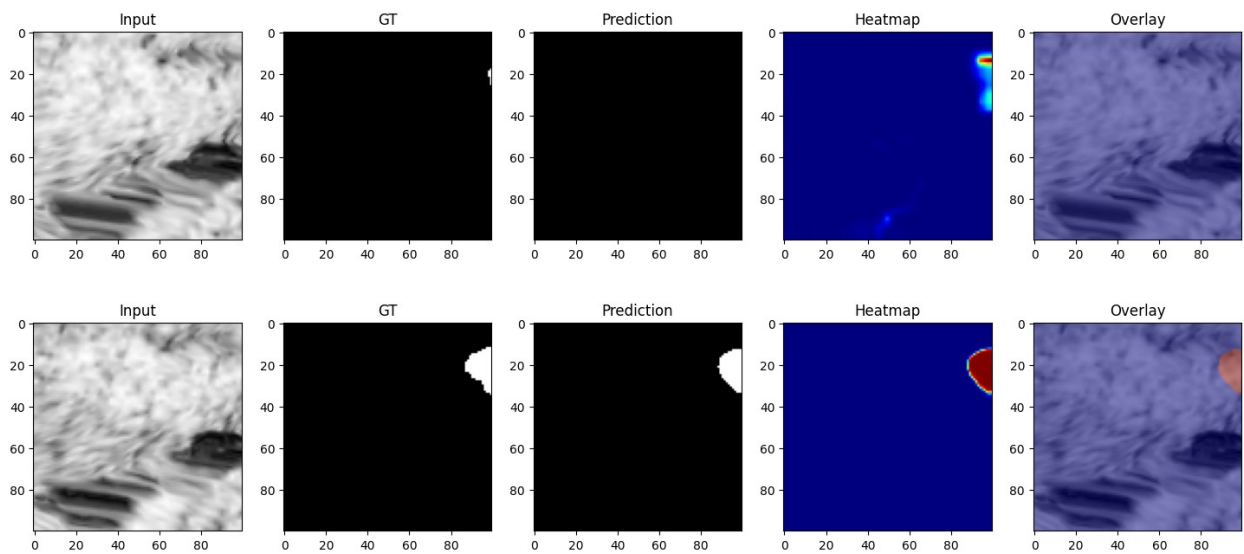
QueyrasPark

=====

=====

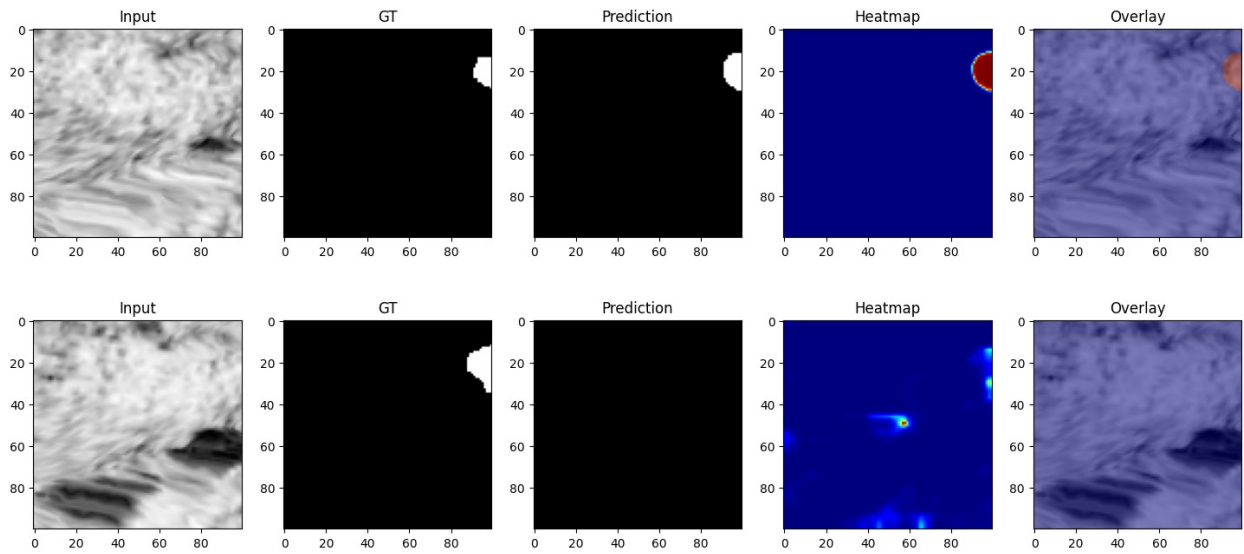
Comparison 1

01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018



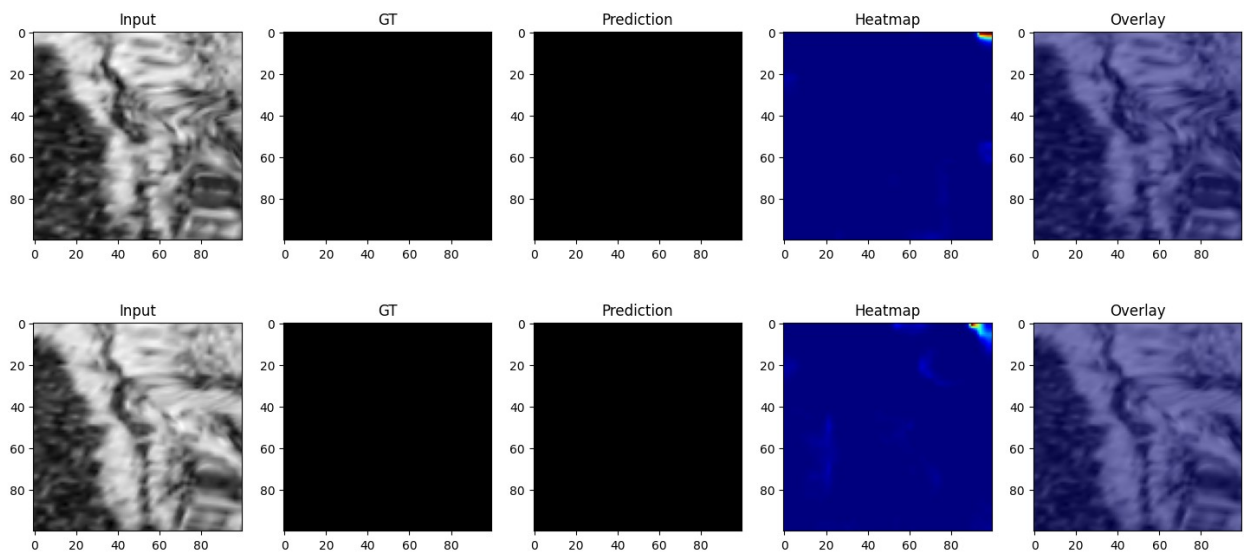
Comparison 2

07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018

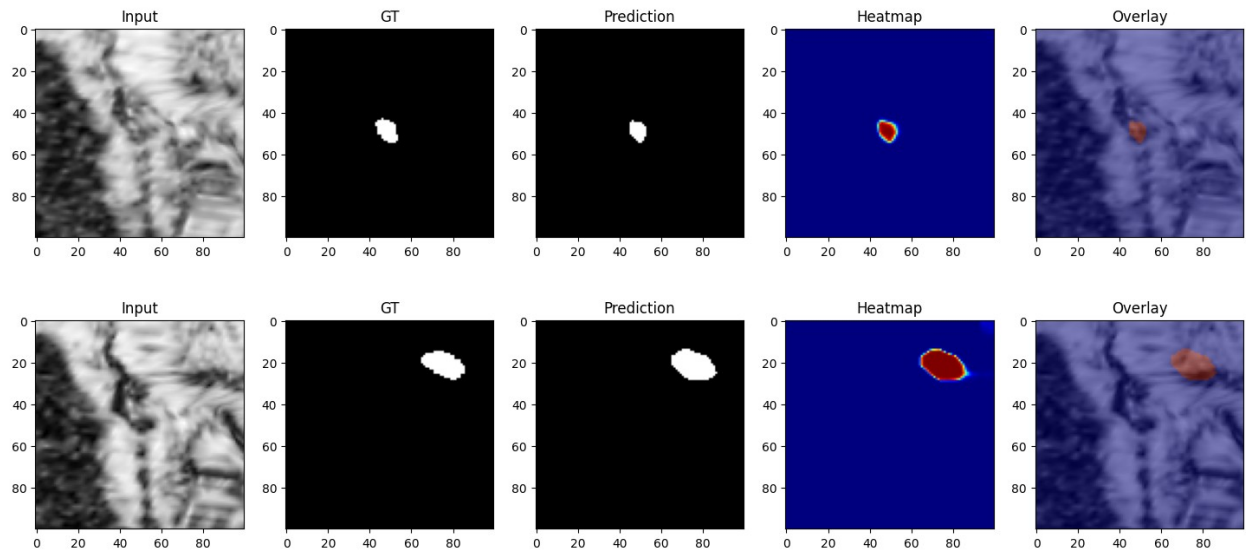


Vanoise

Comparison 1
01Aug2018_07Aug2018 vs 13Aug2018_19Aug2018



Comparison 2
07Aug2018_13Aug2018 vs 19Aug2018_25Aug2018



cell 27C - Two-Month Temporal Evolution Metrics Table

```
import pandas as pd
import numpy as np
import tifffile as tiff
import torch
import os

temporal_analysis = []

for region in regions:
    if region not in selected_pairs:
        continue

    for pair_id, pair in enumerate(
        selected_pairs[region]
    ):
        pair_metrics = []

        for target_folder in pair:
            sample_found = None

            for sample in dataset.samples:
```

```

temporal, reg, coh_file, phase_file, mask_file =
sample
    folder_name = os.path.basename(
        os.path.dirname(
            os.path.dirname(
                coh_file
            )
        )
    )
    if (
        temporal == "6days"
        and
        reg == region
        and
        folder_name == target_folder
    ):
        sample_found = sample
        break
    if sample_found is None:
        continue
temporal_, reg_, coh_file, phase_file, mask_file =
sample_found
    coherence = tiff.imread(
        coh_file
    ).astype(np.float32)
    phase = tiff.imread(
        phase_file
    ).astype(np.float32)
    gt_mask = tiff.imread(
        mask_file
    ).astype(np.float32)

```

```
cos_phase = np.cos(
    phase
)

sin_phase = np.sin(
    phase
)

image = np.stack(
    [
        coherence,
        cos_phase,
        sin_phase
    ],
    axis=0
)

for c in range(
    image.shape[0]
):
    image[c] = (
        image[c]
        -
        image[c].mean()
    ) / (
        image[c].std()
        +
        1e-8
    )

tensor = torch.tensor(
    image,
    dtype=torch.float32
```

```

).unsqueeze(0).to(device)
with torch.no_grad():
    output = model(
        tensor
    )
    prob = torch.sigmoid(
        output
    )
pred_mask = (
    prob > 0.5
).float()
pred_np = pred_mask.squeeze().cpu().numpy()
intersection = np.sum(
    gt_mask
    *
    pred_np
)
dice = (
    2 * intersection
) / (
    gt_mask.sum()
    +
    pred_np.sum()
    +
    1e-8
)
iou = (

```

```

        intersection
    ) / (
        gt_mask.sum()
        +
        pred_np.sum()
        -
        intersection
        +
        1e-8
    )
precision = intersection / (
    pred_np.sum()
    +
    1e-8
)
recall = intersection / (
    gt_mask.sum()
    +
    1e-8
)
f1 = (
    2
    *
    precision
    *
    recall

```



```

        ) / (
            precision
            +
            recall
            +
            1e-8
        )
    pair_metrics.append(
        [
            target_folder,
            coherence.mean(),
            phase.std(),
            np.sum(gt_mask > 0.5),
            np.sum(pred_np > 0.5),
            dice,
            iou,
            f1
        ]
    )
    if len(pair_metrics) == 2:
        temporal_analysis.append(
            [
                region,
                pair_id + 1
            ]
        )
        +

```

```
        pair_metrics[0]
        +
        pair_metrics[1]
    )
columns = [
    "Region",
    "Comparison",
    "Date A",
    "Coh A",
    "Phase A",
    "GT A",
    "Pred A",
    "Dice A",
    "IoU A",
    "F1 A",
    "Date B",
    "Coh B",
    "Phase B",
    "GT B",
    "Pred B",
    "Dice B",
    "IoU B",
    "F1 B"
]
temporal_df = pd.DataFrame(
    temporal_analysis,
```

```

        columns=columns
    )

    print("="*120)
    print("TWO-MONTH TEMPORAL EVOLUTION TABLE")
    print("="*120)

    display(temporal_df)

```

```

=====
=====
TWO-MONTH TEMPORAL EVOLUTION TABLE
=====
=====

```

	Region	Comparison		Date A	Coh A	Phase A
GT A \						
0	EcrinPark	1	01Aug2018_07Aug2018	0.643472	2.666620	
0						
1	EcrinPark	2	07Aug2018_13Aug2018	0.760157	0.306982	
0						
2	OutsideTheBox	1	01Aug2018_07Aug2018	0.717441	0.271199	
0						
3	OutsideTheBox	2	07Aug2018_13Aug2018	0.730428	0.439796	
0						
4	QueyrasPark	1	01Aug2018_07Aug2018	0.713728	0.552841	
11						
5	QueyrasPark	2	07Aug2018_13Aug2018	0.716277	1.444131	
106						
6	Vanoise	1	01Aug2018_07Aug2018	0.494646	1.252581	
0						
7	Vanoise	2	07Aug2018_13Aug2018	0.574972	0.781156	
89						

	Pred A	Dice A	IoU A	F1 A	Date B	Coh B
\						
0	0	0.000000	0.000000	0.000000	13Aug2018_19Aug2018	0.700833
1	0	0.000000	0.000000	0.000000	19Aug2018_25Aug2018	0.726017
2	0	0.000000	0.000000	0.000000	13Aug2018_19Aug2018	0.703055
3	0	0.000000	0.000000	0.000000	19Aug2018_25Aug2018	0.724768
4	0	0.000000	0.000000	0.000000	13Aug2018_19Aug2018	0.687648
5	135	0.879668	0.785185	0.879668	19Aug2018_25Aug2018	0.716613
6	0	0.000000	0.000000	0.000000	13Aug2018_19Aug2018	0.543626

7	64	0.836601	0.719101	0.836601	19Aug2018_25Aug2018	0.574721
---	----	----------	----------	----------	---------------------	----------

	Phase B	GT B	Pred B	Dice B	IoU B	F1 B
0	0.818495	0	0	0.000000	0.000000	0.000000
1	0.701000	0	0	0.000000	0.000000	0.000000
2	0.567300	0	0	0.000000	0.000000	0.000000
3	0.499065	0	0	0.000000	0.000000	0.000000
4	1.061664	189	182	0.932615	0.873737	0.932615
5	2.543961	183	0	0.000000	0.000000	0.000000
6	0.759732	0	0	0.000000	0.000000	0.000000
7	1.495364	192	232	0.905660	0.827586	0.905660

CELL 27D -> Automatic Temporal Change Analysis

```
for i in range(
    len(temporal_df)
):
    row = temporal_df.iloc[i]
    print("\n")
    print("="*100)
    print(
        f"Region : {row['Region']}"
    )
    print(
        f"Comparison : {row['Comparison']}"
    )
    print("="*100)
    print()
    print(
        f>Date A : {row['Date A']}"
```

```
)  
print(  
    f"Date B : {row['Date B']}"  
)  
print()  
coh_change = (  
    row["Coh B"]  
    -  
    row["Coh A"]  
)  
phase_change = (  
    row["Phase B"]  
    -  
    row["Phase A"]  
)  
dice_change = (  
    row["Dice B"]  
    -  
    row["Dice A"]  
)  
iou_change = (  
    row["IoU B"]  
    -  
    row["IoU A"]  
)  
f1_change = (  

```

```
        row["F1 B"]
        -
        row["F1 A"]
    )
gt_change = (
    row["GT B"]
    -
    row["GT A"]
)
pred_change = (
    row["Pred B"]
    -
    row["Pred A"]
)
print(
    f"Coherence Change : {coh_change:.4f}"
)
print(
    f"Phase Change      : {phase_change:.4f}"
)
print(
    f"GT Area Change    : {gt_change}"
)
print(
    f"Pred Area Change   : {pred_change}"
)
print(
    f"Dice Change        : {dice_change:.4f}"
)
print(
    f"IoU Change         : {iou_change:.4f}"
)
```

```
print(
    f"F1 Change          : {f1_change:.4f}"
)
print()
if coh_change < 0:
    print(
        "Observation: Ground stability decreased over time."
    )
else:
    print(
        "Observation: Ground remained relatively stable."
    )
if phase_change > 0:
    print(
        "Observation: Surface deformation increased."
    )
else:
    print(
        "Observation: Surface deformation reduced."
    )
if dice_change > 0:
    print(
        "Observation: Segmentation performance improved."
    )
else:
    print(
        "Observation: Segmentation performance degraded."
    )
if abs(gt_change) > 0:
    print(
        "Observation: Landslide extent changed over time."
    )
print()
```

```
=====
=====
Region : EcrinPark
Comparison : 1
=====
=====
```

```
Date A : 01Aug2018_07Aug2018
Date B : 13Aug2018_19Aug2018
```

```
Coherence Change : 0.0574
Phase Change      : -1.8481
GT Area Change    : 0
Pred Area Change  : 0
Dice Change       : 0.0000
IoU Change        : 0.0000
F1 Change         : 0.0000
```

```
Observation: Ground remained relatively stable.
Observation: Surface deformation reduced.
Observation: Segmentation performance degraded.
```

```
=====
=====
Region : EcrinPark
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_13Aug2018
Date B : 19Aug2018_25Aug2018
```

```
Coherence Change : -0.0341
Phase Change      : 0.3940
GT Area Change    : 0
Pred Area Change  : 0
Dice Change       : 0.0000
IoU Change        : 0.0000
F1 Change         : 0.0000
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation increased.
Observation: Segmentation performance degraded.
```

```
=====
```



```
=====  
Region : OutsideTheBox  
Comparison : 1  
=====
```

```
=====  
Date A : 01Aug2018_07Aug2018  
Date B : 13Aug2018_19Aug2018
```

```
Coherence Change : -0.0144  
Phase Change      : 0.2961  
GT Area Change    : 0  
Pred Area Change  : 0  
Dice Change       : 0.0000  
IoU Change        : 0.0000  
F1 Change         : 0.0000
```

```
Observation: Ground stability decreased over time.  
Observation: Surface deformation increased.  
Observation: Segmentation performance degraded.
```

```
=====  
Region : OutsideTheBox  
Comparison : 2  
=====
```

```
=====  
Date A : 07Aug2018_13Aug2018  
Date B : 19Aug2018_25Aug2018
```

```
Coherence Change : -0.0057  
Phase Change      : 0.0593  
GT Area Change    : 0  
Pred Area Change  : 0  
Dice Change       : 0.0000  
IoU Change        : 0.0000  
F1 Change         : 0.0000
```

```
Observation: Ground stability decreased over time.  
Observation: Surface deformation increased.  
Observation: Segmentation performance degraded.
```

```
=====  
Region : QueyrasPark  
Comparison : 1
```

```
=====
=====
Date A : 01Aug2018_07Aug2018
Date B : 13Aug2018_19Aug2018
```

```
Coherence Change : -0.0261
Phase Change      : 0.5088
GT Area Change    : 178
Pred Area Change  : 182
Dice Change       : 0.9326
IoU Change        : 0.8737
F1 Change         : 0.9326
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation increased.
Observation: Segmentation performance improved.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : QueyrasPark
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_13Aug2018
Date B : 19Aug2018_25Aug2018
```

```
Coherence Change : 0.0003
Phase Change      : 1.0998
GT Area Change    : 77
Pred Area Change  : -135
Dice Change       : -0.8797
IoU Change        : -0.7852
F1 Change         : -0.8797
```

```
Observation: Ground remained relatively stable.
Observation: Surface deformation increased.
Observation: Segmentation performance degraded.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : Vanoise
Comparison : 1
=====
=====
```

=====
Date A : 01Aug2018_07Aug2018
Date B : 13Aug2018_19Aug2018

Coherence Change : 0.0490
Phase Change : -0.4928
GT Area Change : 0
Pred Area Change : 0
Dice Change : 0.0000
IoU Change : 0.0000
F1 Change : 0.0000

Observation: Ground remained relatively stable.
Observation: Surface deformation reduced.
Observation: Segmentation performance degraded.

=====
=====
Region : Vanoise
Comparison : 2
=====

=====
Date A : 07Aug2018_13Aug2018
Date B : 19Aug2018_25Aug2018

Coherence Change : -0.0003
Phase Change : 0.7142
GT Area Change : 103
Pred Area Change : 168
Dice Change : 0.0691
IoU Change : 0.1085
F1 Change : 0.0691

Observation: Ground stability decreased over time.
Observation: Surface deformation increased.
Observation: Segmentation performance improved.
Observation: Landslide extent changed over time.

12 DAYS ANALYSIS

cell 28A -> Find Two-Month Date Pairs

```
import os

regions = [
    "EcrinPark",
    "OutsideTheBox",
    "QueyrasPark",
    "Vanoise"
]

selected_pairs_12 = {}

for region in regions:
    date_folders = []
    for sample in dataset.samples:
        temporal, reg, coh_file, _, _ = sample
        if (
            temporal == "12days"
            and
            reg == region
        ):
            folder_name = os.path.basename(
                os.path.dirname(
                    os.path.dirname(
                        coh_file
                    )
                )
            )
```

```

        date_folders.append(
            folder_name
        )
date_folders = sorted(
    list(
        set(date_folders)
    )
)
if len(date_folders) < 4:
    continue
pair1 = (
    date_folders[0],
    date_folders[8]
)
pair2 = (
    date_folders[4],
    date_folders[12]
)
selected_pairs_12[region] = [
    pair1,
    pair2
]
print()
print("="*80)
print(region)
print("="*80)
print(

```

```

        "Pair 1 :",
        pair1[0],
        "vs",
        pair1[1]
    )
    print(
        "Pair 2 :",
        pair2[0],
        "vs",
        pair2[1]
    )

```

```

=====
=====
EcrinPark
=====
=====
Pair 1 : 01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018
Pair 2 : 07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018
=====
=====
OutsideTheBox
=====
=====
Pair 1 : 01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018
Pair 2 : 07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018
=====
=====
QueyrasPark
=====
=====
Pair 1 : 01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018
Pair 2 : 07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018
=====
=====
Vanoise
=====
=====

```

=====

Pair 1 : 01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018

Pair 2 : 07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018

cell 28B - Visual Comparison (8 Comparisons / 16 Images)

```
import matplotlib.pyplot as plt
import tiffio as tiff
import numpy as np
import torch

temporal_results = []

for region in regions:
    if region not in selected_pairs_12:
        continue

    print("\n")
    print("="*100)
    print(region)
    print("="*100)

    for pair_id, pair in enumerate(
        selected_pairs_12[region]
    ):
        print()
        print(
            f"Comparison {pair_id+1}"
        )
        print(
            pair[0],
            "vs",
            pair[1]
        )

        selected_images = []

        for target_folder in pair:
            found = False
```

```

        for sample in dataset.samples:
            temporal, reg, coh_file, phase_file, mask_file =
sample
                folder_name = os.path.basename(
                    os.path.dirname(
                        os.path.dirname(
                            coh_file
                        )
                    )
                )
                if (
                    temporal == "12days"
                    and
                    reg == region
                    and
                    folder_name == target_folder
                ):
                    selected_images.append(
                        sample
                    )
                    found = True
                    break
            if not found:
                selected_images.append(
                    None
                )
    fig, ax = plt.subplots(
        2,

```



```

    5,
    figsize=(18,8)
)
for row, sample in enumerate(
    selected_images
):
    if sample is None:
        continue

    temporal, reg, coh_file, phase_file, mask_file = sample
    coherence = tiff.imread(
        coh_file
    ).astype(np.float32)
    phase = tiff.imread(
        phase_file
    ).astype(np.float32)
    gt_mask = tiff.imread(
        mask_file
    ).astype(np.float32)
    cos_phase = np.cos(
        phase
    )
    sin_phase = np.sin(
        phase
    )
    image = np.stack(
        [

```

```

        coherence,
        cos_phase,
        sin_phase
    ],
    axis=0
)
for c in range(
    image.shape[0]
):
    image[c] = (
        image[c]
        -
        image[c].mean()
    ) / (
        image[c].std()
        +
        1e-8
    )
tensor = torch.tensor(
    image,
    dtype=torch.float32
).unsqueeze(0).to(device)
with torch.no_grad():
    output = model(
        tensor
    )
    prob = torch.sigmoid(

```

```

        output
    )
pred = (
    prob > 0.5
).float()
pred_np = pred.squeeze().cpu().numpy()
heatmap = prob.squeeze().cpu().numpy()

ax[row][0].imshow(
    coherence,
    cmap="gray"
)

ax[row][0].set_title(
    "Input"
)

ax[row][1].imshow(
    gt_mask,
    cmap="gray"
)

ax[row][1].set_title(
    "GT"
)

ax[row][2].imshow(
    pred_np,
    cmap="gray"
)

ax[row][2].set_title(
    "Prediction"
)

ax[row][3].imshow(
    heatmap,
    cmap="jet"
)

ax[row][3].set_title(
    "Heatmap"
)

ax[row][4].imshow(
    coherence,

```

```

        cmap="gray"
    )

    ax[row][4].imshow(

        pred_np,

        cmap="jet",

        alpha=0.5

    )

    ax[row][4].set_title(
        "Overlay"
    )

plt.show()

```

=====

=====

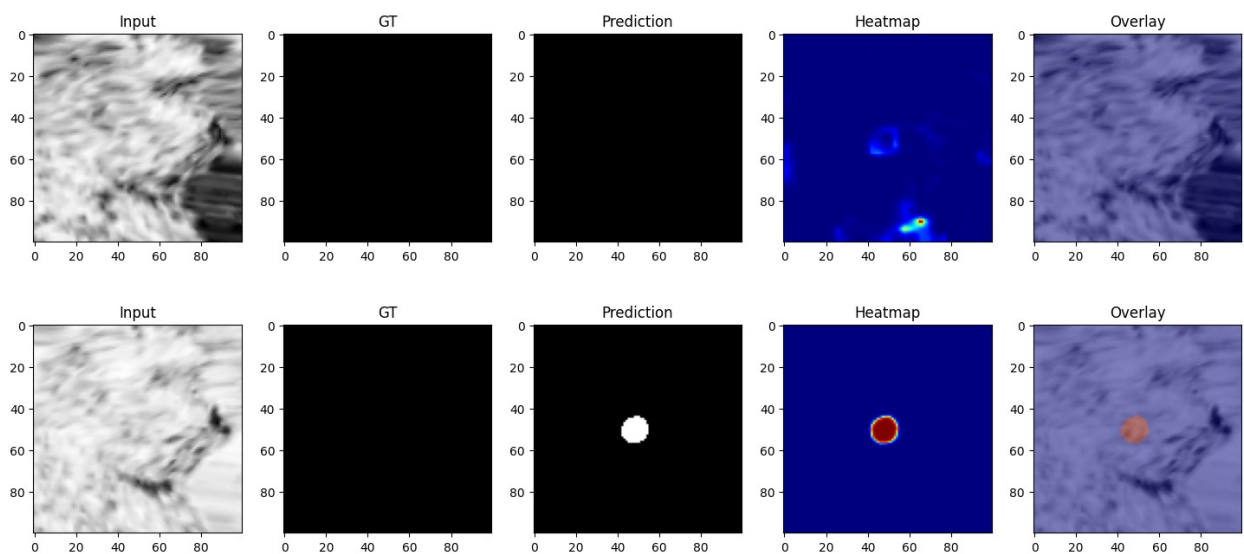
EcrinPark

=====

=====

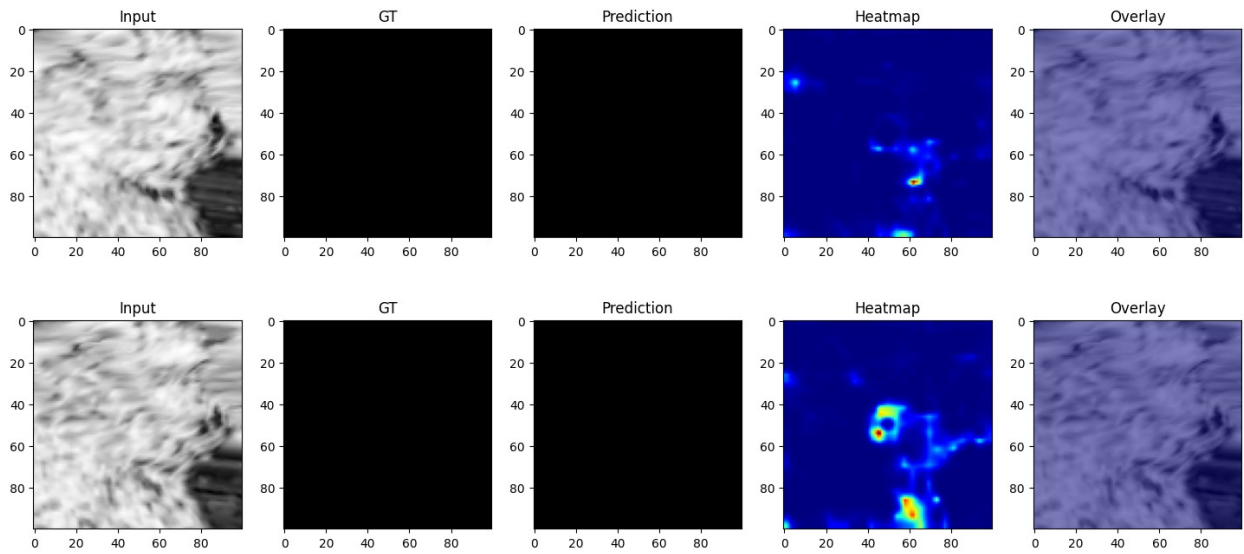
Comparison 1

01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018



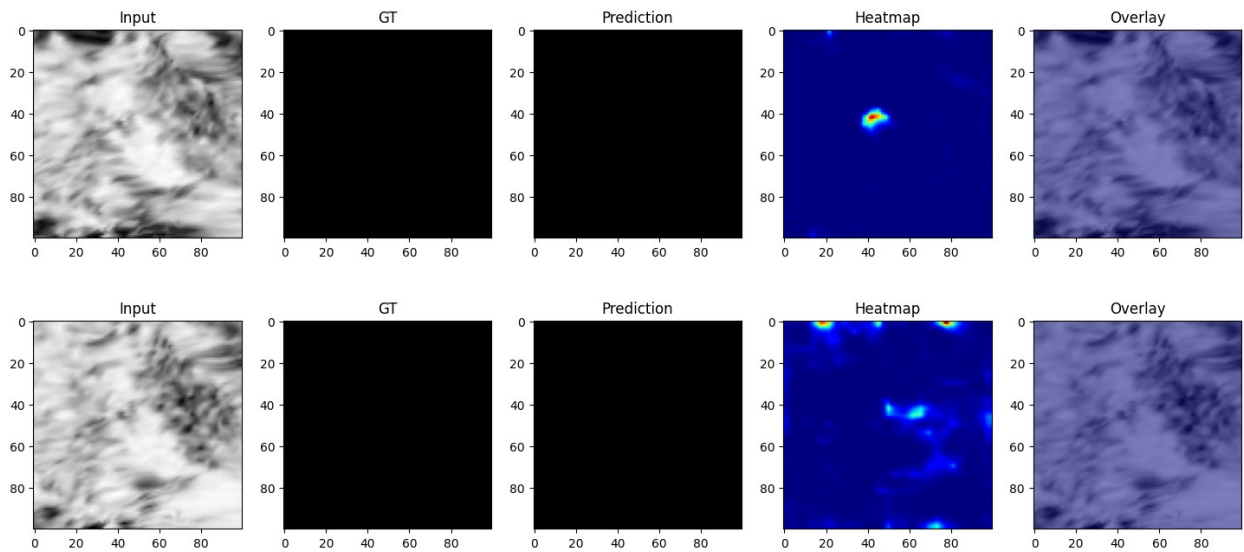
Comparison 2

07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018

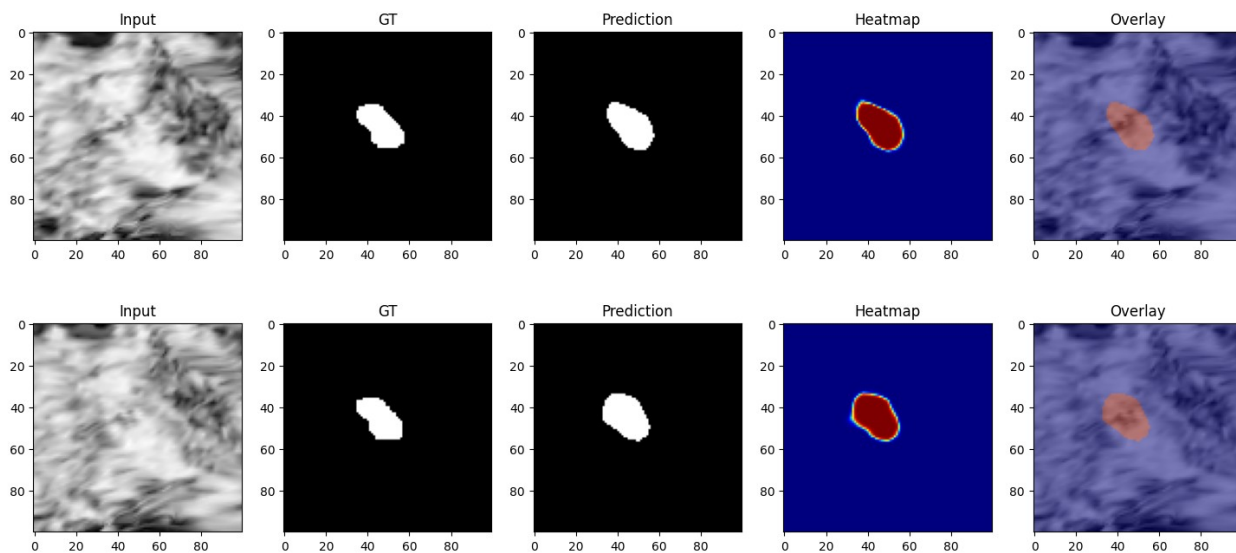


=====
 =====
 OutsideTheBox
 =====
 =====

Comparison 1
 01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018



Comparison 2
 07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018



=====

=====

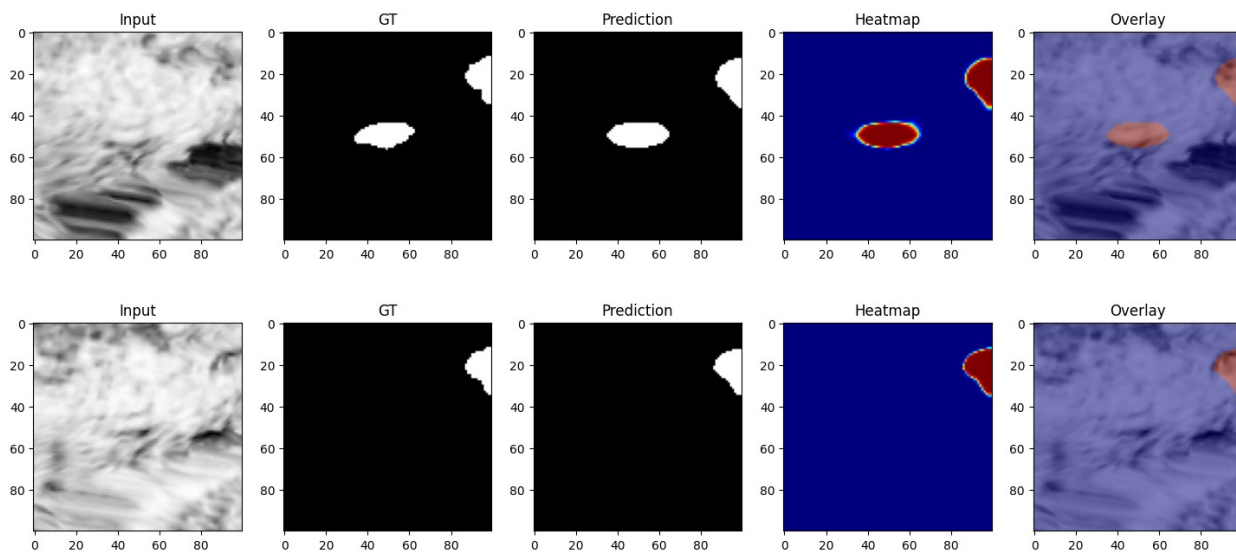
QueyrasPark

=====

=====

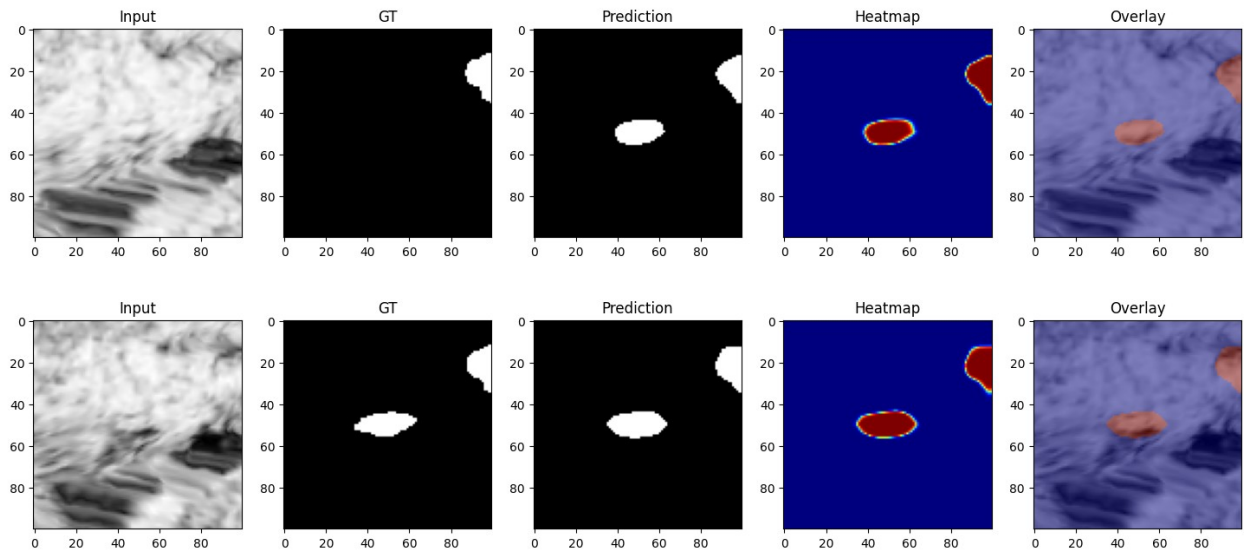
Comparison 1

01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018



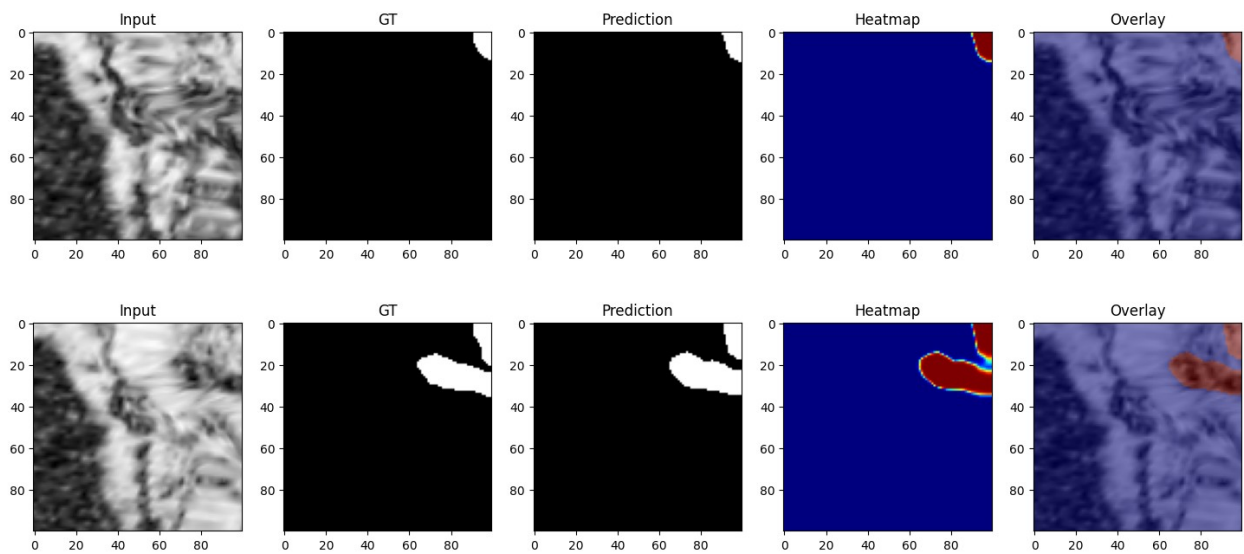
Comparison 2

07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018

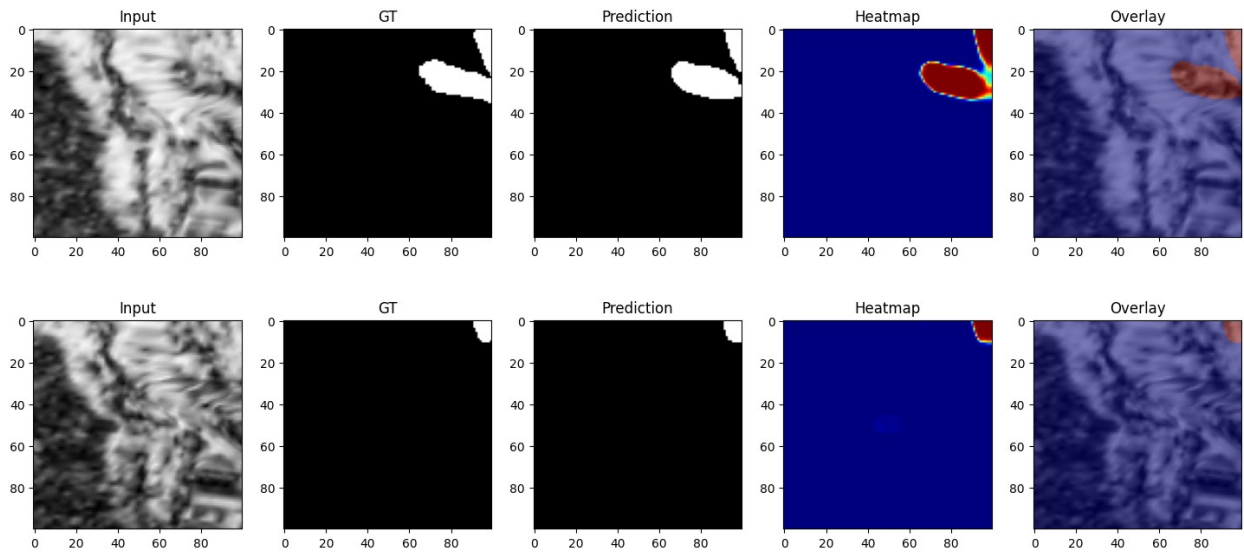


Vanoise

Comparison 1
01Aug2018_13Aug2018 vs 13Aug2018_25Aug2018



Comparison 2
07Aug2018_19Aug2018 vs 20Jul2018_01Aug2018



cell 28C - Two-Month Temporal Evolution Metrics Table

```
import pandas as pd
import numpy as np
import tifffile as tiff
import torch
import os

temporal_analysis = []

for region in regions:
    if region not in selected_pairs_12:
        continue
    for pair_id, pair in enumerate(
        selected_pairs_12[region]
    ):
        pair_metrics = []
        for target_folder in pair:
            sample_found = None
            for sample in dataset.samples:
```



```

temporal, reg, coh_file, phase_file, mask_file =
sample
    folder_name = os.path.basename(
        os.path.dirname(
            os.path.dirname(
                coh_file
            )
        )
    )
    if (
        temporal == "12days"
        and
        reg == region
        and
        folder_name == target_folder
    ):
        sample_found = sample
        break
    if sample_found is None:
        continue
temporal_, reg_, coh_file, phase_file, mask_file =
sample_found
    coherence = tiff.imread(
        coh_file
    ).astype(np.float32)
    phase = tiff.imread(
        phase_file
    ).astype(np.float32)
    gt_mask = tiff.imread(
        mask_file
    ).astype(np.float32)

```

```
cos_phase = np.cos(
    phase
)

sin_phase = np.sin(
    phase
)

image = np.stack(
    [
        coherence,
        cos_phase,
        sin_phase
    ],
    axis=0
)

for c in range(
    image.shape[0]
):
    image[c] = (
        image[c]
        -
        image[c].mean()
    ) / (
        image[c].std()
        +
        1e-8
    )

tensor = torch.tensor(
    image,
    dtype=torch.float32
```

```

).unsqueeze(0).to(device)
with torch.no_grad():
    output = model(
        tensor
    )
    prob = torch.sigmoid(
        output
    )
pred_mask = (
    prob > 0.5
).float()
pred_np = pred_mask.squeeze().cpu().numpy()
intersection = np.sum(
    gt_mask
    *
    pred_np
)
dice = (
    2 * intersection
) / (
    gt_mask.sum()
    +
    pred_np.sum()
    +
    1e-8
)
iou = (

```

```

        intersection
    ) / (
        gt_mask.sum()
        +
        pred_np.sum()
        -
        intersection
        +
        1e-8
    )
precision = intersection / (
    pred_np.sum()
    +
    1e-8
)
recall = intersection / (
    gt_mask.sum()
    +
    1e-8
)
f1 = (
    2
    *
    precision
    *
    recall

```

```

        ) / (
            precision
            +
            recall
            +
            1e-8
        )
    pair_metrics.append(
        [
            target_folder,
            coherence.mean(),
            phase.std(),
            np.sum(gt_mask > 0.5),
            np.sum(pred_np > 0.5),
            dice,
            iou,
            f1
        ]
    )
    if len(pair_metrics) == 2:
        temporal_analysis.append(
            [
                region,
                pair_id + 1
            ]
        )
        +

```

```
        pair_metrics[0]
        +
        pair_metrics[1]
    )
columns = [
    "Region",
    "Comparison",
    "Date A",
    "Coh A",
    "Phase A",
    "GT A",
    "Pred A",
    "Dice A",
    "IoU A",
    "F1 A",
    "Date B",
    "Coh B",
    "Phase B",
    "GT B",
    "Pred B",
    "Dice B",
    "IoU B",
    "F1 B"
]
temporal_df_12 = pd.DataFrame(
    temporal_analysis,
```

```

columns=columns
)
print("="*120)
print("TWO-MONTH TEMPORAL EVOLUTION TABLE")
print("="*120)
display(temporal_df_12)

```

TWO-MONTH TEMPORAL EVOLUTION TABLE						
GT A \	Region	Comparison	Date A	Coh A	Phase A	
0	EcrinPark	1	01Aug2018_13Aug2018	0.697877	0.625040	
0	EcrinPark	2	07Aug2018_19Aug2018	0.740479	0.702691	
2	OutsideTheBox	1	01Aug2018_13Aug2018	0.710533	2.822346	
3	OutsideTheBox	2	07Aug2018_19Aug2018	0.688411	1.168025	
4	QueyrasPark	1	01Aug2018_13Aug2018	0.730476	0.548828	
5	QueyrasPark	2	07Aug2018_19Aug2018	0.734522	1.687368	
6	Vanoise	1	01Aug2018_13Aug2018	0.526599	0.857483	
7	Vanoise	2	07Aug2018_19Aug2018	0.567527	0.922688	
Pred A	Dice A	IoU A	F1 A	Date B	Coh B	
0	0	0.000000	0.000000	13Aug2018_25Aug2018	0.830859	
1	0	0.000000	0.000000	20Jul2018_01Aug2018	0.710800	
2	0	0.000000	0.000000	13Aug2018_25Aug2018	0.753579	
3	344	0.922137	0.855524	20Jul2018_01Aug2018	0.688474	
4	499	0.912355	0.838835	13Aug2018_25Aug2018	0.805797	
5	416	0.598706	0.427252	20Jul2018_01Aug2018	0.691697	
6	111	0.943396	0.892857	13Aug2018_25Aug2018	0.595521	

7 560 0.936777 0.881072 0.936776 20Jul2018_01Aug2018 0.483889

	Phase B	GT B	Pred B	Dice B	IoU B	F1 B
0	1.243134	0	134	0.000000	0.000000	0.000000
1	0.733258	0	0	0.000000	0.000000	0.000000
2	0.763512	0	0	0.000000	0.000000	0.000000
3	0.514281	300	364	0.834337	0.715762	0.834337
4	0.541916	187	202	0.946015	0.897561	0.946015
5	0.666531	429	487	0.906114	0.828343	0.906114
6	0.826448	578	602	0.940678	0.888000	0.940678
7	0.860846	83	88	0.970760	0.943182	0.970760

CELL 28D -> Automatic Temporal Change Analysis

```
for i in range(
    len(temporal_df_12)
):
    row = temporal_df_12.iloc[i]
    print("\n")
    print("="*100)
    print(
        f"Region : {row['Region']}"
    )
    print(
        f"Comparison : {row['Comparison']}"
    )
    print("="*100)
    print()
    print(
        f>Date A : {row['Date A']}"
```



```
)  
print(  
    f"Date B : {row['Date B']}"  
)  
print()  
coh_change = (  
    row["Coh B"]  
    -  
    row["Coh A"]  
)  
phase_change = (  
    row["Phase B"]  
    -  
    row["Phase A"]  
)  
dice_change = (  
    row["Dice B"]  
    -  
    row["Dice A"]  
)  
iou_change = (  
    row["IoU B"]  
    -  
    row["IoU A"]  
)  
f1_change = (  

```

```
        row["F1 B"]
        -
        row["F1 A"]
    )
gt_change = (
    row["GT B"]
    -
    row["GT A"]
)
pred_change = (
    row["Pred B"]
    -
    row["Pred A"]
)
print(
    f"Coherence Change : {coh_change:.4f}"
)
print(
    f"Phase Change      : {phase_change:.4f}"
)
print(
    f"GT Area Change    : {gt_change}"
)
print(
    f"Pred Area Change  : {pred_change}"
)
print(
    f"Dice Change       : {dice_change:.4f}"
)
print(
    f"IoU Change        : {iou_change:.4f}"
)
```

```
print(
    f"F1 Change          : {f1_change:.4f}"
)
print()
if coh_change < 0:
    print(
        "Observation: Ground stability decreased over time."
    )
else:
    print(
        "Observation: Ground remained relatively stable."
    )
if phase_change > 0:
    print(
        "Observation: Surface deformation increased."
    )
else:
    print(
        "Observation: Surface deformation reduced."
    )
if dice_change > 0:
    print(
        "Observation: Segmentation performance improved."
    )
else:
    print(
        "Observation: Segmentation performance degraded."
    )
if abs(gt_change) > 0:
    print(
        "Observation: Landslide extent changed over time."
    )
print()
```

```
=====
=====
Region : EcrinPark
Comparison : 1
=====
=====
```

```
Date A : 01Aug2018_13Aug2018
Date B : 13Aug2018_25Aug2018
```

```
Coherence Change : 0.1330
Phase Change      : 0.6181
GT Area Change    : 0
Pred Area Change  : 134
Dice Change       : 0.0000
IoU Change        : 0.0000
F1 Change         : 0.0000
```

```
Observation: Ground remained relatively stable.
Observation: Surface deformation increased.
Observation: Segmentation performance degraded.
```

```
=====
=====
Region : EcrinPark
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_19Aug2018
Date B : 20Jul2018_01Aug2018
```

```
Coherence Change : -0.0297
Phase Change      : 0.0306
GT Area Change    : 0
Pred Area Change  : 0
Dice Change       : 0.0000
IoU Change        : 0.0000
F1 Change         : 0.0000
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation increased.
Observation: Segmentation performance degraded.
```

```
=====
```

=====
Region : OutsideTheBox
Comparison : 1
=====

=====
Date A : 01Aug2018_13Aug2018
Date B : 13Aug2018_25Aug2018

Coherence Change : 0.0430
Phase Change : -2.0588
GT Area Change : 0
Pred Area Change : 0
Dice Change : 0.0000
IoU Change : 0.0000
F1 Change : 0.0000

Observation: Ground remained relatively stable.
Observation: Surface deformation reduced.
Observation: Segmentation performance degraded.

=====
Region : OutsideTheBox
Comparison : 2
=====

=====
Date A : 07Aug2018_19Aug2018
Date B : 20Jul2018_01Aug2018

Coherence Change : 0.0001
Phase Change : -0.6537
GT Area Change : -11
Pred Area Change : 20
Dice Change : -0.0878
IoU Change : -0.1398
F1 Change : -0.0878

Observation: Ground remained relatively stable.
Observation: Surface deformation reduced.
Observation: Segmentation performance degraded.
Observation: Landslide extent changed over time.

=====
Region : QueyrasPark

Comparison : 1

=====

Date A : 01Aug2018_13Aug2018

Date B : 13Aug2018_25Aug2018

Coherence Change : 0.0753

Phase Change : -0.0069

GT Area Change : -261

Pred Area Change : -297

Dice Change : 0.0337

IoU Change : 0.0587

F1 Change : 0.0337

Observation: Ground remained relatively stable.

Observation: Surface deformation reduced.

Observation: Segmentation performance improved.

Observation: Landslide extent changed over time.

=====

Region : QueyrasPark

Comparison : 2

=====

Date A : 07Aug2018_19Aug2018

Date B : 20Jul2018_01Aug2018

Coherence Change : -0.0428

Phase Change : -1.0208

GT Area Change : 227

Pred Area Change : 71

Dice Change : 0.3074

IoU Change : 0.4011

F1 Change : 0.3074

Observation: Ground stability decreased over time.

Observation: Surface deformation reduced.

Observation: Segmentation performance improved.

Observation: Landslide extent changed over time.

=====

Region : Vanoise

Comparison : 1

```
=====
=====
Date A : 01Aug2018_13Aug2018
Date B : 13Aug2018_25Aug2018
```

```
Coherence Change : 0.0689
Phase Change      : -0.0310
GT Area Change    : 477
Pred Area Change  : 491
Dice Change       : -0.0027
IoU Change        : -0.0049
F1 Change         : -0.0027
```

```
Observation: Ground remained relatively stable.
Observation: Surface deformation reduced.
Observation: Segmentation performance degraded.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : Vanoise
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_19Aug2018
Date B : 20Jul2018_01Aug2018
```

```
Coherence Change : -0.0836
Phase Change      : -0.0618
GT Area Change    : -480
Pred Area Change  : -472
Dice Change       : 0.0340
IoU Change        : 0.0621
F1 Change         : 0.0340
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation reduced.
Observation: Segmentation performance improved.
Observation: Landslide extent changed over time.
```

18 DAYS ANALYSIS

CELL 29A -> Find Two-Month Date Pairs

```
import os

regions = [
    "EcrinPark",
    "OutsideTheBox",
    "QueyrasPark",
    "Vanoise"
]

selected_pairs_18 = {}

for region in regions:
    date_folders = []
    for sample in dataset.samples:
        temporal, reg, coh_file, _, _ = sample
        if (
            temporal == "18days"
            and
            reg == region
        ):
            folder_name = os.path.basename(
                os.path.dirname(
                    os.path.dirname(
                        coh_file
                    )
                )
            )
```



```

        date_folders.append(
            folder_name
        )
date_folders = sorted(
    list(
        set(date_folders)
    )
)
if len(date_folders) < 4:
    continue
pair1 = (
    date_folders[0],
    date_folders[8]
)
pair2 = (
    date_folders[4],
    date_folders[12]
)
selected_pairs_18[region] = [
    pair1,
    pair2
]
print()
print("="*80)
print(region)
print("="*80)
print(

```

```

        "Pair 1 :",
        pair1[0],
        "vs",
        pair1[1]
    )
    print(
        "Pair 2 :",
        pair2[0],
        "vs",
        pair2[1]
    )

```

```

=====
=====
EcrinPark
=====
=====
Pair 1 : 01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018
Pair 2 : 07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018
=====
=====
OutsideTheBox
=====
=====
Pair 1 : 01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018
Pair 2 : 07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018
=====
=====
QueyrasPark
=====
=====
Pair 1 : 01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018
Pair 2 : 07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018
=====
=====
Vanoise
=====
=====

```

=====

Pair 1 : 01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018

Pair 2 : 07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018

CELL 29B -> Visual Comparison (8 Comparisons / 16 Images)

```
import matplotlib.pyplot as plt
import tiffio as tiff
import numpy as np
import torch

temporal_results = []

for region in regions:
    if region not in selected_pairs_18:
        continue

    print("\n")
    print("="*100)
    print(region)
    print("="*100)

    for pair_id, pair in enumerate(
        selected_pairs_18[region]
    ):
        print()
        print(
            f"Comparison {pair_id+1}"
        )
        print(
            pair[0],
            "vs",
            pair[1]
        )

        selected_images = []

        for target_folder in pair:
            found = False
```

```

        for sample in dataset.samples:
            temporal, reg, coh_file, phase_file, mask_file =
sample
                folder_name = os.path.basename(
                    os.path.dirname(
                        os.path.dirname(
                            coh_file
                        )
                    )
                )
            if (
                temporal == "18days"
                and
                reg == region
                and
                folder_name == target_folder
            ):
                selected_images.append(
                    sample
                )
                found = True
                break
            if not found:
                selected_images.append(
                    None
                )
    fig, ax = plt.subplots(
        2,

```

```

    5,
    figsize=(18,8)
)
for row, sample in enumerate(
    selected_images
):
    if sample is None:
        continue

    temporal, reg, coh_file, phase_file, mask_file = sample
    coherence = tiff.imread(
        coh_file
    ).astype(np.float32)
    phase = tiff.imread(
        phase_file
    ).astype(np.float32)
    gt_mask = tiff.imread(
        mask_file
    ).astype(np.float32)
    cos_phase = np.cos(
        phase
    )
    sin_phase = np.sin(
        phase
    )
    image = np.stack(
        [

```

```

        coherence,
        cos_phase,
        sin_phase
    ],
    axis=0
)
for c in range(
    image.shape[0]
):
    image[c] = (
        image[c]
        -
        image[c].mean()
    ) / (
        image[c].std()
        +
        1e-8
    )
tensor = torch.tensor(
    image,
    dtype=torch.float32
).unsqueeze(0).to(device)
with torch.no_grad():
    output = model(
        tensor
    )
    prob = torch.sigmoid(

```

```

        output
    )
pred = (
    prob > 0.5
).float()
pred_np = pred.squeeze().cpu().numpy()
heatmap = prob.squeeze().cpu().numpy()

ax[row][0].imshow(
    coherence,
    cmap="gray"
)

ax[row][0].set_title(
    "Input"
)

ax[row][1].imshow(
    gt_mask,
    cmap="gray"
)

ax[row][1].set_title(
    "GT"
)

ax[row][2].imshow(
    pred_np,
    cmap="gray"
)

ax[row][2].set_title(
    "Prediction"
)

ax[row][3].imshow(
    heatmap,
    cmap="jet"
)

ax[row][3].set_title(
    "Heatmap"
)

ax[row][4].imshow(
    coherence,

```

```

        cmap="gray"
    )

    ax[row][4].imshow(
        pred_np,
        cmap="jet",
        alpha=0.5
    )

    ax[row][4].set_title(
        "Overlay"
    )

plt.show()

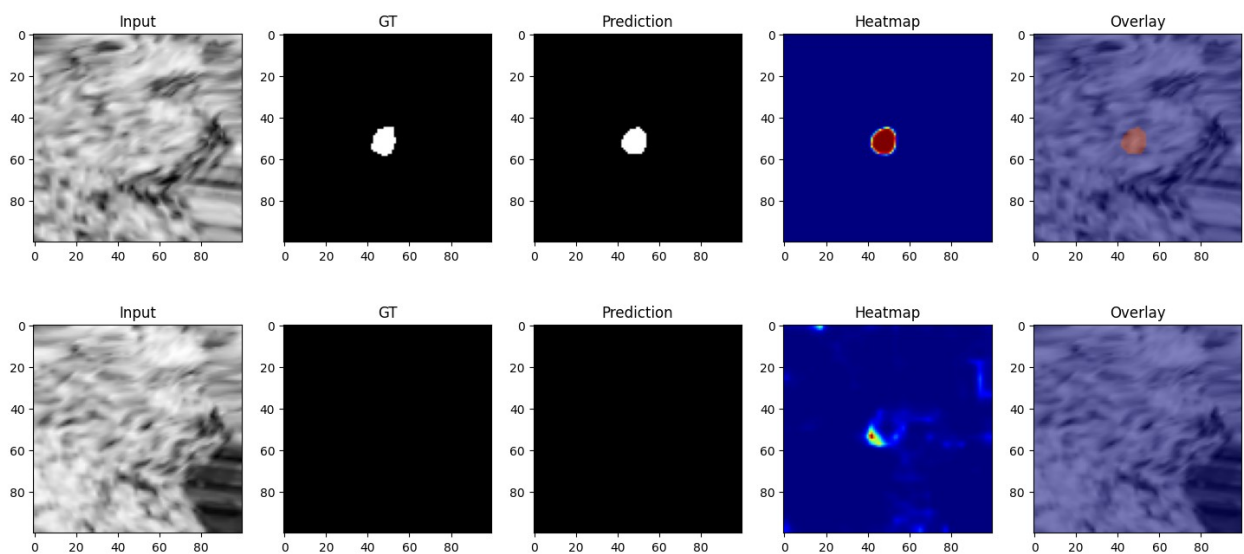
```

```

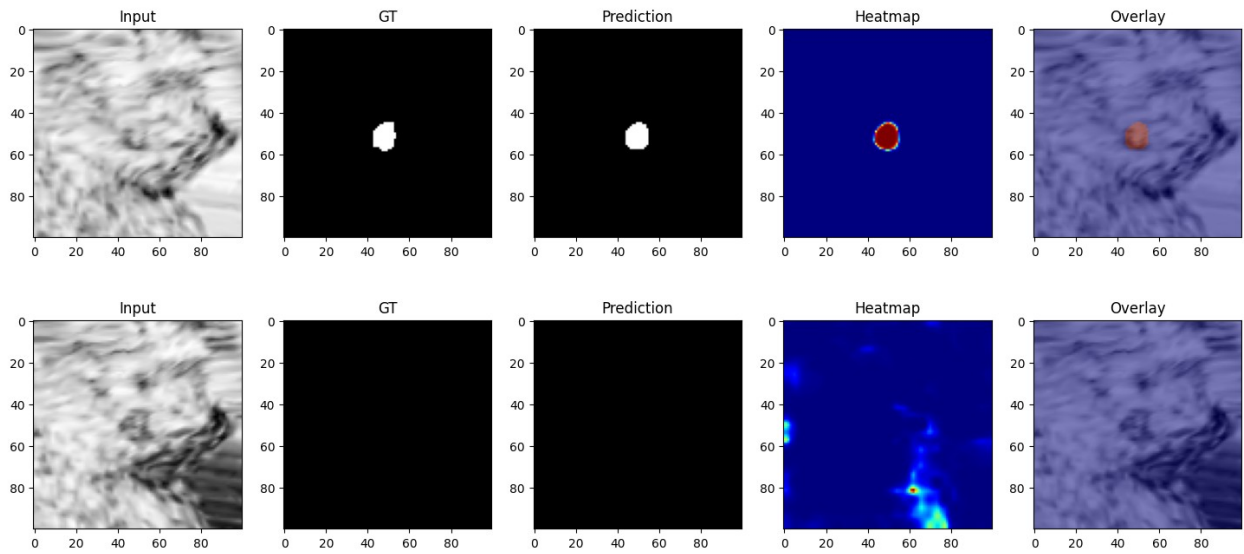
=====
=====
EcrinPark
=====
=====

```

Comparison 1
01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018

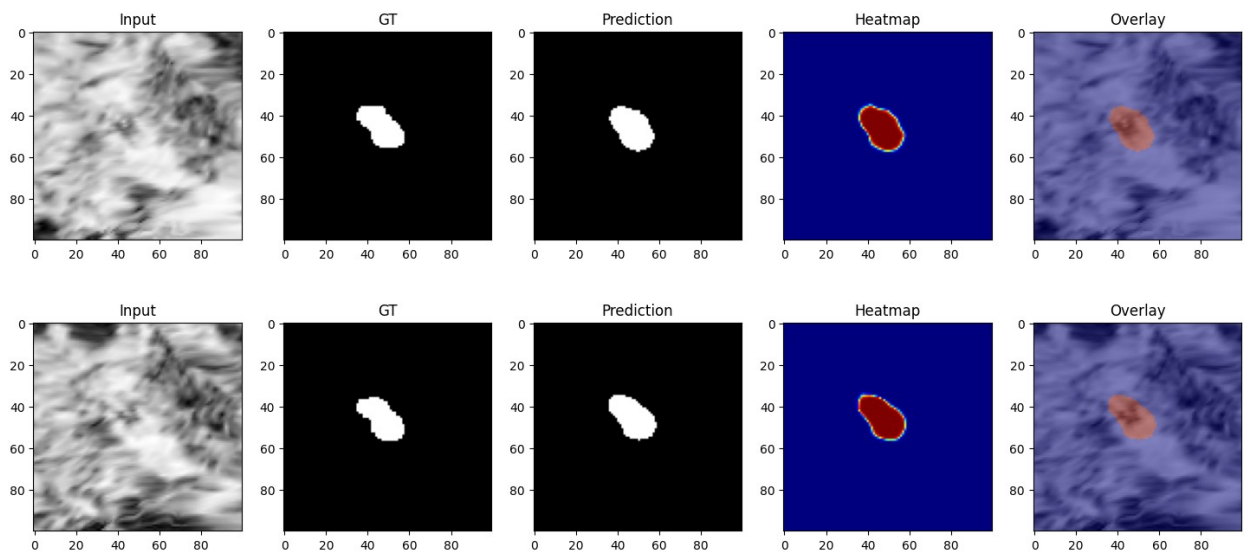


Comparison 2
07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018

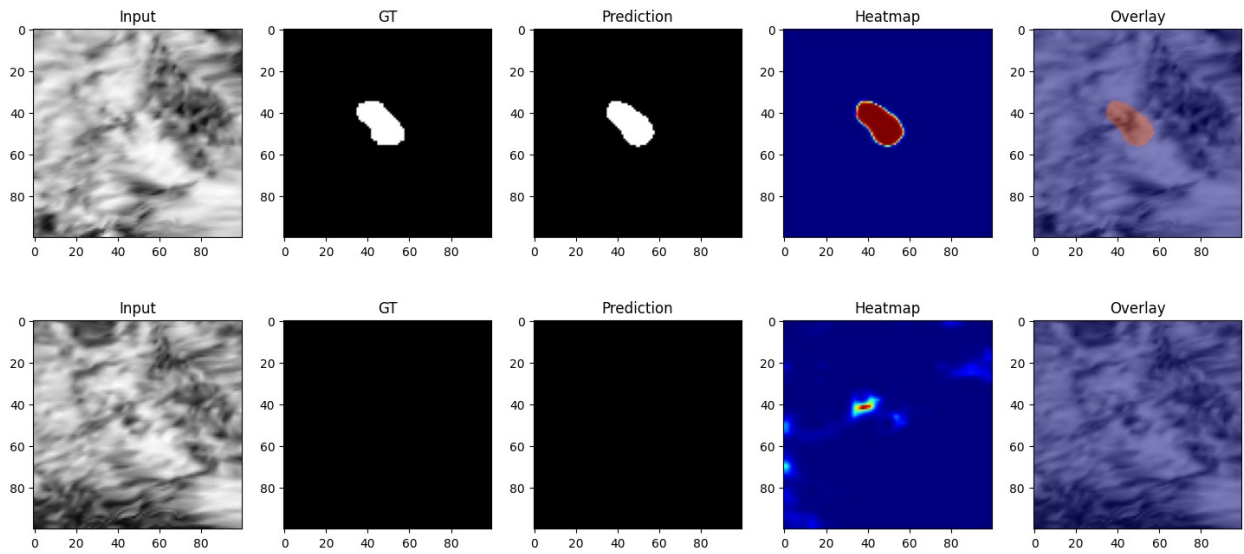


=====
 =====
 OutsideTheBox
 =====
 =====

Comparison 1
 01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018



Comparison 2
 07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018



=====

=====

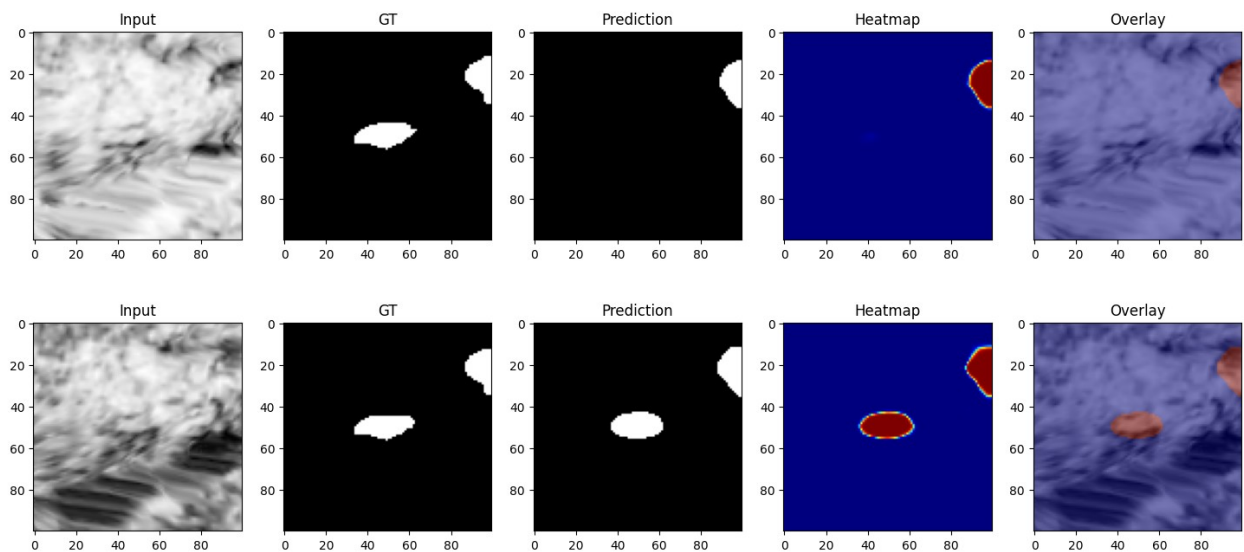
QueyrasPark

=====

=====

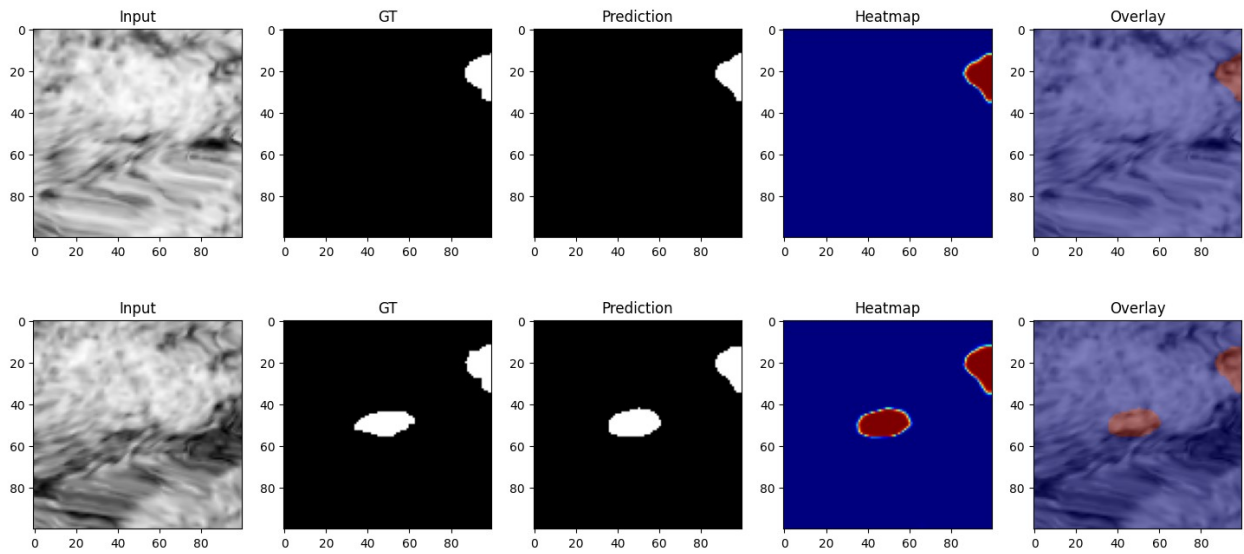
Comparison 1

01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018



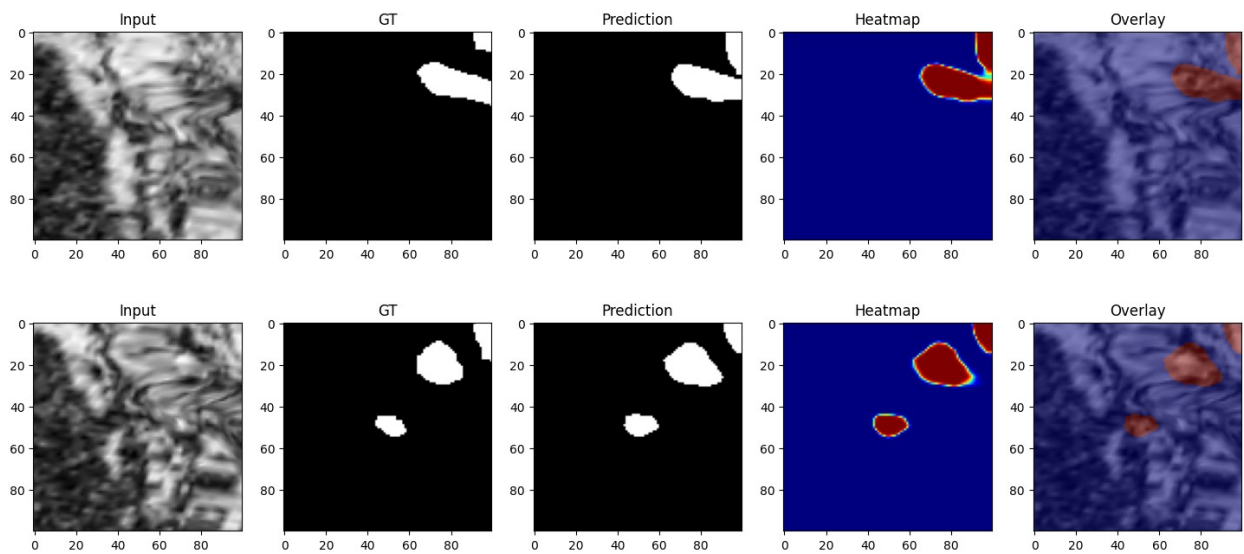
Comparison 2

07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018

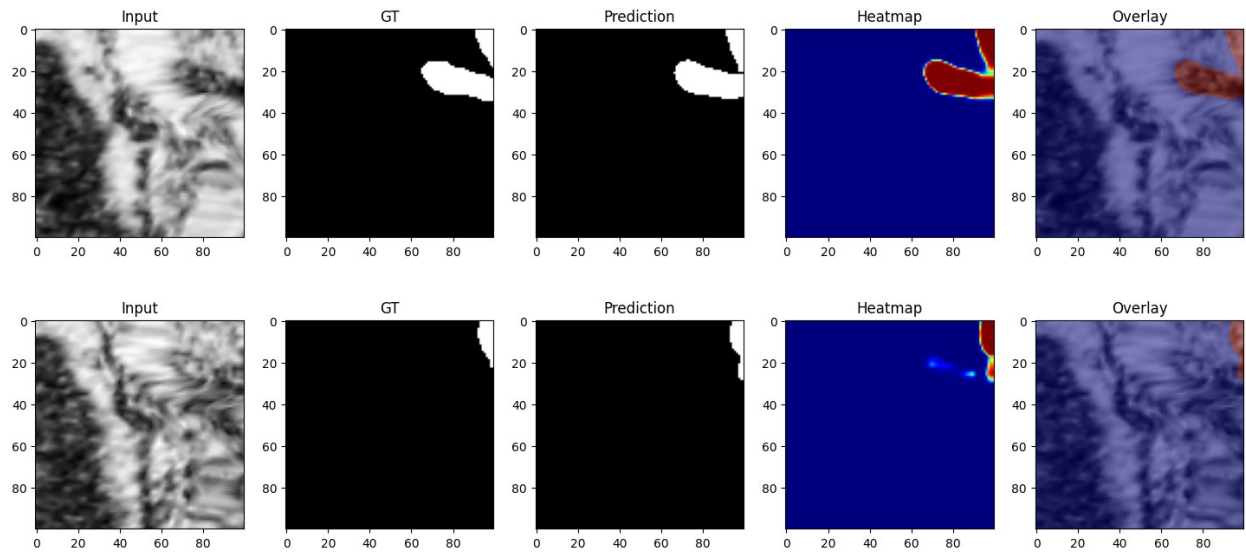


Vanoise

Comparison 1
01Aug2018_19Aug2018 vs 14Jul2018_01Aug2018



Comparison 2
07Aug2018_25Aug2018 vs 24Sep2018_12Oct2018



cell 29C - Two-Month Temporal Evolution Metrics Table

```
import pandas as pd
import numpy as np
import tifffile as tiff
import torch
import os

temporal_analysis = []

for region in regions:
    if region not in selected_pairs_18:
        continue

    for pair_id, pair in enumerate(
        selected_pairs_18[region]
    ):
        pair_metrics = []

        for target_folder in pair:
            sample_found = None

            for sample in dataset.samples:
```

```

temporal, reg, coh_file, phase_file, mask_file =
sample
    folder_name = os.path.basename(
        os.path.dirname(
            os.path.dirname(
                coh_file
            )
        )
    )
    if (
        temporal == "18days"
        and
        reg == region
        and
        folder_name == target_folder
    ):
        sample_found = sample
        break
    if sample_found is None:
        continue
temporal_, reg_, coh_file, phase_file, mask_file =
sample_found
    coherence = tiff.imread(
        coh_file
    ).astype(np.float32)
    phase = tiff.imread(
        phase_file
    ).astype(np.float32)
    gt_mask = tiff.imread(
        mask_file
    ).astype(np.float32)

```

```
cos_phase = np.cos(
    phase
)

sin_phase = np.sin(
    phase
)

image = np.stack(
    [
        coherence,
        cos_phase,
        sin_phase
    ],
    axis=0
)

for c in range(
    image.shape[0]
):
    image[c] = (
        image[c]
        -
        image[c].mean()
    ) / (
        image[c].std()
        +
        1e-8
    )

tensor = torch.tensor(
    image,
    dtype=torch.float32
```

```

).unsqueeze(0).to(device)
with torch.no_grad():
    output = model(
        tensor
    )
    prob = torch.sigmoid(
        output
    )
pred_mask = (
    prob > 0.5
).float()
pred_np = pred_mask.squeeze().cpu().numpy()
intersection = np.sum(
    gt_mask
    *
    pred_np
)
dice = (
    2 * intersection
) / (
    gt_mask.sum()
    +
    pred_np.sum()
    +
    1e-8
)
iou = (

```

```

        intersection
    ) / (
        gt_mask.sum()
        +
        pred_np.sum()
        -
        intersection
        +
        1e-8
    )
precision = intersection / (
    pred_np.sum()
    +
    1e-8
)
recall = intersection / (
    gt_mask.sum()
    +
    1e-8
)
f1 = (
    2
    *
    precision
    *
    recall

```



```

        ) / (
            precision
            +
            recall
            +
            1e-8
        )
    pair_metrics.append(
        [
            target_folder,
            coherence.mean(),
            phase.std(),
            np.sum(gt_mask > 0.5),
            np.sum(pred_np > 0.5),
            dice,
            iou,
            f1
        ]
    )
    if len(pair_metrics) == 2:
        temporal_analysis.append(
            [
                region,
                pair_id + 1
            ]
        )
        +

```

```
        pair_metrics[0]
        +
        pair_metrics[1]
    )
columns = [
    "Region",
    "Comparison",
    "Date A",
    "Coh A",
    "Phase A",
    "GT A",
    "Pred A",
    "Dice A",
    "IoU A",
    "F1 A",
    "Date B",
    "Coh B",
    "Phase B",
    "GT B",
    "Pred B",
    "Dice B",
    "IoU B",
    "F1 B"
]
temporal_df_18 = pd.DataFrame(
    temporal_analysis,
```

```

columns=columns
)
print("="*120)
print("TWO-MONTH TEMPORAL EVOLUTION TABLE")
print("="*120)
display(temporal_df_18)

```

=====						
=====						
TWO-MONTH TEMPORAL EVOLUTION TABLE						
=====						
=====						
GT A \	Region	Comparison		Date A		Phase A
0	EcrinPark	1	01Aug2018_19Aug2018	0.677497	2.355025	
125						
1	EcrinPark	2	07Aug2018_25Aug2018	0.774613	0.475587	
121						
2	OutsideTheBox	1	01Aug2018_19Aug2018	0.727066	0.389884	
302						
3	OutsideTheBox	2	07Aug2018_25Aug2018	0.686001	2.741142	
311						
4	QueyrasPark	1	01Aug2018_19Aug2018	0.809014	1.720627	
452						
5	QueyrasPark	2	07Aug2018_25Aug2018	0.729411	0.528741	
199						
6	Vanoise	1	01Aug2018_19Aug2018	0.493347	1.532011	
519						
7	Vanoise	2	07Aug2018_25Aug2018	0.579753	1.416686	
546						
\	Pred A	Dice A	IoU A	F1 A	Date B	Coh B
0	122	0.931174	0.871212	0.931174	14Jul2018_01Aug2018	0.667541
1	121	0.859504	0.753623	0.859504	24Sep2018_12Oct2018	0.686153
2	317	0.904685	0.825959	0.904685	14Jul2018_01Aug2018	0.648624
3	302	0.946166	0.897833	0.946166	24Sep2018_12Oct2018	0.573734
4	183	0.485039	0.320166	0.485039	14Jul2018_01Aug2018	0.613610
5	187	0.958549	0.920398	0.958549	24Sep2018_12Oct2018	0.622439
6	572	0.883593	0.791461	0.883593	14Jul2018_01Aug2018	0.429897

7	560	0.947559	0.900344	0.947559	24Sep2018_12Oct2018	0.509599
---	-----	----------	----------	----------	---------------------	----------

	Phase B	GT B	Pred B	Dice B	IoU B	F1 B
0	0.752824	0	0	0.000000	0.000000	0.000000
1	2.621217	0	0	0.000000	0.000000	0.000000
2	0.656318	304	323	0.909091	0.833333	0.909091
3	1.102866	0	0	0.000000	0.000000	0.000000
4	1.087760	437	464	0.894562	0.809237	0.894562
5	0.655298	445	464	0.924092	0.858896	0.924092
6	1.024584	571	641	0.881188	0.787611	0.881188
7	1.340660	131	135	0.872180	0.773333	0.872180

CELL 29D -> Automatic Temporal Change Analysis

```
for i in range(
    len(temporal_df_18)
):
    row = temporal_df_18.iloc[i]
    print("\n")
    print("="*100)
    print(
        f"Region : {row['Region']}"
    )
    print(
        f"Comparison : {row['Comparison']}"
    )
    print("="*100)
    print()
    print(
        f>Date A : {row['Date A']}"
```

```
)  
print(  
    f"Date B : {row['Date B']}"  
)  
print()  
coh_change = (  
    row["Coh B"]  
    -  
    row["Coh A"]  
)  
phase_change = (  
    row["Phase B"]  
    -  
    row["Phase A"]  
)  
dice_change = (  
    row["Dice B"]  
    -  
    row["Dice A"]  
)  
iou_change = (  
    row["IoU B"]  
    -  
    row["IoU A"]  
)  
f1_change = (  

```

```
        row["F1 B"]
        -
        row["F1 A"]
    )
gt_change = (
    row["GT B"]
    -
    row["GT A"]
)
pred_change = (
    row["Pred B"]
    -
    row["Pred A"]
)
print(
    f"Coherence Change : {coh_change:.4f}"
)
print(
    f"Phase Change      : {phase_change:.4f}"
)
print(
    f"GT Area Change    : {gt_change}"
)
print(
    f"Pred Area Change   : {pred_change}"
)
print(
    f"Dice Change        : {dice_change:.4f}"
)
print(
    f"IoU Change         : {iou_change:.4f}"
)
```

```
print(
    f"F1 Change          : {f1_change:.4f}"
)
print()
if coh_change < 0:
    print(
        "Observation: Ground stability decreased over time."
    )
else:
    print(
        "Observation: Ground remained relatively stable."
    )
if phase_change > 0:
    print(
        "Observation: Surface deformation increased."
    )
else:
    print(
        "Observation: Surface deformation reduced."
    )
if dice_change > 0:
    print(
        "Observation: Segmentation performance improved."
    )
else:
    print(
        "Observation: Segmentation performance degraded."
    )
if abs(gt_change) > 0:
    print(
        "Observation: Landslide extent changed over time."
    )
print()
```

```
=====
=====
Region : EcrinPark
Comparison : 1
=====
=====
```

```
Date A : 01Aug2018_19Aug2018
Date B : 14Jul2018_01Aug2018
```

```
Coherence Change : -0.0100
Phase Change      : -1.6022
GT Area Change    : -125
Pred Area Change  : -122
Dice Change       : -0.9312
IoU Change        : -0.8712
F1 Change         : -0.9312
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation reduced.
Observation: Segmentation performance degraded.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : EcrinPark
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_25Aug2018
Date B : 24Sep2018_12Oct2018
```

```
Coherence Change : -0.0885
Phase Change      : 2.1456
GT Area Change    : -121
Pred Area Change  : -121
Dice Change       : -0.8595
IoU Change        : -0.7536
F1 Change         : -0.8595
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation increased.
Observation: Segmentation performance degraded.
Observation: Landslide extent changed over time.
```



```
=====
=====
Region : OutsideTheBox
Comparison : 1
=====
=====
```

```
Date A : 01Aug2018_19Aug2018
Date B : 14Jul2018_01Aug2018
```

```
Coherence Change : -0.0784
Phase Change      : 0.2664
GT Area Change    : 2
Pred Area Change  : 6
Dice Change       : 0.0044
IoU Change        : 0.0074
F1 Change         : 0.0044
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation increased.
Observation: Segmentation performance improved.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : OutsideTheBox
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_25Aug2018
Date B : 24Sep2018_12Oct2018
```

```
Coherence Change : -0.1123
Phase Change      : -1.6383
GT Area Change    : -311
Pred Area Change  : -302
Dice Change       : -0.9462
IoU Change        : -0.8978
F1 Change         : -0.9462
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation reduced.
Observation: Segmentation performance degraded.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : QueyrasPark
Comparison : 1
=====
=====
```

```
Date A : 01Aug2018_19Aug2018
Date B : 14Jul2018_01Aug2018
```

```
Coherence Change : -0.1954
Phase Change      : -0.6329
GT Area Change    : -15
Pred Area Change  : 281
Dice Change       : 0.4095
IoU Change        : 0.4891
F1 Change         : 0.4095
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation reduced.
Observation: Segmentation performance improved.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : QueyrasPark
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_25Aug2018
Date B : 24Sep2018_12Oct2018
```

```
Coherence Change : -0.1070
Phase Change      : 0.1266
GT Area Change    : 246
Pred Area Change  : 277
Dice Change       : -0.0345
IoU Change        : -0.0615
F1 Change         : -0.0345
```

```
Observation: Ground stability decreased over time.
Observation: Surface deformation increased.
Observation: Segmentation performance degraded.
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : Vanoise
```

```
Comparison : 1
=====
=====
```

```
Date A : 01Aug2018_19Aug2018
```

```
Date B : 14Jul2018_01Aug2018
```

```
Coherence Change : -0.0634
```

```
Phase Change      : -0.5074
```

```
GT Area Change    : 52
```

```
Pred Area Change  : 69
```

```
Dice Change       : -0.0024
```

```
IoU Change        : -0.0039
```

```
F1 Change         : -0.0024
```

```
Observation: Ground stability decreased over time.
```

```
Observation: Surface deformation reduced.
```

```
Observation: Segmentation performance degraded.
```

```
Observation: Landslide extent changed over time.
```

```
=====
=====
Region : Vanoise
```

```
Comparison : 2
=====
=====
```

```
Date A : 07Aug2018_25Aug2018
```

```
Date B : 24Sep2018_12Oct2018
```

```
Coherence Change : -0.0702
```

```
Phase Change      : -0.0760
```

```
GT Area Change    : -415
```

```
Pred Area Change  : -425
```

```
Dice Change       : -0.0754
```

```
IoU Change        : -0.1270
```

```
F1 Change         : -0.0754
```

```
Observation: Ground stability decreased over time.
```

```
Observation: Surface deformation reduced.
```

```
Observation: Segmentation performance degraded.
```

```
Observation: Landslide extent changed over time.
```